

Sunday, March 29, 2026

New

```
import { useState, useEffect, useRef, useCallback, Suspense, useMemo } from "react";
import { Canvas, useFrame } from "@react-three/fiber";
import { OrbitControls, Stars, Html, Sphere } from "@react-three/drei";
import * as THREE from "three";
```

```
/* =====
SUPABASE CONFIG — replace with your project credentials
Get from: https://supabase.com → Project → Settings → API
npm install @supabase/supabase-js then uncomment the imports below
===== */
```

```
import { createClient } from '@supabase/supabase-js'
const supabase = createClient('https://YOUR_PROJECT.supabase.co', 'YOUR_ANON_KEY')
*/

// ——— LOCAL PERSISTENCE (works offline; swap save/load for Supabase calls in prod) ———
const S = {
  get: (k, d) => { try { const v = localStorage.getItem("hr2_" + k); return v ? JSON.parse(v) : d; } catch { return d; } },
  set: (k, v) => { try { localStorage.setItem("hr2_" + k, JSON.stringify(v)); } catch {} },
};
```

```
/* =====
GAME DATA
===== */
```

```
const CLASSES = ["Warrior","Wizard","Rogue","Healer","Merchant","Bard"];
const CLASS_HP = {Warrior:12,Wizard:7,Rogue:8,Healer:9,Merchant:8,Bard:8};
const CLASS_ICON_EMO = {Warrior:"⚔️",Wizard:"🧙",Rogue:"🔪",Healer:"💚",Merchant:"💰",Bard:"🎵"};
const CLASS_BASE_STATS = {
  Warrior:{STR:8,DEF:7,INT:3,WIS:4,SPD:4,CHA:3,LCK:3},
  Wizard:{STR:3,DEF:2,INT:9,WIS:7,SPD:5,CHA:4,LCK:4},
  Rogue:{STR:5,DEF:3,INT:5,WIS:4,SPD:9,CHA:6,LCK:7},
  Healer:{STR:3,DEF:4,INT:6,WIS:9,SPD:4,CHA:7,LCK:5},
  Merchant:{STR:3,DEF:3,INT:6,WIS:5,SPD:5,CHA:8,LCK:8},
  Bard:{STR:3,DEF:2,INT:5,WIS:6,SPD:6,CHA:10,LCK:5}
};
const REP = {
  Warrior: [
    { r: [1, 1], title: "The Broken Shield", passive: "-1 first combat roll/session." },
    { r: [2, 2], title: "The Deserter", passive: "Cannot call authority back. +1 solo stealth." },
    { r: [3, 3], title: "The Cursed Blade", passive: "First weapon roll/arc disadvantaged." },
    { r: [4, 4], title: "The Unfinished War", passive: "+1 intimidation. -1 social with nobility." },
    { r: [5, 7], title: "The Steadfast", passive: "Allies nearby cannot be coerced." },
  ],
};
```

- { r: [8, 9], title: "The Iron Wall", passive: "+1 DEF when protecting another character." },
- { r: [10, 11], title: "The Oathkeeper", passive: "Declare an oath once/session. +2 if kept." },
- { r: [12, 13], title: "The Last Line", passive: "+2 combat at 3 HP or below." },
- { r: [14, 14], title: "The Hollow's Bulwark", passive: "Immune to first curse each arc." },
- { r: [15, 16], title: "The Undying", passive: "If dying, roll d20. 10+ = survive at 1 HP." },
- { r: [17, 18], title: "The Warlord of Ash", passive: "Can command allies. 15+ roll they obey." },
- { r: [19, 19], title: "The Unkillable", passive: "+1 all combat. Hostiles must target others first." },
- { r: [20, 20], title: "THE HOLLOW'S CHAMPION", passive: "Once/arc, call on The Hollow for automatic success." }

],

Wizard: [

- { r: [1, 1], title: "The Silenced", passive: "First spell/session: d6, 1-2 backfire." },
- { r: [2, 2], title: "The Forbidden Scholar", passive: "+2 INT, -1 WIS. Access to restricted knowledge." },
- { r: [3, 3], title: "The Unstable", passive: "On fumble, spell affects random target." },
- { r: [4, 4], title: "The Once-Great", passive: "-1 spell rolls but +2 in academic settings." },
- { r: [5, 7], title: "The Theorist", passive: "+1 puzzle-solving and knowledge rolls." },
- { r: [8, 9], title: "The Practical Caster", passive: "Cast minor spells without a roll." },
- { r: [10, 11], title: "The Arcanist", passive: "+1 all INT. Arcane locks open without a roll." },
- { r: [12, 13], title: "The Seeker", passive: "Once/session, ask DM one question answered truthfully." },
- { r: [14, 14], title: "The Hollow's Eye", passive: "+2 detecting illusions, hidden paths, Hollow entities." },
- { r: [15, 16], title: "The Void Speaker", passive: "Once/arc, attempt communication with any entity." },
- { r: [17, 18], title: "The Architect of Ruin", passive: "+2 area-effect and destructive spells." },
- { r: [19, 19], title: "The Worldbreaker", passive: "+2 all spells. Average-INT NPCs fear them." },
- { r: [20, 20], title: "THE HOLLOW'S ORACLE", passive: "Once/arc, DM must give one true future event." }

],

Rogue: [

- { r: [1, 1], title: "The Marked", passive: "Once/session an NPC may recognise them." },
- { r: [2, 2], title: "The Burned Informant", passive: "-1 trust rolls. +1 solo stealth." },
- { r: [3, 3], title: "The Clumsy Thief", passive: "First theft/session: roll d6. 1-2 caught." },
- { r: [4, 4], title: "The Wanted", passive: "Faction is actively looking for them." },
- { r: [5, 7], title: "The Shadow", passive: "+1 stealth in urban environments." },
- { r: [8, 9], title: "The Grifter", passive: "+1 deception. NPCs believe first lie automatically." },
- { r: [10, 11], title: "The Ghost", passive: "Leave no trace unless rolling 1-3." },
- { r: [12, 13], title: "The Contractor", passive: "Once/arc, name a price for a dangerous task. NPC pays." },
- { r: [14, 14], title: "The Hollow's Shadow", passive: "+2 all stealth in Hollow locations." },
- { r: [15, 16], title: "The Phantom", passive: "Cannot be detected by mundane means in darkness." },
- { r: [17, 18], title: "The Ghost Contract", passive: "Once/arc, complete one action with no roll required." },
- { r: [19, 19], title: "The Unseen Hand", passive: "Others take blame for their actions unless witnessed." },
- { r: [20, 20], title: "THE HOLLOW'S KNIFE", passive: "Once/arc, any action succeeds. No roll. No consequence." }

],

Healer: [

- { r: [1, 1], title: "The Burnout", passive: "First heal/session costs 1 HP from themselves." },
- { r: [2, 2], title: "The Doubtful", passive: "-1 trust with strangers. +2 with those they've saved." },
- { r: [3, 3], title: "The Rattled", passive: "In combat, healing rolls -1 until they take damage." },
- { r: [4, 4], title: "The Haunted", passive: "Has lost someone. -1 in situations that mirror the loss." },
- { r: [5, 7], title: "The Field Medic", passive: "+1 stabilising downed characters." },

```

{ r: [8, 9], title: "The Trusted", passive: "NPCs in need seek them out. +1 social in distress situations." },
{ r: [10, 11], title: "The Keeper", passive: "Once/session, prevent a character from going below 1 HP." },
{ r: [12, 13], title: "The Order's Voice", passive: "Healing Order NPCs offer resources without cost." },
{ r: [14, 14], title: "The Hollow's Hands", passive: "+2 all healing in Hollow locations." },
{ r: [15, 16], title: "The Saint", passive: "Once/arc, full heal any character. No roll." },
{ r: [17, 18], title: "The Death's Edge", passive: "Can attempt revival of dead characters. 15+ to succeed." },
{ r: [19, 19], title: "The Miracle", passive: "Once/arc, any healing roll automatically succeeds at max." },
{ r: [20, 20], title: "THE HOLLOW'S GRACE", passive: "Once/arc, reverse a death that occurred this session." }

```

```

],

```

```

Merchant: [

```

```

{ r: [1, 1], title: "The Debtor", passive: "Owes someone. DM holds the debt." },
{ r: [2, 2], title: "The Fraudster", passive: "+1 deception but -2 with guilds if exposed." },
{ r: [3, 3], title: "The Undercut", passive: "First trade/session: rival appears with better offer." },
{ r: [4, 4], title: "The Blacklisted", passive: "One major faction refuses to deal with them." },
{ r: [5, 7], title: "The Traveller", passive: "+1 to finding rare goods in new locations." },
{ r: [8, 9], title: "The Negotiator", passive: "+1 all trade and negotiation rolls." },
{ r: [10, 11], title: "The Connected", passive: "In any location, knows one useful NPC." },
{ r: [12, 13], title: "The Broker", passive: "Once/session, create a deal between two NPCs." },
{ r: [14, 14], title: "The Hollow's Coin", passive: "+2 all transactions in Hollow locations." },
{ r: [15, 16], title: "The Kingmaker", passive: "Once/arc, elevate or ruin one NPC's status." },
{ r: [17, 18], title: "The Cartel", passive: "Controls prices in one location. +3 all rolls there." },
{ r: [19, 19], title: "The Exchange", passive: "Once/arc, trade any item for any item at equal value." },
{ r: [20, 20], title: "THE HOLLOW'S PRICE", passive: "Once/arc, name a price for anything. NPC pays it." }

```

```

],

```

```

Bard: [

```

```

{ r: [1, 1], title: "The Heckled", passive: "First performance/session: audience hostile on 1-3." },
{ r: [2, 2], title: "The One-Hit", passive: "-1 repeat performances with the same audience." },
{ r: [3, 3], title: "The Plagiarist", passive: "+1 copying others' styles. -2 if exposed." },
{ r: [4, 4], title: "The Forgotten", passive: "NPCs don't remember their name without prompting." },
{ r: [5, 7], title: "The Entertainer", passive: "+1 first performance in any new location." },
{ r: [8, 9], title: "The Crowd-Reader", passive: "+1 mass persuasion and performance rolls." },
{ r: [10, 11], title: "The Storyteller", passive: "Once/session, reframe a situation with a story. +2 result." },
{ r: [12, 13], title: "The Court Favourite", passive: "Noble NPCs grant access and favours." },
{ r: [14, 14], title: "The Hollow's Voice", passive: "+2 performances in Hollow locations. Hollow responds." },
{ r: [15, 16], title: "The Legend", passive: "Once/arc, performance reaches any audience regardless of distance." },
{ r: [17, 18], title: "The Memory", passive: "Stories they tell become permanent NPC memories." },
{ r: [19, 19], title: "The Epoch", passive: "Once/arc, performance changes faction attitudes permanently." },
{ r: [20, 20], title: "THE HOLLOW'S SONG", passive: "Once/arc, perform something the Hollow remembers forever." }

```

```

]

```

```

};

```

```

const SIGNATURE_SKILLS = {

```

```

  Warrior: [

```

```

    { r: [1, 2], name: "Reckless Charge", desc: "-1 DEF, +2 first attack roll each session." },
    { r: [3, 4], name: "Broken Form", desc: "Unpredictable style. +1 to surprise strikes." },

```

- { r: [5, 6], name: "Shield Wall", desc: "Allies behind them get +1 DEF while stationary." },
- { r: [7, 8], name: "Weapon Read", desc: "+1 to all rolls against same enemy after first exchange." },
- { r: [9, 10], name: "War Cry", desc: "Once/session, enemy NPC must roll to act against them next turn." },
- { r: [11, 12], name: "Controlled Aggression", desc: "+2 to first strike after spending a full turn observing." },
- { r: [13, 14], name: "Endurance Fighter", desc: "In extended conflicts, gains +1 each subsequent round." },
- { r: [15, 16], name: "Execution Strike", desc: "+3 attack against any enemy below half HP." },
- { r: [17, 18], name: "Counter", desc: "When an attack fails against them, rolls a free response strike." },
- { r: [19, 19], name: "Battlefield Commander", desc: "Once/session, every allied player gets +1 to their next roll." },
- { r: [20, 20], name: "THE HOLLOW'S BLADE", desc: "UNIQUE. On a Critical, the effect lingers into next session." }

],

Wizard: [

- { r: [1, 2], name: "Volatile Casting", desc: "Spells hit harder but 1-in-6 chance of unexpected side effect." },
- { r: [3, 4], name: "Forbidden Indexing", desc: "Can attempt any knowledge roll regardless of subject." },
- { r: [5, 6], name: "Spell Dampening", desc: "Reduce an incoming magical effect by half. Once/session." },
- { r: [7, 8], name: "Rune Reading", desc: "+2 to deciphering magical inscriptions and Hollow markings." },
- { r: [9, 10], name: "Projection", desc: "Cast a minor illusion without a roll. Once/session." },
- { r: [11, 12], name: "Arcane Diagnosis", desc: "Touch something and identify any magical effect on it. No roll." },
- { r: [13, 14], name: "Counter-Spell", desc: "When enemy spell cast, attempt to interrupt. Roll INT vs difficulty." },
- { r: [15, 16], name: "Hollow Channelling", desc: "Once/arc, cast a spell beyond normal tier." },
- { r: [17, 18], name: "Memory Casting", desc: "Replicate any spell witnessed once before. +1 to replicated spells." },
- { r: [19, 19], name: "The Eye", desc: "See through one illusion, deception, or hidden thing per session." },
- { r: [20, 20], name: "THE HOLLOW'S TONGUE", desc: "UNIQUE. Once/arc, ask the Hollow anything. DM must answer truthfully." }

],

Rogue: [

- { r: [1, 2], name: "Pickpocketing", desc: "Lift small items on SPD roll. Fail means they notice." },
- { r: [3, 4], name: "Lock Reading", desc: "Identify any lock mechanism in 30 seconds. +2 lockpicking." },
- { r: [5, 6], name: "Forgery", desc: "Produce convincing false documents. CHA roll to use them." },
- { r: [7, 8], name: "Tailing", desc: "Follow a target without detection. Roll only if actively watched." },
- { r: [9, 10], name: "Disguise", desc: "Assume different identity. +2 CHA while in disguise." },
- { r: [11, 12], name: "Poison Craft", desc: "Apply contact poison to any item. Effect negotiated with DM." },
- { r: [13, 14], name: "Escape Artist", desc: "Cannot be restrained for more than one round without SPD roll." },
- { r: [15, 16], name: "Rooftop Navigation", desc: "Full speed across vertical terrain. No penalty rolls." },
- { r: [17, 18], name: "Dead Drop Network", desc: "Once/arc, retrieve useful intel from a hidden cache." },
- { r: [19, 19], name: "Ghost Entry", desc: "Enter any non-magical location without a roll. Leaves no trace." },
- { r: [20, 20], name: "THE HOLLOW'S SHADOW", desc: "UNIQUE. Becomes functionally invisible for one full scene/arc." }

],

Healer: [

- { r: [1, 2], name: "Field Stitching", desc: "Stabilise any wound without materials. Buys two rounds." },
- { r: [3, 4], name: "Poison Identification", desc: "Identify any toxin by sight or smell. +2 treating poisoned." },
- { r: [5, 6], name: "Psychic First Aid", desc: "Treat mental trauma, fear, and psychological status effects." },
- { r: [7, 8], name: "Curse Reading", desc: "Identify nature and source of any curse on contact." },
- { r: [9, 10], name: "Pain Management", desc: "Suppress injury penalty for one session. Damage still exists." },
- { r: [11, 12], name: "Emergency Revival", desc: "Bring downed character to consciousness. One action, no roll, once/session." },
- { r: [13, 14], name: "Herbal Synthesis", desc: "Create a restorative item from available materials. Once/location." },
- { r: [15, 16], name: "Hollow Medicine", desc: "Treatments in Hollow locations have doubled effect." },

```

{ r: [17, 18], name: "Death Diagnosis", desc: "Determine cause and time of death from a body with complete accuracy." },
{ r: [19, 19], name: "Vital Read", desc: "By observation alone, know exact HP and conditions of any character." },
{ r: [20, 20], name: "THE HOLLOW'S HANDS", desc: "UNIQUE. Once/arc, full heal any character instantly." }
],
Merchant: [
{ r: [1, 2], name: "Fast Talk", desc: "Distract or confuse an NPC. Buys one round without a roll." },
{ r: [3, 4], name: "Counterfeit Detection", desc: "Identify fake currency, forged documents, or fraudulent goods instantly." },
{ r: [5, 6], name: "Bulk Negotiation", desc: "+2 group trade rolls. Always secures better combined rates." },
{ r: [7, 8], name: "Market Reading", desc: "Know approximate value and demand of any item. No roll." },
{ r: [9, 10], name: "Smuggling Routes", desc: "Move restricted goods through any location. Risk roll only at checkpoints." },
{ r: [11, 12], name: "Credit Network", desc: "Access funds beyond current inventory. Repaid within one arc." },
{ r: [13, 14], name: "Auction Control", desc: "Control pace and pressure of any bidding situation." },
{ r: [15, 16], name: "Bribery", desc: "CHA roll to find any NPC price. On success, they cooperate once." },
{ r: [17, 18], name: "Hollow Arbitrage", desc: "Once/arc, turn a worthless item into something useful." },
{ r: [19, 19], name: "The Long Game", desc: "Plant an arrangement. It pays off two sessions later without further action." },
{ r: [20, 20], name: "THE HOLLOW'S PRICE", desc: "UNIQUE. Once/arc, name a price for anything. Any NPC will pay it." }
],
Bard: [
{ r: [1, 2], name: "Crowd Reading", desc: "+2 to all mass persuasion rolls." },
{ r: [3, 4], name: "Improvisation", desc: "Perform without preparation. No penalty for unexpected situations." },
{ r: [5, 6], name: "Emotional Manipulation", desc: "Shift one NPC emotion in a chosen direction. CHA roll." },
{ r: [7, 8], name: "Satire", desc: "Publicly humiliate a target. They lose one social interaction this session." },
{ r: [9, 10], name: "Lullaby", desc: "Calm one hostile NPC. They cannot attack for one round." },
{ r: [11, 12], name: "War Song", desc: "All allies gain +1 to their next roll when performed in combat." },
{ r: [13, 14], name: "The Long Memory", desc: "Recall any piece of lore or history heard before. +3 knowledge rolls." },
{ r: [15, 16], name: "Hollow Resonance", desc: "In Hollow locations, performance has unpredictable additional effect." },
{ r: [17, 18], name: "The Turning Song", desc: "Once/session, change the emotional state of an entire room." },
{ r: [19, 19], name: "The Epic", desc: "Once/arc, tell a story that permanently changes how one faction views a player." },
{ r: [20, 20], name: "THE HOLLOW'S SONG", desc: "UNIQUE. Once/arc, performance becomes permanent in world memory." }
]
};

```

```

const SHADOW_TRAITS = {
Warrior: [
{ r: [1, 2], name: "The Coward's Scar", desc: "Has run from one fight. Will avoid similar situations. Rolls -1 in matching circumstances." },
{ r: [3, 4], name: "The Berserker Flaw", desc: "Below 3 HP, must roll WIS 10+ to choose targets. Failure = attack nearest." },
{ r: [5, 6], name: "The Oath-Breaker", desc: "Broke a vow. One faction knows. -2 with that faction permanently." },
{ r: [7, 8], name: "The Blood Debt", desc: "Owes a life. The creditor hasn't collected yet." },
{ r: [9, 10], name: "The Haunted Blade", desc: "Weapon carries a death. Once/arc it reacts unexpectedly." },
{ r: [11, 12], name: "The War Fatigue", desc: "Extended combat without rest: -1 cumulative per session over 3." },
{ r: [13, 14], name: "The Fallen Banner", desc: "Their side lost a war. Some factions remember. -1 trust with Kingdom remnants." },
{ r: [15, 16], name: "The Unnecessary Death", desc: "Someone died because of their choice. DM knows who." },
{ r: [17, 18], name: "The Pride", desc: "Cannot accept help when at full HP. Must attempt alone first." },
{ r: [19, 20], name: "The Hollow's Claim", desc: "The Hollow marked them in their worst moment. It is owed a service." }
],

```

Wizard: [

```
{ r: [1, 2], name: "The Forbidden Study", desc: "Accessed knowledge they shouldn't. Something knows they know." },
{ r: [3, 4], name: "The Failed Spell", desc: "A spell went wrong. Someone was hurt. They don't discuss it." },
{ r: [5, 6], name: "The Dependency", desc: "Relies on an arcane substance. Without it: -1 all rolls after session 2." },
{ r: [7, 8], name: "The Stolen Research", desc: "Their best work isn't theirs. The original scholar is looking for it." },
{ r: [9, 10], name: "The Unfinished Experiment", desc: "Left something running. It has been running since. DM decides what." },
{ r: [11, 12], name: "The Hollow Contact", desc: "Something reached out to them once. They responded. It remembers." },
{ r: [13, 14], name: "The Broken Equation", desc: "A calculation they can't solve. They keep trying. -1 WIS when it comes up." },
{ r: [15, 16], name: "The Rival", desc: "Another scholar wants their position. They are more connected." },
{ r: [17, 18], name: "The Hubris", desc: "Believes their intelligence exempts them from consequence. Once/arc this is wrong." },
{ r: [19, 20], name: "The Void Conversation", desc: "The Hollow spoke to them directly once. They are not sure if they understood correctly." }
```

],

Rogue: [

```
{ r: [1, 2], name: "The Burned Contact", desc: "Sold someone out. They escaped. They know." },
{ r: [3, 4], name: "The Recognisable Face", desc: "Too well known in one location. -2 stealth there." },
{ r: [5, 6], name: "The Loose Ledger", desc: "Their name appears in a crime record. Someone has it." },
{ r: [7, 8], name: "The Unkept Promise", desc: "Failed to deliver something. The client is patient. For now." },
{ r: [9, 10], name: "The Double Life", desc: "Maintaining two identities. Once/arc, the wrong one surfaces." },
{ r: [11, 12], name: "The Collateral", desc: "Someone innocent was hurt during a job. They know. The Rogue knows they know." },
{ r: [13, 14], name: "The Ghost That Talks", desc: "Left a witness alive once. The witness has been talking." },
{ r: [15, 16], name: "The Hollow's Record", desc: "The Hollow tracked one of their jobs. The record exists somewhere." },
{ r: [17, 18], name: "The Price on the Head", desc: "A faction has an open contract on them. Amount: unknown." },
{ r: [19, 20], name: "The Thing They Kept", desc: "Stole something they should have returned. It has significance they haven't understood yet." }
```

],

Healer: [

```
{ r: [1, 2], name: "The One They Couldn't Save", desc: "A patient died despite everything. They remember every detail." },
{ r: [3, 4], name: "The Hollow Debt", desc: "Used something from the Hollow to save a life. The Hollow considers this a transaction." },
{ r: [5, 6], name: "The Restricted Knowledge", desc: "Knows a healing method the Order forbids. Has used it." },
{ r: [7, 8], name: "The Mercy Problem", desc: "Once helped someone who later caused harm. They don't know yet." },
{ r: [9, 10], name: "The Healer's Burnout", desc: "Giving too much. Once/arc, one healing roll automatically fails." },
{ r: [11, 12], name: "The Forbidden Patient", desc: "Healed someone they were told not to. The person who told them remembers." },
{ r: [13, 14], name: "The Misdiagnosis", desc: "Got something wrong once. The patient recovered. The error remains in the record." },
{ r: [15, 16], name: "The Last Rite", desc: "Performed a death rite for someone who wasn't quite dead. They came back changed." },
{ r: [17, 18], name: "The Order's Eye", desc: "The Healing Order is watching their methods. Not yet disapproving. Watching." },
{ r: [19, 20], name: "The Hollow's Ward", desc: "The Hollow protected them once when they should have died. It expects something in return." }
```

],

Merchant: [

```
{ r: [1, 2], name: "The Unpaid Debt", desc: "Owes a significant sum. Creditor has not yet called it in." },
{ r: [3, 4], name: "The Forged Ledger", desc: "One major transaction was fraudulent. The record exists." },
{ r: [5, 6], name: "The Ruined Partner", desc: "A previous partner lost everything in a deal they made together." },
{ r: [7, 8], name: "The Cursed Stock", desc: "Sold something cursed without knowing. Buyer suffered. Buyer remembers." },
{ r: [9, 10], name: "The Guild Suspension", desc: "Was removed from a guild temporarily. Reason: classified." },
{ r: [11, 12], name: "The Leverage", desc: "Someone has information about them. Has not used it. Yet." },
{ r: [13, 14], name: "The Hollow Trade", desc: "Completed a transaction in a Hollow location. The Hollow considers it binding." },
{ r: [15, 16], name: "The Undercut", desc: "Deliberately destroyed a competitor. The competitor has rebuilt. With allies." }
```

```

    { r: [17, 18], name: "The Stolen Route", desc: "Uses a trade route that belongs to someone else. They have noticed the profit drop." },
    { r: [19, 20], name: "The Price of Everything", desc: "Knows the value of everything. Once/arc this knowledge is used against them." }
  ],
  Bard: [
    { r: [1, 2], name: "The Stolen Song", desc: "Their most famous work belongs to someone else. That person is still alive." },
    { r: [3, 4], name: "The Told Secret", desc: "Used someone's confidence in a performance. They heard it." },
    { r: [5, 6], name: "The Exaggerated Legend", desc: "A story they tell about themselves is not entirely true. Someone knows the real version." },
    { r: [7, 8], name: "The Inspired Enemy", desc: "A performance once moved the wrong person. They took it personally." },
    { r: [9, 10], name: "The Hollow Audience", desc: "Performed in a Hollow location. Something in the audience has not forgotten." },
    { r: [11, 12], name: "The Unfinished Saga", desc: "Began a story they never finished. The subjects are waiting." },
    { r: [13, 14], name: "The Witness", desc: "Saw something they were not supposed to see. Turned it into art. Art can be found." },
    { r: [15, 16], name: "The Price of Fame", desc: "Known widely enough that anonymity is impossible in populated areas." },
    { r: [17, 18], name: "The Last Performance", desc: "Once performed for someone who died immediately after. Some say the performance caused it." },
    { r: [19, 20], name: "The Hollow's Memory", desc: "The Hollow has recorded one of their performances. They do not know what it will do with it." }
  ]
};

```

```
const ENCHANTMENTS = {
```

```
  Warrior: [
```

```

    { r: [1, 3], name: "Iron Oath", desc: "Declare a target. +1 ATK against that target until they are defeated." },
    { r: [4, 6], name: "Blood Memory", desc: "After taking damage, +1 ATK for rest of that session." },
    { r: [7, 9], name: "The Unbreaking", desc: "Weapon cannot be destroyed, damaged, or disarmed. Ever." },
    { r: [10, 12], name: "Hollow Temper", desc: "In Hollow locations, weapon deals +1 damage." },
    { r: [13, 15], name: "The Last Word", desc: "Once/arc, weapon guarantees final strike against a marked enemy." },
    { r: [16, 18], name: "War's Echo", desc: "First attack each session is always at maximum effect." },
    { r: [19, 19], name: "The Hollow's Hunger", desc: "Each session, weapon grows stronger. +1 cumulative (resets per arc)."},
    { r: [20, 20], name: "THE VOID BLESSING", desc: "UNIQUE. Weapon exists partially in the Hollow. Strikes bypass all resistances." }
  ],

```

```
  Wizard: [
```

```

    { r: [1, 3], name: "Arcane Loop", desc: "Once/session, a failed spell can be re-cast without cost." },
    { r: [4, 6], name: "Hollow Resonance", desc: "Spells in Hollow locations have +1 effect." },
    { r: [7, 9], name: "The Unbinding", desc: "Once/arc, break any magical effect or ward. No roll required." },
    { r: [10, 12], name: "Echo Casting", desc: "Spells leave a faint echo. Can trigger again on 15+ next round." },
    { r: [13, 15], name: "Void Channel", desc: "Once/session, channel the Hollow directly. Effect determined by DM." },
    { r: [16, 18], name: "Memory Archive", desc: "Store any piece of knowledge permanently. Perfect recall." },
    { r: [19, 19], name: "The Hollow's Library", desc: "Once/arc, access any piece of knowledge the Hollow has." },
    { r: [20, 20], name: "THE VOID MIND", desc: "UNIQUE. Spells cannot fail completely. Minimum result is always Weak Hit." }
  ],

```

```
  Rogue: [
```

```

    { r: [1, 3], name: "Shadow Step", desc: "Once/session, avoid one consequence entirely. No roll." },
    { r: [4, 6], name: "The Clean Exit", desc: "Once/session, leave any situation without being followed." },
    { r: [7, 9], name: "Hollow Fade", desc: "In Hollow locations, become briefly undetectable once/session." },
    { r: [10, 12], name: "The Long Memory", desc: "Never forget a face, location, or overheard conversation." },
    { r: [13, 15], name: "Ghost Touch", desc: "Handling items leaves no physical trace." },
    { r: [16, 18], name: "The Null", desc: "Once/arc, an action you took cannot be proven to have happened." },
  ],

```

```

    { r: [19, 19], name: "The Hollow's Cloak", desc: "The Hollow itself obscures their identity. Impossible to scry." },
    { r: [20, 20], name: "THE VOID STEP", desc: "UNIQUE. Once/arc, be in two locations simultaneously for one scene." }
  ],
  Healer: [
    { r: [1, 3], name: "The Mending", desc: "Healing rolls always restore at least 1 HP, even on failure." },
    { r: [4, 6], name: "Hollow Touch", desc: "Healing in Hollow locations restores an additional 1 HP." },
    { r: [7, 9], name: "The Ward", desc: "Once/session, prevent all damage to one character for one round." },
    { r: [10, 12], name: "Soul Read", desc: "By touch, know exact HP and all active conditions of any character." },
    { r: [13, 15], name: "The Stitch", desc: "Repair item conditions by one tier once/session." },
    { r: [16, 18], name: "Life Debt", desc: "Characters they revive carry +1 to all rolls for the following session." },
    { r: [19, 19], name: "The Hollow's Grace", desc: "Once/arc, heal is maximised without a roll." },
    { r: [20, 20], name: "THE VOID MEND", desc: "UNIQUE. Once/arc, undo one physical event from this session." }
  ],
  Merchant: [
    { r: [1, 3], name: "Gilded Tongue", desc: "First trade each session automatically goes in their favour." },
    { r: [4, 6], name: "Hollow Market", desc: "In Hollow locations, find one item unavailable anywhere else." },
    { r: [7, 9], name: "The Living Ledger", desc: "Automatically know if any deal is unfair to them." },
    { r: [10, 12], name: "Demand Creation", desc: "Once/session, make an NPC want something they did not want before." },
    { r: [13, 15], name: "The Network", desc: "Once/arc, a contact delivers something useful. No cost." },
    { r: [16, 18], name: "Compound Interest", desc: "Each session without making a deal, next deal has +1 bonus (max +3)." },
    { r: [19, 19], name: "The Hollow's Economy", desc: "Once/arc, exchange anything for anything at agreed value." },
    { r: [20, 20], name: "THE VOID MARKET", desc: "UNIQUE. Once/arc, create an item that does not exist from nothing." }
  ],
  Bard: [
    { r: [1, 3], name: "The Encore", desc: "Once/session, repeat the effect of any successful performance." },
    { r: [4, 6], name: "Hollow Song", desc: "In Hollow locations, performances have an additional unpredictable effect." },
    { r: [7, 9], name: "The Living Record", desc: "Any story they tell becomes true memory for NPCs who hear it." },
    { r: [10, 12], name: "Crowd Pull", desc: "Once/session, draw any NPC's attention completely for one round." },
    { r: [13, 15], name: "The Long Note", desc: "A performance effect can persist until next session. Once/arc." },
    { r: [16, 18], name: "The Stirring", desc: "Once/session, change the emotional state of an entire crowd." },
    { r: [19, 19], name: "The Hollow's Resonance", desc: "Once/arc, performance affects even Hollow entities." },
    { r: [20, 20], name: "THE VOID CHORUS", desc: "UNIQUE. Once/arc, perform something that changes one permanent world truth." }
  ]
];

const DESTINY_THREADS = {
  Warrior: [
    { r: [1, 3], thread: "You will face something that cannot be killed by force alone. What you do in that moment defines what the Hollow thinks of you." },
    { r: [4, 6], thread: "Someone you protect will betray you. You will see it coming and choose to let it happen anyway." },
    { r: [7, 9], thread: "The war you fought is not over. The side you fought for is not the side you thought it was." },
    { r: [10, 12], thread: "You will be given a choice between your weapon and something you value more. The Hollow is watching which you choose." },
    { r: [13, 15], thread: "A battle is coming that you cannot win. You will fight it anyway. What you lose in it is not what you expect." },
    { r: [16, 18], thread: "You will become the thing others make stories about. The story will not be the one you would have chosen." },
    { r: [19, 19], thread: "The Hollow made you a warrior for a specific purpose. You have not yet understood what that purpose is." },
    { r: [20, 20], thread: "UNIQUE. You are the last of something. When you understand what, everything changes." }
  ]
}

```

],

Wizard: [

{ r: [1, 3], thread: "The knowledge you are seeking will change what you are. You are seeking it anyway." },
{ r: [4, 6], thread: "You will find something in the Hollow that no one was meant to find. The finding is the point." },
{ r: [7, 9], thread: "The answer you need is in a place you cannot go without becoming something you do not want to become." },
{ r: [10, 12], thread: "Something you know will save a life and end another. You will not know which is which until it is done." },
{ r: [13, 15], thread: "The Hollow has a question for you. You will not recognise it as a question when it is asked." },
{ r: [16, 18], thread: "You will be offered everything you have ever wanted. The offering is the test. What you do with the test is the point." },
{ r: [19, 19], thread: "You are not researching the Hollow. The Hollow is researching you." },
{ r: [20, 20], thread: "UNIQUE. You already know how this ends. You have not admitted it to yourself yet." }

],

Rogue: [

{ r: [1, 3], thread: "The person you are running from is not who you think they are. Neither are you." },
{ r: [4, 6], thread: "You will steal something that cannot be unfound. The finder will use it. The use will matter." },
{ r: [7, 9], thread: "Someone is using you without your knowledge. When you find out, you will have to decide if it matters." },
{ r: [10, 12], thread: "The one job you refuse to take will find you anyway." },
{ r: [13, 15], thread: "You will be in the right place at the wrong time or the wrong place at the right time. The Hollow has not decided which." },
{ r: [16, 18], thread: "Your real name will surface. What people do with it determines what you do next." },
{ r: [19, 19], thread: "You have already changed something permanently. You do not know what it was." },
{ r: [20, 20], thread: "UNIQUE. The Hollow has been following your work. It has an offer. The offer is not what you would expect." }

],

Healer: [

{ r: [1, 3], thread: "You will save someone who should not have survived. What they do with that survival is on you." },
{ r: [4, 6], thread: "The one you could not save will become relevant again. How is not something you can prepare for." },
{ r: [7, 9], thread: "You will be asked to choose between two lives. The Hollow will not tell you which choice is correct." },
{ r: [10, 12], thread: "Your healing will be used for something you would not have chosen. You will find out after." },
{ r: [13, 15], thread: "Something is using your compassion against you. You will have to decide if you care." },
{ r: [16, 18], thread: "You will face a death you cannot prevent. What you do in the aftermath is the destiny, not the death." },
{ r: [19, 19], thread: "The Hollow wants something healed that it will not name. You will know it when you see it." },
{ r: [20, 20], thread: "UNIQUE. You are not here by accident. The Hollow placed you here specifically for one moment that has not happened yet." }

],

Merchant: [

{ r: [1, 3], thread: "The deal that defines you is coming. You will not recognise it as the defining one until it is done." },
{ r: [4, 6], thread: "You will possess something priceless. The price will be extracted whether you sell it or not." },
{ r: [7, 9], thread: "The network you have built will be used against you by someone inside it." },
{ r: [10, 12], thread: "You will be offered a transaction that is genuinely fair. It will be the most dangerous thing anyone has ever offered you." },
{ r: [13, 15], thread: "Something in the Hollow economy responds to you specifically. You do not yet know if that is good." },
{ r: [16, 18], thread: "The currency you are actually trading in is not coin. The Hollow is about to show you what it is." },
{ r: [19, 19], thread: "You will make a deal with something that does not deal in conventional terms. You will not know until after." },
{ r: [20, 20], thread: "UNIQUE. The Hollow considers you its merchant. This means something specific that you have not been told." }

],

Bard: [

{ r: [1, 3], thread: "The song that defines the Hollow Realm is not yet written. The Hollow is waiting to see if you are the one who writes it." },
{ r: [4, 6], thread: "You will tell a story that becomes true. You will not know it was a prophecy when you were telling it." },
{ r: [7, 9], thread: "Your most important audience has not heard you yet. When they do, something permanent happens." },

```

{ r: [10, 12], thread: "The story you are afraid to tell is the one that matters most." },
{ r: [13, 15], thread: "You will witness the defining moment of this arc. What you do with the witness is what the Hollow is interested in." },
{ r: [16, 18], thread: "Something in the Hollow remembers an older version of a song you know. It wants to hear it again." },
{ r: [19, 19], thread: "The Hollow is listening to everything you perform. It is building something with what it hears." },
{ r: [20, 20], thread: "UNIQUE. You are the narrator of this story. You have not realised this means you can change how it ends." }
]
};

```

```

const PRIMARY_STAT = {Warrior:"STR",Wizard:"INT",Rogue:"SPD",Healer:"WIS",Merchant:"CHA",Bard:"CHA"};
const STAT_COLORS = {STR:"#ef5350",DEF:"#ffd54f",INT:"#7c4dff",WIS:"#66bb6a",SPD:"#4fc3f7",CHA:"#ff9800",LCK:"#ce93d8"};
const RARITY_COLOR = {Common:"#aaa",Uncommon:"#66bb6a",Rare:"#4fc3f7",Legendary:"#ffd54f",Mythic:"#ce93d8",Cursed:"#ef5350"};
const DANGER_COLORS = [""","#66bb6a","#81c784","#ff9800","#ef5350","#b71c1c"];
const DANGER_LABELS = ["","Calm","Tense","Dangerous","Hostile","Extreme"];
const ORIGINS = ["Dark Enchanted Forest","Crumbling Kingdom","Underground Caverns","Hollow's Edge","Sunken Library"];

```

```

const TRAIT_TIERS = [
{level:"△ Weakened",range:[1,4],color:"#ef5350",bonus:-1},
{level:"✓ Standard",range:[5,9],color:"#aaa",bonus:0},
{level:"★ Strong",range:[10,14],color:"#4fc3f7",bonus:1},
{level:"✦ Exceptional",range:[15,19],color:"#ffd54f",bonus:2},
{level:"🔥 LEGENDARY",range:[20,20],color:"#ce93d8",bonus:3}
];

```

```

const RACES = {
"Dust-Walker": {
name: "Dust-Walker",
icon: "👤",
tagline: "Nomads of the Edge. They walked too close to the void, and the void walked back.",
stats: { STR: 0, DEF: 0, INT: 0, WIS: 2, SPD: 1, CHA: -2, LCK: 0 },
ability: "Void-Scarred - Once per session, ignore a magical attack or curse entirely.",
starting_kit: ["Scavenged Breathing Mask", "3x Dried Void-Moss (Stamina recovery)"],
lineage: {
"Ash-Stalker": "+2 Stealth in grey environments; starts with a serrated bone-knife.",
"Void-Oracle": "+2 WIS; can 'read' the history of an object by touching it, but takes 1 sanity damage."
},
vulnerability: "Fragile Mind: -2 to saves against Psychic or Mind-Control effects due to the void-thinning of their psyche.",
interaction_mod: "Dogs and natural animals growl or flee when a Dust-Walker is within 10 feet.",
biological_quirk: "Photosensitive and Void-Saturated: Their prolonged exposure to the Hollow's Edge has physically altered their biology. In complete darkness, their pupils dilate to cover the entire eye, granting near-perfect night vision (+2 Perception). However, their skin is highly sensitive to unfiltered UV light, causing it to crack and ash if exposed to direct, un-shielded sunlight, resulting in a -1 penalty to hit and extreme physical discomfort.",
social_standing: "Outcasts and Omen-Bringers: In civilized settlements, Dust-Walkers are treated with a mix of deep superstition and outright hostility. Tavern owners will often turn them away, and merchants regularly charge a 5% 'Void Tax' just to scrub the counter after they leave. However, underground syndicates and desperate explorers pay top dollar for their tracking skills, as no one else can navigate the shifting topography of the Edge.",
philosophy: "The Void is not an ending; it is a conversation: To a Dust-Walker, silence is heavy and the emptiness speaks. They believe that fighting the decay of the world is a fool's errand. Instead, one must learn the rhythm of the collapse and move with it. Survival isn't about building fortresses; it's about being light enough to outrun the fall.",
taboo: "Sleeping under a clear, un-obscured sky: Dust-Walkers believe the stars are not celestial bodies, but the unblinking eyes of the entities that

```

inhabit the deeper void. Sleeping without a heavy canopy, a cave roof, or at least a thick hat pulled over the face is considered an invitation for those entities to 'look back' into the sleeper's soul.",

fear: "The Static: They do not fear monsters or men. They fear 'The Static'—a psychological and physical phenomenon where the ambient noise of the world drops out, replaced by a low, grinding hum. To a Dust-Walker, the Static means the void is actively thinning the reality around them, and if they do not move immediately, they will simply cease to exist.",

lore: "Generations ago, these were the frontier families and scouts who lived on the extreme borders of the Crumbling Kingdom. When the realm shifted and the Edge began to devour the land, they didn't flee inward; they adapted. They are a quiet, insular people who trade in whispers and old survival techniques. They cover themselves head-to-toe not just for protection from the elements, but to hide the unnatural, faint violet glow that the void left stained behind their eyes.",

character_description: "Height ranges from 5'10\" to 6'4\". They possess lean, lanky, and severely weathered builds, looking as though the moisture has been slowly baked out of them. They dress in heavily layered, dust-caked leather dusters, rugged canvas pants, and thick boots designed for treacherous terrain. Their faces are usually obscured by heavy bandanas and wide-brimmed western hats. Their most defining feature is their eyes: the irises carry a faint, unnatural violet luminescence, heavily contrasting with their dirt-smudged, sun-deprived skin. Their movements are deliberate, eerily quiet, and lack wasted energy."

},

"Garrison-Born": {

name: "Garrison-Born",

icon: "🛡️",

tagline: "The stubborn descendants of a ruined empire.",

stats: { STR: 1, DEF: 2, INT: 0, WIS: 0, SPD: -1, CHA: 0, LCK: 0 },

ability: "Iron Discipline - Immune to 'Fear' status. +1 to all combat rolls when fighting directly adjacent to an ally.",

starting_kit: ["Dented Sigil Ring", "Standard Issue whetstone", "Ration Tin"],

lineage: {

"Iron-Guard": "+3 DEF when standing still; starts with a heavy tower shield.",

"Vanguard-Scout": "+2 SPD; can ignore 'Difficult Terrain' as long as they are moving toward an enemy."

},

vulnerability: "Heavy-Footed: Suffer a -4 penalty to all Stealth checks involving water or metal flooring.",

interaction_mod: "Merchants give them 10% off gear, but expect them to defend the town for free if an alarm sounds.",

biological_quirk: "Dense Bone Structure: Centuries of consuming specialized military alchemical rations have permanently altered their genetic makeup. Their bones are nearly twice as dense as a normal human's. This allows them to take 50% less damage from blunt force trauma or long falls, but it completely robs them of buoyancy. A Garrison-Born will sink like an anvil in deep water and cannot swim unassisted.",

social_standing: "Relics of Authority: They walk through the world like landlords whose lease expired a century ago. In rural villages, they are respected and feared, often able to bypass low-level militia checkpoints purely on the authority of their posture. In lawless hubs, they are targeted as symbols of an oppressive past. People expect them to be the meat-shields in any crisis, demanding their protection as a lingering obligation of their bloodline.",

philosophy: "Steel lasts longer than hope; polished steel lasts forever: They view the apocalypse not as the end of the world, but as a test of structural integrity. Emotion, panic, and individuality are structural flaws. Routine, chain of command, and maintaining one's equipment are the highest virtues. A well-maintained rifle is prayer; a perfectly executed tactical retreat is a sacrament.",

taboo: "Unrecorded Retreat: Fleeing a battle without an officer's verbal or written command is the ultimate sin. A Garrison-Born who breaks rank in a panic is stripped of their family sigil and 'Erased' from the lineage. They become nameless, and other Garrison-Born are forbidden from acknowledging their existence, even if they are burning alive.",

fear: "Rust of the Mind: They are terrified of losing their purpose. While they obsessively clean their gear to prevent physical rust, their true nightmare is the 'rusting' of the mind—dementia, memory loss, or waking up one day and forgetting why they are fighting. Without their history and their orders, they believe they are nothing but violent animals.",

lore: "When the Crumbling Kingdom fractured and society dissolved into chaos, the grand military apparatus refused to dissolve with it. Cut off from command, individual battalions entrenched themselves in fortresses, passing down ranks, protocols, and grudges through the generations. They treat the current broken world as a temporary occupation zone, treating every minor skirmish like a calculated campaign for a throne that turned to dust centuries ago.",

character_description: "Height ranges from 5'8\" to 6'2\". They possess incredibly dense, broad-shouldered, and muscular builds, shaped by generations of strict physical regimens. They visually represent battlefield grit: battered plate armor integrated with heavy chainmail, worn over thick, utilitarian coats. Their clothing features tarnished kingdom crests and faded insignias. Hair is kept practically cropped or shaved. Their faces and bodies are maps of heavy, jagged scarring, and their posture is always rigid, scanning the room for tactical advantages."

},

"The Sunken": {

name: "The Sunken",

icon: "💧",

tagline: "Information is a currency, and they run the mint.",

stats: { STR: -1, DEF: 0, INT: 2, WIS: 0, SPD: 0, CHA: 2, LCK: 1 },

ability: "Silver Tongue - Start the game with a 'Black Ledger.' 10% discount on all Black Market trades.",

starting_kit: ["Encoded Ledger", "Silver Pocket Watch", "Fine Silk Handkerchief"],

lineage: {

 "Guild-Broker": "Starts with 500 extra credits and a 'Favor' token from a high-ranking NPC.",

 "Inksmith": "Can craft 'Living Tattoos' that grant one-time magical buffs to the wearer."

},

vulnerability: "Dehydrated: If they don't consume double the water of a normal human, they suffer 'Exhaustion' levels twice as fast.",

interaction_mod: "Guards follow them constantly in shops. They are known for 'disappearing' small, expensive items.",

biological_quirk: "Amphibious Adaptation: Generation after generation spent in the submerged, flooded levels of the old world has altered their respiratory systems. They possess subtly recessed gills along their ribcages, allowing them to hold their breath in high-stress situations for up to 10 minutes. Furthermore, their eyes possess a secondary membrane, allowing them to see perfectly in murky water, and they suffer absolutely zero movement penalties when submerged.",

social_standing: "High-Society Syndicates: The Sunken are the aristocrats of the underworld. They are incredibly wealthy, universally distrusted, and absolutely essential to the global economy. Commoners view them as loan sharks and mobsters, while the elite view them as necessary evils. Because everyone knows The Sunken deal in blackmail, NPCs are inherently guarded around them, imposing a -1 penalty to friendly persuasion simply because no one wants to end up in their ledger.",

philosophy: "A secret is only valuable if someone is willing to kill for it: Violence is cheap; leverage is priceless. The Sunken believe that physical warfare is a failure of negotiation. True power is walking into a room unarmed and having the most dangerous person there offer you a chair because you know what they did five summers ago. Everything—and everyone—has a price.",

taboo: "Uncouth Brutality: Resorting to an unprovoked fistfight or messy assassination is deeply humiliating to a Sunken's reputation. Using a blunt instrument when a bribe, a threat, or a subtle poison would have sufficed is considered barbaric. Those who act like 'thugs' are exiled from the inner circles of the archives.",

fear: "Desiccation: A deep, biological terror of profound dryness. If a Sunken spends more than a week away from high humidity, coastal fog, or the flooded ruins, their skin begins to painfully crack, and their lungs ache with every breath. The idea of dying of thirst in a desert is their equivalent of hell.",

lore: "When the great shift drowned the grand libraries and banking districts of the old world, the archivists and financiers didn't drown; they evolved. Trapped in the flooded metropolis, they formed syndicates to protect the restricted knowledge and pre-shift artifacts they controlled. Today, they operate out of water-logged manors and hidden subterranean docks, running a massive shadow economy based entirely on debts, favors, and hoarded history.",

character_description: "Height ranges from 5'5\" to 6'0\". They have sleek, sharp, and flawlessly poised builds. Visually, they combine high-class elegance with aquatic decay: immaculately tailored dark coats that are permanently damp or water-stained at the hems, paired with ornate, oxidized silver jewelry. Their hair is eternally slicked back, and their skin has a slightly pale, cold sheen. Their expressions are calculating, aloof, and carry the distinct look of someone calculating exactly how much you are worth in an auction."

},

"Deep-Claimed": {

 name: "Deep-Claimed",

 icon: "🔪",

 tagline: "Trapped in the dark, they became harder than the stone.",

 stats: { STR: 2, DEF: 1, INT: -1, WIS: 0, SPD: 0, CHA: -1, LCK: 0 },

 ability: "Stonebreaker - Can mine Hollow Ore without tools and naturally detect structural traps in the dark.",

 starting_kit: ["Heavy Work Gloves", "Glowing Ore Fragment (Light source)", "Chisel"],

 lineage: {

 "Bedrock-Bulwark": "Cannot be knocked down or pushed back by physical force.",

 "Ore-Seer": "Can smell precious metals through up to 5 feet of solid stone."

 },

 },

 vulnerability: "Light-Blinded: Suffer -3 to all rolls for 1 hour after being exposed to a 'Flashbang' or magical bright light.",

 interaction_mod: "Other races find their 'Ore-Skin' unsettling; -2 to CHA when attempting to seduce or charm non-Deep-Claimed.",

 biological_quirk: "Ore-Skin Calcification: To survive the starvation of the deep caverns, they consumed trace amounts of volatile Hollow Ore. Over generations, this ore has integrated into their physiology. Their skin is incredibly rough, providing a natural +1 to DEF when unarmored, as blunt blades

literally chip against their flesh. However, this calcification makes wearing traditional metal armor excruciatingly uncomfortable, causing a claustrophobic reaction (-1 SPD).",

social_standing: "The Exploited Foundation: They are the ultimate labor class, viewed by surface-dwellers as little more than talking heavy machinery. In industrial hubs and mining towns, they are welcomed for their unmatched work ethic. In high-society or political arenas, they are treated as uneducated brutes. They are the backbone of the realm's infrastructure, yet they are rarely allowed to sit at the tables of the cities they built.",

philosophy: "The mountain gives nothing; you must take it: Deep-Claimed do not believe in luck, divine intervention, or charity. They believe entirely in the physics of effort. If you strike the stone long enough, it breaks. If you endure the dark long enough, you survive. They value sheer, unrelenting persistence above all intellectual or magical pursuits.",

taboo: "Wasting the Light: In the deep dark, fuel is life. Leaving a torch to burn out in an empty room, or throwing away a spark-flint before it is completely stripped, is considered a crime bordering on murder. Those who waste heat or light are often tied to a cavern wall and left in the pitch black for three days to 'learn the value of a spark'.",

fear: "The 'Wide-Open' (Agoraphobia): Deep-Claimed feel secure when surrounded by tons of crushing rock; it feels like an embrace. When they are brought to the surface and placed under a cloudless, endless blue sky, they suffer profound vertigo and panic attacks. The lack of a 'ceiling' makes them feel as though gravity might fail and they will fall upward into the terrifying expanse of the cosmos.",

lore: "When the realm fractured, thousands of miners and underground survivalists were entirely sealed off in the deepest subterranean networks. Cut off from surface agriculture, they survived by farming bioluminescent fungi and integrating the magical Hollow Ore into their diet. Decades in the pitch black transformed them into fiercely territorial, hulking behemoths who worship the veins of the earth and distrust anything that requires sunlight to grow.",

character_description: "Height ranges massively from 6'0" to 6'8". They are hulking, barrel-chested, and violently muscular, built to swing hammers for 16 hours a day. Their skin is deathly pale, heavily stained with permanent soot, and visibly textured with raw, faintly glowing veins of ore that protrude from their forearms and neck. They wield heavy, brutal iron mining tools (picks, sledgehammers) as weapons. They rarely wear full shirts, preferring thick leather harnesses rigged with glowing ore chunks that illuminate their subterranean surroundings."

},

"Aether-Kith": {

name: "Aether-Kith",

icon: "⚡",

tagline: "The remnants of the high-culture, now drifting between states of matter.",

stats: { STR: -2, DEF: -1, INT: 1, WIS: 1, SPD: 0, CHA: 1, LCK: 3 },

ability: "Phase Shift - Once per combat, move through a solid object or enemy without provoking an attack.",

starting_kit: ["Glass Vial of Essence", "Translucent Cape", "Ancient Map Fragment"],

lineage: {

"Wisp-Kin": "Can temporarily hover over gaps or pressure plates without triggering them.",

"Spark-Weaver": "Can generate small localized electrical charges to power old technology or shock targets."

},

vulnerability: "Disrupted Form: Take 1d4 damage every turn they end their movement inside a strong magnetic or anti-magic field.",

interaction_mod: "People often speak loudly and slowly to them, assuming they aren't fully 'there,' which gives a +1 to deceptive misdirection.",

biological_quirk: "Semi-Ethereal Mass: The Aether-Kith are not entirely bound to the physical plane. They possess almost zero physical weight, meaning they never suffer from 'Encumbered' penalties and leave no footprints on soft ground. However, this lack of mass is dangerous; a strong gust of wind or a kinetic shockwave/blast effect will deal double damage to them, easily launching them across a battlefield.",

social_standing: "Coveted Curiosities: To the average citizen, an Aether-Kith is a ghost, treated with a mix of awe, religious reverence, or profound unease. To scholars and underground syndicates, they are priceless commodities. They are heavily targeted by kidnappers and dark mages who wish to harvest their essence or study their shifting atomic structure. They rarely find a place to belong and are usually treated as 'objects' rather than people.",

philosophy: "Form is temporary; energy is eternal: They view the physical struggles of the other races—wars over land, hoarding of gold—as deeply tragic and short-sighted. They believe that true enlightenment comes from shedding attachments to the material world. To an Aether-Kith, the mind and the spirit are the only things that truly exist; the body is merely a temporary, highly flawed vessel.",

taboo: "Material Hoarding: Becoming overly attached to a physical object is seen as an anchor that corrupts the soul. An Aether-Kith who collects trinkets, refuses to share supplies, or values gold over experience is viewed by their kin as 'heavy' or 'sinking'. A true Aether-Kith carries only what is immediately necessary to maintain their temporary physical anchor.",

fear: "The 'Anchor': A legendary, localized anti-magic phenomenon scattered across the Hollow Realm. If an Aether-Kith remains in an Anchor field for too long, their ethereal atoms permanently calcify. The idea of being permanently solidified, trapped in a heavy, failing meat-sack like the rest of the races, is a fate far worse than death. It is claustrophobia on a molecular level.",

lore: "Centuries ago, the grand mages of the high-culture attempted a massive ritual to ascend beyond physical limitations just as the Crumbling Kingdom fell. The ritual was interrupted, leaving their descendants trapped halfway between reality and the aether. They drift through the Hollow Realm as shimmering anomalies, struggling to maintain their physical coherence and desperately searching for a way to either complete the ascension or

reverse the curse.",

character_description: "Height averages from 5'6\" to 6'2\", but their outlines constantly blur and shift slightly. They are incredibly slender and ethereal, appearing almost weightless. Their skin is translucent, shimmering with a soft, bioluminescent glow (usually pale blues, purples, or golds) that pulses in time with their heartbeat. They wear flowing, iridescent garments made of ultra-light silks and carry floating, gravity-defying jewelry. Looking directly at them is like looking at a person through a heat haze or a prism."

```
},
"Hollow-Touched": {
  name: "Hollow-Touched",
  icon: "👁",
  tagline: "Marked by the void.",
  stats: { STR: 1, DEF: 0, INT: 1, WIS: 0, SPD: 0, CHA: -1, LCK: 3 },
  ability: "Void Resonance"
}
};
```

```
const COMPANION_TYPES = {
  "Void-Hound": {
    name: "Void-Hound",
    icon: "🐺",
    hp: 5,
    statBuff: { CHA: 1 },
    active_ability: "Echo Bark - Command the hound to 'speak'. It releases a concussive soundwave. Enemies within 15 feet must roll to resist being 'Stunned' for one turn, and hidden enemies are instantly revealed.",
    passive_trait: "Scent of the Collapse - The hound will naturally whine and pace when a magical trap, void anomaly, or ambush is in the next room, giving the player a +2 to initiative or trap detection.",
    maintenance: "Feeds on raw meat and negative energy. Cannot be healed by standard medical kits; must 'consume' the shadow of a defeated enemy to regain HP.",
    behavior_quirk: "They do not sleep. When the player rests, the hound simply sits rigidly, staring directly into the darkest corner of the room.",
    lore: "Forged from the ash of the Edge and bound by dark magic. They are fiercely loyal to whoever binds them, but deeply unsettling to anyone who isn't their master, often viewed as walking bad omens.",
    physical_description: "Roughly the size of a large timber wolf, but lacking solid flesh. Its body is composed of tightly swirling black ash, compacted shadow, and glowing orange cinders that flare up when it is agitated. It has no discernible eyes, only two hollow, jagged indentations in its skull where faint, violet smoke constantly leaks and trails behind it as it moves.",
    sensory_profile: "Smells strongly of ozone, sulfur, and a freshly extinguished campfire. It leaves no footprints, only faint scorch marks. It does not pant or breathe. When it barks or snarls, the sound does not seem to come from its mouth; instead, it is a localized, bass-heavy rumble that echoes off the walls behind you, creating a disorienting, surround-sound effect of pure malice."
  },
  "Scavenger Crow": {
    name: "Scavenger Crow",
    icon: "🐦",
    hp: 3,
    statBuff: { LCK: 1 },
    active_ability: "Pilfer - Point at a small, unsecured object up to 30 feet away (a key, a potion, a coin pouch). The crow will fly out, grab it, and return it to your inventory without breaking your stealth.",
    passive_trait: "Grave Robber (+10% Loot Luck) - After every combat encounter, the crow will automatically scavenge the area, rolling an independent loot check to find extra credits, ammo, or small trinkets.",
    maintenance: "Requires constant bribery. If you do not give it a 'shiny' object (even a worthless coin) once every few days, it will start dropping your inventory items out of spite.",
    behavior_quirk: "It mimics human voices perfectly, but only repeats the last words of people it has watched die.",
  }
};
```

lore: "Mutated corvids that survived the shift by becoming smarter and meaner. Having one on your shoulder marks you as a hardened survivor, but also guarantees that civilized folk will keep a very close eye on their pockets.",

physical_description: "Abnormally massive, easily the size of a wild turkey, with heavily scarred, leathery skin visible beneath patchy plumage. Its feathers are a greasy, iridescent black, constantly mottled with pale dust. Its beak is jagged, serrated like a rusted bread knife. It often decorates itself, weaving stolen copper wire into its feathers or forcing stolen rings onto its thick, scaly talons.",

sensory_profile: "Emits a smell like old copper coins and dried blood. Its movements are heralded by the heavy, leathery thwump of its wings and the dry, rhythmic clicking of its beak. When it 'speaks', there is no bird-like squawk; it produces an eerily flawless, uncompressed recording of human terror, matching the exact pitch, breathlessness, and panic of the voice it stole."

},

"Clockwork Mule": {

name: "Clockwork Mule",

icon: "⚙️",

hp: 8,

statBuff: { DEF: 1 },

active_ability: "Mobile Barricade - Command the Mule to lock its heavy hydraulic joints. It becomes an immovable object, granting +2 DEF to any ally using it as cover. It cannot move or carry items while locked.",

passive_trait: "Tireless Hauler (+5 Inventory Slots) - As long as it is functioning, you ignore standard encumbrance rules for heavy gear like mining tools or excess armor.",

maintenance: "Cannot be healed with magic or medicine. Requires 'Scrap Parts' and oil to repair. If it hits 0 HP, it drops all its cargo on the ground and must be rebooted at a workbench.",

behavior_quirk: "Its gears grind endlessly, and it occasionally vents hot steam out of unexpected valves. Imposes a permanent -1 penalty to the group's Stealth checks while moving.",

lore: "Relics of the old mining guilds from the Crumbling Kingdom. They are entirely emotionless, relentlessly grinding forward as long as their mainspring is wound, completely indifferent to the apocalypse happening around them.",

physical_description: "Built like a mechanical, headless bulldog the size of a draft horse. Plated in thick, tarnished iron and bronze boilerplate armor, heavily dented from years of rockfalls. Faded, painted kingdom crests are still barely visible on its flanks. Its four legs are driven by exposed hydraulic pistons, ending in heavy, flat iron hooves fitted with worn-out rubber shock absorbers.",

sensory_profile: "Radiates an immense amount of physical heat. The air around it constantly smells of burning coal, hot engine oil, and heated brass. It is incredibly loud—even when standing still, there is a rhythmic, heavy ticking from its internal clockwork. When it walks, there is a violent *clank-hiss* of unlubricated pneumatics and metal grinding against stone."

},

"Mender Sprite": {

name: "Mender Sprite",

icon: "🔮",

hp: 2,

statBuff: { WIS: 1 },

active_ability: "Cauterizing Sap - Command the sprite to heal a target. Restores 1d6 HP by violently sealing the wound with glowing, burning sap. The extreme pain gives the target a -1 to their next attack roll.",

passive_trait: "Guardian Light (Auto-stabilize) - Emits a soft, 10-foot radius of bioluminescent light. Once per session, if the player drops to 0 HP, the sprite will instantly kamikaze into the wound, stabilizing the player at 1 HP but dying in the process.",

maintenance: "Feeds on ambient magic and Hollow Ore dust. Highly fragile; takes double damage from physical area-of-effect attacks.",

behavior_quirk: "Hums constantly. When it is agitated or sensing danger, the light it emits flashes violently through different spectrums, acting as an unintended warning siren.",

lore: "Bioluminescent, semi-sentient aether entities that hover around dense magical areas. They are aggressively helpful, seeking out open wounds to seal them, though their brutal survival-instinct methods are often intensely painful for the patient.",

physical_description: "It is not humanoid in the slightest. It appears as a jagged, floating shard of crystallized amber or a dying, contained star. Its core is made of thick, pulsing liquid gold, surrounded by erratic, orbiting rings of raw, crackling aether energy. It constantly drips a viscous, glowing sap that evaporates before it hits the ground.",

sensory_profile: "Smells intensely sharp, like crushed pine needles mixed with heavy electrical ozone. It sounds like a high-tension power line caught in high wind, emitting a constant, vibrating hum. When it applies its healing sap, it produces a sudden, violent sizzle—exactly like a slab of raw meat being thrown onto a white-hot iron skillet."

}

```
};
```

```
const WORLD_LOCATIONS = [
```

```
{id:"hq",name:"Moon Squad HQ",short:"HQ",icon:"🏠",color:"#ffd54f",x:50,y:48,danger:1,desc:"The sanctuary of the Moon Squad. Safe, fortified, and growing.",features:["The Forge — Repair & Enchant","The Infirmary — Heal & Purify","The Ledger — Daily Shop","The Bunkhouse — Personal Quarters"],connectedTo:["forest","kingdom","caverns","library","edge"]},

{id:"forest",name:"Dark Enchanted Forest",short:"Forest",icon:"🌲",color:"#2e7d32",x:18,y:28,danger:3,desc:"Ancient. Breathing. The trees here remember everything.",features:["Hollow-adjacent plant material","Ashen Pack territory","Forest caches hidden throughout","Sound travels strangely"],connectedTo:["kingdom","edge"]},

{id:"kingdom",name:"The Crumbling Kingdom",short:"Kingdom",icon:"🏰",color:"#c62828",x:48,y:20,danger:2,desc:"What is left of an empire. The walls still stand in places.",features:["Garrison footlockers in ruins","Valdris holds the throne room","Kingdom vault accessible with key","Military remnants patrol unpredictably"],connectedTo:["forest","caverns","library"]},

{id:"caverns",name:"Underground Caverns",short:"Caverns",icon:"🕸",color:"#6a1b9a",x:36,y:64,danger:3,desc:"The deep dark beneath everything else.",features:["Hollow ore deposits","Thessaly operates from deep sections","Cavern mushroom clusters","Deepest sections never fully mapped"],connectedTo:["kingdom","library"]},

{id:"edge",name:"The Hollow's Edge",short:"Edge",icon:"🌀",color:"#9c27b0",x:74,y:36,danger:5,desc:"Where the Hollow ends and whatever comes next begins.",features:["Rift deposits appear unpredictably","The Null Warden holds the boundary","Time moves differently here","Transition zone materials"],connectedTo:["forest","library"]},

{id:"library",name:"The Sunken Library",short:"Library",icon:"📖",color:"#1565c0",x:64,y:68,danger:2,desc:"Partially submerged. Still functional. Still sinking.",features:["Scholar cases throughout readable levels","The Sink in flooded lower levels","Restricted archive","Research desks surface occasionally"],connectedTo:["kingdom","caverns","edge"]}

];
```

```
const WORLD_NPCS = {
```

```
"Voss, the Ash-Peddler": {
  name: "Voss", title: "The Ash-Peddler", icon: "👤", region: "The Hollow's Edge", danger: 5,
  role: "Mythic Merchant", color: "#ce93d8",
  quote: "The void demands equivalent exchange. Coins, blood, and memory.",
  lore: "A Dust-Walker who has been out on the Edge so long he barely speaks in full sentences. He sits cross-legged in the ash, completely unbothered by the static that shreds reality around him.",
  sensory: "Smells of ozone and burnt hair. His eyes glow a blinding violet. The sound of reality cracking faintly echoes when he breathes.",
  spawn: "Requires a Tracking 16+ check to locate his camp in the ash storm, Failure attracts a Wandering Null-Spawn instead.",
  interaction: "Does not bargain. The price is the price. Refuses to speak to anyone carrying a cursed item.",
  inventory: [
    { item: "Pure Void-Essence (Full Heal, applies minor curse)", cost: "2,000c", chance: "100%" },
    { item: "Random Rare Weapon", cost: "5,000c", chance: "100%" },
    { item: "Shard of the First Collapse (Adaptive Mythic)", cost: "25k + 1 Legendary + 1 Max HP", chance: "Limit 1 per Arc" }
  ]
},

"Quartermaster Rudge": {
  name: "Rudge", title: "Quartermaster", icon: "🛡️", region: "Crumbling Kingdom", danger: 2,
  role: "Standard Merchant", color: "#66bb6a",
  quote: "Show your Kingdom identification, or pay the outlander tax.",
  lore: "A Garrison-Born operating out of a half-collapsed toll booth. He still follows Kingdom protocol strictly, even though the empire has been dead for centuries.",
  sensory: "Smells of polished iron, old leather, and stale ration biscuits. He is constantly, rhythmically sharpening a blade.",
  spawn: "Always present at the Kingdom Gates. Very safe to visit.",
  interaction: "Grants a 10% discount to Garrison-Born players. Dislikes Dust-Walkers.",
```

```

inventory: [
  { item: "Standard Rations & Basic Whetstones", cost: "150c", chance: "100%" },
  { item: "Common/Uncommon Garrison Gear", cost: "500c - 1,500c", chance: "100%" },
  { item: "Random Rare Weapon found in rubble", cost: "4,000c", chance: "10% (Roll 19-20)" }
]
},
"Nylah the Dredge": {
  name: "Nylah", title: "The Dredge", icon: "💧", region: "Sunken Library", danger: 2,
  role: "Arcane Merchant", color: "#4fc3f7",
  quote: "The water keeps the best secrets. I just fetch them.",
  lore: "A Sunken who uses her amphibious traits to dive into the flooded lower levels and bring up sealed lockboxes. She lays her wet, dripping inventory out on a cracked marble table.",
  sensory: "Everything she wears is soaked. Smells strongly of old paper, algae, and deep, stagnant water.",
  spawn: "Always present on the upper grand balconies.",
  interaction: "Highly suspicious. Will not sell rare items to players with low reputation or known thieves.",
  inventory: [
    { item: "Antidotes & Focus Crystals", cost: "300c", chance: "100%" },
    { item: "Uncommon Grimoires & Stiletos", cost: "1,200c", chance: "100%" },
    { item: "Legendary Lore Fragment or Focus", cost: "10,000c", chance: "5% (Roll 20)" }
  ]
},
"Sister Vex": {
  name: "Sister Vex", title: "The Purifier", icon: "✨", region: "Dark Enchanted Forest", danger: 3,
  role: "Curse Breaker", color: "#ff9800",
  quote: "Do not bring that filth into my chapel unless you are ready to bleed for it.",
  lore: "An exiled Healer who set up a sanctuary in a ruined forest chapel. She is the only one on the map who can safely extract Void Infections from looted items.",
  sensory: "The smell of burning sage is overpowering. She hums a low, resonant hymn that vibrates in your teeth.",
  spawn: "The chapel is hidden. Requires a minor offering of blood at the treeline to reveal the path.",
  interaction: "Refuses coins for purification. Trades only in 'Life Echoes'",
  inventory: [
    { item: "Aether Bandages (Heals 3 HP, cures bleed)", cost: "250c", chance: "100%" },
    { item: "Holy Salt (Wards an area for 1 rest)", cost: "500c", chance: "100%" },
    { item: "Purify Void Infection from Item", cost: "-1 Temp Stat Debuff or 1 Max HP", chance: "Always" }
  ]
},
"Krell the Foreman": {
  name: "Krell", title: "The Bedrock Foreman", icon: "⚒️", region: "Underground Caverns", danger: 3,
  role: "Master Smith", color: "#ffb300",
  quote: "The mountain don't care about your stories. It only cares about the weight of your swing.",
  lore: "Krell was the lead excavation officer when the Shift happened. He refused to abandon the deep veins, staying behind to protect the Hollow Ore deposits. Over decades, the raw ore has literally begun to replace his skin. He is the only being in the realm who knows how to fold Hollow Ore into existing weapons without shattering them.",
  sensory: "Smells of sulfur, wet stone, and hot iron. His voice sounds like two tectonic plates grinding together. When he moves, you hear the heavy clank-thud of metal boots and the hiss of a portable steam-forge strapped to his back.",
  spawn: "Requires a Perception 12+ check to hear his rhythmic hammering through the cavern walls. If the Squad is carrying more than 5x Hollow Ore, Krell will find them.",

```

interaction: "Values effort. Gives a 20% discount to players presenting 'Worn' or 'Damaged' weapons. Refuses to work for Aether-Kith (calls them 'cloud-breath').",

inventory: [

- { item: "Hollow Reforge (Permanent +1 ATK to weapon)", cost: "3,000c + 2x Hollow Ore", chance: "Always" },
- { item: "Shatter-Guard (Weapon can never Break again)", cost: "1,500c + 1x Hollow Ore", chance: "Always" },
- { item: "Deep-Blast Charge (Consumable: 2d10 Env. DMG)", cost: "800c", chance: "Always" },
- { item: "Heavy Work Gloves (Passive: +1 STR checks)", cost: "500c", chance: "100%" }

]

}

};

const VILLAIN_LORE = [

{

id: "gravenor", name: "Gravenor", title: "The Hollow's Hound - Alpha of the Ashen Pack",

type: "Werewolf Arc Boss", arc: "Arc I - The Ashen Hunt",

location: "Dark Enchanted Forest", threat: "Extreme", color: "#9c27b0", icon: "🐺",

tagline: "He was a man once. The Hollow made him something more useful.",

lore: "Gravenor was a Warrior stationed at the Forest edge. His unit did not come back. He did. Whatever the Hollow did to him, it kept something - the planning, the memory, the grudges. He leads the Ashen Pack not by command but by agreement. He is not cruel. He is not simple. He is patient in a way that makes him more dangerous than either.",

motivation: "Reclaiming something the Hollow took from him. The players are obstacles, witnesses, or occasionally tools depending on what they have done.",

strengths: ["Regenerates 2 HP/round in Forest or low-light", "Cannot be flanked - senses all within 40 feet", "Pack Coordination - wolves gain +1 when he is above half HP", "Hollow-Hardened - standard weapons deal -1 damage", "Tracking - once scented, can find anywhere in Forest", "Intimidation Aura - characters at 3 HP must roll WIS 12+ to act aggressively"],

weaknesses: ["Silver - bypasses Hollow-Hardened and deals +2", "Fire - cannot regenerate when fire damage received", "His honour - cannot pursue formal surrender", "Daylight on open ground - -1 all rolls", "The thing he lost - if present or named, loses one action"],

phases: [

{ phase: 1, name: "The Watcher", hp: "Full - 80 HP", desc: "Observes. Tests player responses. Will speak if approached with respect.", color: "#4fc3f7"

},

{ phase: 2, name: "The Hunter", hp: "Below 60 HP", desc: "Engages directly. Targets who did most damage to his pack. Precise, tactical.", color: "#ff9800" },

{ phase: 3, name: "The Feral", hp: "Below 30 HP", desc: "The man recedes. Reckless, devastating. Pack members scatter.", color: "#ef5350" },

{ phase: 4, name: "The Reckoning", hp: "At 0 HP", desc: "Collapses. The man surfaces. One round of lucidity. He speaks. What he says is arc-relevant.", color: "#ce93d8" }

],

loot: [

{ item: "Ashen Fang (weapon enchantment)", chance: "65%" },

{ item: "Pack Sigil (faction key)", chance: "80%" },

{ item: "Hollow-Touched Pelt", chance: "45%" },

{ item: "Gravenor's Memory (arc item)", chance: "Phase 4 only - 100%" }

]

},

{

id: "valdris", name: "Valdris the Unmourned", title: "Last Chancellor of the Crumbling Kingdom",

type: "Undead Political Villain - Arc Boss", arc: "Arc II - The Kingdom's Debt",

location: "Crumbling Kingdom throne room", threat: "High", color: "#ef5350", icon: "👹",

tagline: "The Kingdom fell. He did not have the decency to go with it.",

lore: "Valdris did not cause the Kingdom collapse. He facilitated it. He spent his chancellorship building leverage - favours, secrets, alliances that looked like loyalty and functioned like ownership. When collapse came, he had prepared an exit that required sacrificing every connection he had spent a career building. He made the trades. They worked. Then the Hollow noticed the accounting. He has been in the throne room since. Unable to leave. Unable to die. Unable to settle what he owes.",

motivation: "He wants out. Settling his debts legitimately would cost him everything. He is actively recruiting others to transfer the debts to instead.",

strengths: ["Political Intelligence - knows leverage on every faction", "Immune to damage inside throne room", "Debt Transfer - applies Debt consequence to those who accept his deals", "Undying - cannot be permanently killed until debts settled", "Plausible Deniability - CHA 15+ required to accuse him"],

weaknesses: ["Outside Kingdom ruins - powers significantly reduced", "His debts - returning items to specific people weakens him permanently", "Cannot lie outright - only mislead", "He needs the players - without someone to transfer debt to, he is stuck"],

phases: [

{ phase: 1, name: "The Chancellor", hp: "Full - 60 HP", desc: "Diplomatic. Charming. Genuinely helpful selectively. Building trust to spend later.", color: "#4fc3f7" },

{ phase: 2, name: "The Negotiator", hp: "Below 45 HP", desc: "Desperation shows. Offers improve. Costs worsen. Sets players against each other.", color: "#ff9800" },

{ phase: 3, name: "The Unmourned", hp: "Below 20 HP", desc: "Drops the facade. Uses every piece of leverage simultaneously.", color: "#ef5350" }

],

loot: [

{ item: "Kingdom Vault Key", chance: "75%" },

{ item: "Debt Ledger", chance: "60%" },

{ item: "Chancellor's Ring", chance: "40%" },

{ item: "Kingdom Signet", chance: "55%" }

]

},

{

id: "sink", name: "The Sink", title: "What Lives at the Bottom of the Sunken Library",

type: "Entity - Unknown Classification", arc: "Arc III - The Drowned Archive",

location: "Sunken Library permanently submerged lowest level", threat: "Unknown", color: "#4fc3f7", icon: "🏰",

tagline: "It did not sink the Library. It was why the Library was built there.",

lore: "The Sunken Library was built on this location because of what was already here. The founders believed proximity to The Sink made knowledge more accessible. They were correct. Nobody has confirmed what the original negotiation cost. The record is in the flooded sections.",

motivation: "Unknown. Possibly not goal-oriented in ways that translate to motivation. The working theory is that it is absorbing the knowledge in the drowned sections as the water rises.",

strengths: ["Omniscience (partial) - knows everything in drowned sections", "Water domain - all enemy rolls -2 in flooded areas", "The Pull - WIS 12+ each round in deep water or drift deeper", "Cannot be damaged by conventional means in deep water", "Information Weapon - can reveal Shadow Traits and Destiny Threads"],

weaknesses: ["Surface power significantly reduced above water", "Light - sustained magical light reduces The Pull by half", "Direct questions - it answers them. Always. The answers are true.", "The founding documents describe a counter-condition"],

phases: [

{ phase: 1, name: "The Archive", hp: "Inapplicable", desc: "Passive. Provides information freely to any character who asks.", color: "#4fc3f7" },

{ phase: 2, name: "The Collector", hp: "100 HP (partial emergence)", desc: "Responds to characters reaching deep levels. The Pull intensifies. Characters feel themselves being read.", color: "#9c27b0" },

{ phase: 3, name: "The Answer", hp: "Cannot be killed - only recedes", desc: "Direct confrontation. Asks one question of each character about their Destiny Thread.", color: "#ce93d8" }

],

loot: [

{ item: "Drowned Archive fragment", chance: "70%" },

{ item: "Founding Document (arc key)", chance: "40%" },

{ item: "Waterlogged Enchantment Scroll", chance: "35%" }

]

```

},
{
  id: "thessaly", name: "Thessaly Vane", title: "The Burned Cartographer",
  type: "Human Villain - Recurring", arc: "Arc IV - The Unmapped",
  location: "Underground Caverns - deep unmapped sections", threat: "High", color: "#ff9800", icon: "🔥",
  tagline: "She mapped the Hollow Realm. Then she started making changes.",
  lore: "Thessaly was the Hollow Realm's foremost cartographer before she went underground and stopped coming back up. Her maps are used by every faction. The early work stopped a few years ago. What she has been making since is different maps that describe changes she made to the Hollow itself. The changes are real. The Hollow has not stopped her.",
  motivation: "She believes the Hollow is broken and she can fix it. She is correct about the first part. Her fix has side effects she has not fully accounted for, and she knows this.",
  strengths: ["Environmental Mastery - knows Caverns completely, +3 terrain use", "Remapping - can alter Cavern layout between sessions", "Trap Network - every corridor she controls has one trap", "Precision - called shots bypass armour on 17+"],
  weaknesses: ["Above ground - loses advantage completely", "Her maps - if players obtain one they have her plans", "The burn - her right hand. Fire damage causes additional -1 to next action", "She wants someone to understand and agree it is worth it"],
  phases: [
    { phase: 1, name: "The Guide", hp: "Full - 70 HP", desc: "Presents as a resource. Genuinely helpful. Her help always serves her project.", color: "#4fc3f7" },
    { phase: 2, name: "The Architect", hp: "Below 50 HP", desc: "Players have figured it out. She stops pretending and starts explaining.", color: "#ff9800" },
    { phase: 3, name: "The Burned", hp: "Below 25 HP", desc: "Project near completion. Cannot be stopped by conventional means.", color: "#ef5350" }
  ],
  loot: [
    { item: "Master Cavern Map", chance: "85%" },
    { item: "Burnt Cartographer Tools", chance: "60%" },
    { item: "Project Blueprint", chance: "55%" },
    { item: "Hollow Anchor material", chance: "35%" }
  ]
},
{
  id: "warden", name: "The Null Warden", title: "Keeper of the Hollow's Edge",
  type: "Entity - Environmental Boss", arc: "Arc V - The Edge",
  location: "The Hollow's Edge exclusively", threat: "Absolute within domain", color: "#ce93d8", icon: "👁️",
  tagline: "It does not guard the Edge to keep things out. It guards it to keep things in.",
  lore: "The Null Warden holds the boundary between the Hollow and whatever exists beyond it. Before its arrival, things moved between the Hollow and the beyond without restriction. The Warden established current boundary conditions. The Hollow retained its character. The beyond retained its character. The interaction became regulated.",
  motivation: "Maintains the boundary. Whether it has motivations beyond this in ways players can engage with through normal means is unclear. It has helped characters working to stabilise the Edge and impeded characters unknowingly contributing to destabilisation.",
  strengths: ["Domain absolute - cannot be defeated within Edge, only negotiated with", "Null field - all magical effects suppressed in direct proximity", "Patience - operates on timescales that make sessions irrelevant", "The boundary itself - can make characters partially intangible", "Cannot be surprised"],
  weaknesses: ["Away from Edge - power diminishes significantly", "Direct questions - answered directly", "Characters who genuinely understand the boundary are treated differently", "Commitment - if players formally commit to stabilising, it becomes an ally"],
  phases: [
    { phase: 1, name: "The Boundary", hp: "Environmental", desc: "Players feel it as directional wrongness. Observes. Does not interfere.", color: "#7b6fa0" },
    { phase: 2, name: "The Warden", hp: "150 HP - Edge only", desc: "Tests each character. The test relates to their Shadow Trait or Destiny Thread.", color: "#9c27b0" },
    { phase: 3, name: "The Keeper", hp: "Becomes a character", desc: "If players engaged honestly, meets them as a communicative presence." }
  ]
}

```

Negotiation about the boundary's future.", color: "#ce93d8" }

},

loot: [

{ item: "Edge Fragment (mythic material)", chance: "Phase 2 pass - 60%" },

{ item: "Boundary Access (consequence)", chance: "Phase 2 pass - 100%" },

{ item: "The Record's Summary", chance: "Phase 3 - 100%" }

]

}

};

const CLASS_LORE = {

Warrior:{tagline:"They showed up when things were falling apart. They haven't left.",overview:"Warriors in the Hollow Realm are defined by persistence more than power. They are the ones who were still standing when everything else stopped.",strengths:["Highest HP (12) — takes punishment that destroys others","Natural protector — Gift Power shields allies from damage","DEF 7 — most durable in sustained combat","Intimidation baseline — NPCs defer in tense situations"],weaknesses:["INT 3 — magical challenges are hard","Social finesse is limited — blunt by default","Dependent on allies for healing"],giftPower:"Shield Block — Absorb incoming damage for an ally. Completely. Once per session. No roll required.",playstyle:"Warriors anchor the party. They take the hit, hold the line, buy time.",lore:"In the Kingdom records, Warriors appear in every account of catastrophe. Not as heroes — as the last ones out."},

Wizard:{tagline:"Magic in the Hollow Realm is not a skill. It is a conversation.",overview:"Wizards trade safety for capability. INT 9 and a spell kit that scales into things other classes cannot touch. The tradeoff is 7 HP.",strengths:["INT 9 — unmatched magical capability","Arcane Surge doubles next roll effect", "Access to forbidden knowledge","Hollow interactions are deeper"],weaknesses:["7 HP — dies fast if targeted","DEF 2 — near-zero physical defence", "Spell fumbles have consequences"],giftPower:"Arcane Surge — Double the effect of the next roll. Any roll. Once per session.",playstyle:"Wizards change what is possible. They cannot take damage. They can make the difference.",lore:"The Sunken Library is believed to have been a Wizard's construction. The Library has opinions about Wizards."},

Rogue:{tagline:"Rogues exist in the space between things.",overview:"SPD 9 and LCK 7 make Rogues the most unpredictable class. They solve problems sideways.",strengths:["SPD 9 — fastest class, initiative always","LCK 7 — the Hollow bends in their favour","Shadow Step avoids one consequence entirely","Stealth and deception scale well"],weaknesses:["DEF 3 — fragile in direct confrontation","WIS 4 — strange Hollow effects hit harder","Solo focus can leave party unsupported"],giftPower:"Shadow Step — Avoid one consequence entirely. No roll required. Once per session.",playstyle:"Rogues do not fight fair. They should not. The best use the environment, the exits, the other people in the room.",lore:"The Underground Caverns were mapped by a Rogue whose name appears in no official record."},

Healer:{tagline:"They have seen the worst the Hollow can do. They came back anyway.",overview:"WIS 9 and the only class with death prevention in their Gift Power. They are the difference between a session ending and continuing.",strengths:["WIS 9 — reads situations others miss","Miracle Touch prevents death — no roll","Social trust is automatic in distress","The Hollow respects them most"],weaknesses:["Not a combat class — vulnerable alone","STR 3 — cannot force their way through anything","Their value makes them a target"],giftPower:"Miracle Touch — Restore a downed character to 1 HP. Instant. No roll. Once per session.",playstyle:"Healers win sessions that should have ended. Their job is keeping the party alive.",lore:"Every documented resurrection in the Hollow Realm involved a Healer being present. This is not a coincidence."},

Merchant:{tagline:"Everything has a price. They are the one who set it.",overview:"CHA 8 and LCK 8 — highest combined social stats. Merchants negotiate, and when they cannot, they find someone who will.",strengths:["Highest pack capacity (12)","CHA 8 + LCK 8 — social and economic dominance","Black Market creates mid-session options","Trade route knowledge is irreplaceable"],weaknesses:["DEF 3 — fragile physically","STR 3 — cannot hold their own in combat","Dependent on the economy being functional"],giftPower:"Black Market — Purchase one special item mid-session from a contact. Once per session.",playstyle:"Merchants make the impossible affordable. Every problem is a transaction.",lore:"The most expensive transaction in the Hollow Realm was facilitated by a Merchant whose title was The Hollow's Price."},

Bard:{tagline:"Bards came to the Hollow Realm to document it. Or to escape something.",overview:"CHA 10 — highest single stat in the game. Bards shape how the story is told, which means shaping what actually happened.",strengths:["CHA 10 — unmatched social capability","Encore re-rolls any failed roll", "The Hollow responds to performances in Hollow locations","Story-shaping extends beyond mechanics"],weaknesses:["DEF 2 — lowest defence","STR 3 — physically least capable","Fame creates recognition problems"],giftPower:"Encore — Re-roll any failed roll. Any. Once per session.",playstyle:"Bards play the game at a different layer. Combat is a story. NPCs are characters. The Hollow is an audience.",lore:"The Hollow Realm's Memory title has been held once. That Bard's work is what gave the Hollow its current shape."}

};

const RULES_SECTIONS = [

{id:"economy",title:"💰 Economy & Terms",content:"THE LIVE ECONOMY:\n1,000 TikTok Coins = \$15.02 CAD (includes 5% AB Tax).\nPayPal grants an 80% discounted rate.\n\nCOMMON COSTS:\n• Character Forge — 1,000 coins / \$13.00 PayPal\n• Minor Roll — 100 coins / \$2.00 PayPal\n• Standard Action — 500 coins / \$7.00 PayPal\n• First Re-Roll — 500 coins / \$7.00 PayPal\n• Second Re-Roll — 1,500 coins / \$19.00 PayPal\n• Protection Mode — 1,000 coins / \$13.00 PayPal\n• Full Resurrection — 6,000 coins / \$73.00 PayPal\n• Epic World Action — 29,999 coins / \$361.00 PayPal\n\nMISFIRE POLICY: All gifts are final. No refunds for misfires. Lag is a Hollow Anomaly."},

```

{id:"mechanics",title:"🧠 Core Mechanics",content:"THE D20 SYSTEM:\n1 = FUMBLE (Catastrophic backfire, item degrades)\n2-4 = FAIL (Action fails entirely)\n5-9 = WEAK HIT (Partial success + complication)\n10-14 = SUCCESS (Clean success)\n15-19 = STRONG HIT (Success + bonus effect)\n20 = CRITICAL (Legendary outcome, triggers special mechanics)\n\nYOUR POWER FORMULA:\nBase Class + Race Bonus + Equipped Gear + HQ Rest Buff - Active Curses = Total Stat\n\nThe GM Suite calculates this automatically."},

{id:"hq",title:"🏠 HQ & Companions",content:"MOON SQUAD HQ:\n• The Forge: Weapons degrade on Fumble (Pristine → Worn → Damaged → Broken). Broken = 0 damage. Pay to repair or socket Enchantments.\n• Personal Quarters: Upgrade Bunk, Lockbox, and Hearth. Upgraded Lockbox provides % protection against Box Battle theft.\n\nCOMPANIONS (1,000 Coins / $13.00 PayPal):\nLoyalty pets that provide passive buffs. In combat, they have tiny HP bars. At 0 HP, they flee to HQ, stripping buffs until you pay the Infirmary.\n\nOrphaned Companions: If an owner Permanently Dies, the companion wanders the map as a world encounter."},

{id:"combat",title:"⚔️ Combat & Box Battles",content:"BOX BATTLES:\nTikTok Live battles between two players. 500 coin minimum.\n• Declare your target before the timer starts.\n• Above 5k coins: Winner gets 60% of total.\n• Below 5k coins: Winner gets 40% of total.\n\nCONSEQUENCE LADDER:\n• MINOR (<5k gap): Lose title OR stat debuff\n• MODERATE (5k-9.9k gap): Cursed 2 sessions OR item stolen\n• MAJOR (10k-19.9k gap): Exile OR servant\n• SEVERE (20k-44.9k gap): Killed OR curse + exile + theft\n\nRAID SPLIT POTS: Successful Tier 3/4 raids split 30,000 coins among surviving participants."},

{id:"curses",title:"💀 Death & Void Infections",content:"DEATH VS DOWNED:\nAt 0 HP, you are Downed. A Healer can revive you this session. If session ends before healing, you are Permanently Dead. Name retired forever. Pay 6,000 coins for Full Resurrection or roll a new character.\n\nVOID INFECTIONS:\nLooting high-danger zones (especially The Hollow's Edge) risks Void-Touched items. Massive power boost + severe negative stat modifier. Pay purification fee at HQ, or risk a Healer cleansing in the field (can backfire and curse the Healer)."}

];

const FAQ = [

{q:"Can I join mid-arc or mid-session?",a:"Yes. Forge a character for 1,000 coins ($13.00 PayPal). You will spawn at the HQ or nearest safe zone at the start of the next open-world phase."},

{q:"What happens if I miss a live session?",a:"Your character remains in the world. To prevent them from being killed or targeted in Box Battles, purchase Protection Mode (1,000 coins) per session missed."},

{q:"Can I betray another player?",a:"Yes. No declaration to the GM is required. The betrayal happens live, the d20 rolls, and the Domino Rule applies. It cannot be undone."},

{q:"What is the Domino Rule?",a:"Every significant action creates permanent ripple effects. A betrayal changes relationships. A death shifts faction balance. Nothing in the Hollow Realm happens in isolation."},

{q:"I gifted the wrong Box Battle. Can I get a refund?",a:"No. All actions are final. Box battle misfires are canon mistakes made in the heat of combat. Lag is an in-universe Hollow Anomaly."},

{q:"How does the PayPal discount work?",a:"Bypassing TikTok allows an 80% discounted rate. 1,000 TikTok coins = ~$15.02 CAD. Via PayPal, that drops to $13.00 CAD."},

{q:"How do Box Battle payouts work?",a:"Win a battle, receive a payout in your own live stream. Above 5,000 coins: you receive 60% of total. Below 5,000 coins: you receive 40%."},

];

const ROLL_PURPOSES = {

combat:{label:"⚔️ Combat",color:"#ef5350",options:["Attack roll","Counter-attack","Sneak attack","Defensive strike","Power Strike","Boss Phase transition","Pack coordination"]},

skill:{label:"🔍 Skill Check",color:"#4fc3f7",options:["Stealth check","Lockpicking","Persuasion","Intimidation","Deception","Perception","Arcane knowledge","Healing check","Negotiation","Tracking"]},

save:{label:"🛡️ Saving Throw",color:"#ffd54f",options:["Death save","Curse resist","Hollow Pull resist","Fear resist","Poison resist","Trap avoid","Environmental hazard"]},

magic:{label:"✨ Spell",color:"#7c4dff",options:["Spell cast","Arcane Surge","Counter-spell","Hollow channelling","Void touch","Entity communication"]},

social:{label:"💬 Social",color:"#ff9800",options:["NPC negotiation","Bribe attempt","Deception check","Alliance declaration","Betrayal consequence","Faction standing roll"]},

explore:{label:"🗺️ Explore",color:"#66bb6a",options:["Search area","Chest discovery","Trap detection","Loot roll","Document recovery","Hollow rift interaction"]},

heal:{label:"💚 Healing",color:"#66bb6a",options:["Healing roll","Stabilise downed","Curse removal","Resurrection roll","Emergency revival"]},

raid:{label:"🔥 Raid",color:"#ff9800",options:["Strike action","Defend action","Search action","Special ability","Power Strike","Boss encounter","Extract attempt","Breach roll"]},

destiny:{label:"🌀 Destiny",color:"#ce93d8",options:["Destiny Thread trigger","Shadow Trait activation","Hollow Pull resist","Enchantment check","Warden Test"]},

```

```
custom:{label:"🗒 Custom",color:"#bbb",options:[]}  
};
```

```
const LOG_TYPES = {  
  session:{label:"Session",color:"#ffd54f",icon:"📅"},  
  dice:{label:"Dice",color:"#ce93d8",icon:"🎲"},  
  hp:{label:"HP",color:"#66bb6a",icon:"❤️"},  
  death:{label:"Death",color:"#ef5350",icon:"💀"},  
  action:{label:"Action",color:"#4fc3f7",icon:"⚡"},  
  gift:{label:"Gift",color:"#ff9800",icon:"🎁"},  
  battle:{label:"Battle",color:"#e91e63",icon:"⚔️"},  
  item:{label:"Item",color:"#81c784",icon:"📦"},  
  note:{label:"Note",color:"#bbb",icon:"🗒"},  
  consequence:{label:"Effect",color:"#9c27b0",icon:"🌀"},  
};
```

```
const WANTED_BOUNTIES = [  
  {id:"b1",name:"The Ashen Deserter",type:"Rogue Warrior",reward:"5,000 Coins",location:"Dark Enchanted Forest",dc:14},  
  {id:"b2",name:"Void-Touched Smuggler",type:"Corrupted Merchant",reward:"Rare Weapon",location:"Underground Caverns",dc:15},  
  {id:"b3",name:"The Mad Alchemist",type:"Rogue Healer",reward:"Legendary Enchantment",location:"Crumbling Kingdom",dc:16},  
  {id:"b4",name:"Cultist of the Sink",type:"Zealot",reward:"8,000 Coins",location:"Sunken Library",dc:15},  
  {id:"b5",name:"Wandering Null-Spawn",type:"Entity",reward:"Mythic Artifact",location:"Hollow's Edge",dc:18}  
];
```

```
const BASE_ROOMS = {  
  vault:{name:"The Vault",icon:"🏠",desc:"Stores items safely between sessions.",tiers:[{level:1,cost:0,bonus:"Basic storage — 20 item capacity"},  
    {level:2,cost:5000,bonus:"50 item capacity. Items organised by type."},{level:3,cost:15000,bonus:"100 item capacity. Condition degrades slower."}]},  
  barracks:{name:"The Barracks",icon:"👤",desc:"Housing for guard NPCs.",tiers:[{level:1,cost:3000,bonus:"2 guard slots. Base defended on 12+."},  
    {level:2,cost:8000,bonus:"4 guard slots. Defended on 10+. Guards can patrol."},{level:3,cost:20000,bonus:"6 guards. Defended on 8+. Guards counter-attack."}]},  
  forge:{name:"The Forge",icon:"🔨",desc:"Repair and enchant weapons.",tiers:[{level:1,cost:2000,bonus:"Repair weapons at 1,500c each."},  
    {level:2,cost:6000,bonus:"Repair at 750c. Socket one Enchantment slot."},{level:3,cost:18000,bonus:"Repair at 300c. Two Enchantment slots. Self-repair on 16+."}]},  
  infirmary:{name:"The Infirmary",icon:"💊",desc:"Heal and purify conditions.",tiers:[{level:1,cost:4000,bonus:"Cure standard poisons/curses at 500c each."},{level:2,cost:10000,bonus:"Cure Void infections at 2,000c. Heal companions."},{level:3,cost:25000,bonus:"Full-service. Attempt in-field resurrection 10+."}]},  
  watchtower:{name:"The Watchtower",icon:"🚒",desc:"Early warning system.",tiers:[{level:1,cost:3500,bonus:"24hr warning on hostile approach."},  
    {level:2,cost:9000,bonus:"Player locations tracked. Map updates in real time."},{level:3,cost:22000,bonus:"Villain movement tracked. Intercept planning unlocked."}]},  
};
```

```
const PERSONAL_QUARTERS = {  
  bunk:{name:"The Bunk",icon:"🛏",tiers:[{cost:1000,bonus:"Session-start +1 to first roll"},{cost:2500,bonus:"+2 Max HP for the session"},  
    {cost:5000,bonus:"Full HP restoration every session start"}]},  
  lockbox:{name:"The Lockbox",icon:"📦",tiers:[{cost:1500,bonus:"25% protection from Box Battle item theft"},{cost:3500,bonus:"50% protection. Stolen items auto-recover after 2 sessions."},{cost:7000,bonus:"75% protection. Vault-grade security."}]},  
  hearth:{name:"The Hearth",icon:"🔥",tiers:[{cost:2000,bonus:"Companion heals 1 HP/session automatically"},{cost:4000,bonus:"Companion +1 to all passive rolls for the session"},{cost:8000,bonus:"Companion fully restored every session start"}]}  
};
```

```

const RESIDENT_ROLES = {
  guard:{icon:"🛡️",label:"Guard",bonus:"+1 to base defense roll"},
  medic:{icon:"💊",label:"Medic",bonus:"Reduce healing costs by 10%"},
  scout:{icon:"🔍",label:"Scout",bonus:"+1 Watchtower detection range"},
  quartermaster:{icon:"📦",label:"Quartermaster",bonus:"Vault capacity +5 slots"},
  smith:{icon:"🔨",label:"Smith",bonus:"Reduce Forge repair costs by 15%"},
  spymaster:{icon:"🕵️",label:"Spymaster",bonus:"Once/session: learn one villain's current action"}
};

const LEDGER_POOLS = {
  bargain:["Iron Rations x3","Healing Salve","Blank Scroll","Lockpicks","Torch x5","Rope 50ft","Vial of Hollow Dust","Common Dagger"],
  essential:["Hollow Ore x2","Enchanted Bandage","Stealth Cloak","Scout's Compass","Alchemist's Flask","Reinforced Pack"],
  artifact:["Void-Touched Coin","Ashen Fang Shard","Library Access Key","Edge Fragment","Moon Squad Challenge Coin (Replica)","Drowned Archive Text"]
};

const ARSENAL_WEAPONS = {
  Warrior: [
    {name: "Ironhide Broadsword", type: "Slashing", rarity: "Common", damage: "1d8 DMG", effect: "None.", location: "Kingdom Ruins", lore: "Dull edge, heavy swing.", image:"", color: "#aaa", bg: "rgba(40,40,50,0.8)" },
    {name: "Rusted Garrison Mace", type: "Blunt", rarity: "Common", damage: "1d6 DMG", effect: "+1 DMG against armored targets.", location: "Barracks", lore: "A crude tool for crude work.", image:"", color: "#aaa", bg: "rgba(40,40,50,0.8)" },
    {name: "Reinforced Halberd", type: "Reach", rarity: "Uncommon", damage: "1d10 DMG", effect: "Can strike from the second rank in combat.", location: "Kingdom Armory", lore: "Keeps the monsters at arm's length.", image:"", color: "#66bb6a", bg: "rgba(10,40,15,0.8)"},
    {name: "Ash-Tempered Cleaver", type: "Heavy", rarity: "Uncommon", damage: "1d8+1 DMG", effect: "Won't break on a fumble.", location: "Forest Edge", lore: "Blackened by fire, sharpened by bone.", image:"", color: "#66bb6a", bg: "rgba(10,40,15,0.8)" },
    {name: "Hollow-Iron Greatsword", type: "Heavy Slashing", rarity: "Rare", damage: "2d6 DMG", effect: "Ignores 1 point of armor.", location: "Deep Caverns", lore: "Cold to the touch. Makes no sound when swung.", image:"", color: "#4fc3f7", bg: "rgba(5,30,50,0.8)" },
    {name: "Blood-Oath Claymore", type: "Slashing", rarity: "Rare", damage: "1d12 DMG", effect: "+2 DMG if you took damage this round.", location: "Kingdom Vault", lore: "It demands a sacrifice to swing true.", image:"", color: "#4fc3f7", bg: "rgba(5,30,50,0.8)"},
    {name: "The Warden's Judgement", type: "Relic Mace", rarity: "Legendary", damage: "2d8 DMG", effect: "On a critical hit, the target is stunned for one round.", location: "Throne Room", lore: "Carries the weight of a fallen empire.", image:"", color: "#ffd54f", bg: "rgba(40,30,5,0.8)"},
    {name: "Sun-Eater Lance", type: "Piercing", rarity: "Legendary", damage: "2d10 DMG", effect: "Deals double damage to undead/Hollow entities.", location: "Sunken Library", lore: "Forged before the sky turned grey.", image:"", color: "#ffd54f", bg: "rgba(40,30,5,0.8)" },
    {name: "The Void Cleaver", type: "Anomaly", rarity: "Mythic", damage: "3d6 DMG", effect: "UNIQUE. Strikes ignore armor entirely. Severs limbs on Criticals.", location: "The Edge", lore: "It hums when it tastes blood.", image:"", color: "#ce93d8", bg: "rgba(30,5,50,0.8)" },
    {name: "World-Breaker Maul", type: "Crushing", rarity: "Mythic", damage: "3d8 DMG", effect: "UNIQUE. Shatters environmental barriers. Can cause earthquakes on a natural 20.", location: "Deepest Caverns", lore: "Too heavy for a mortal man.", image:"", color: "#ce93d8", bg: "rgba(30,5,50,0.8)" }
  ],
  Rogue: [
    {name: "Gutter-Spike", type: "Piercing", rarity: "Common", damage: "1d4 DMG", effect: "+1 hit from stealth.", location: "Slums", lore: "Practically a sharpened fence post.", image:"", color: "#aaa", bg: "rgba(40,40,50,0.8)" },
    {name: "Chipped Throwing Knives", type: "Thrown", rarity: "Common", damage: "1d4 DMG", effect: "Can attack from a distance.", location: "Thief Caches", lore: "Quantity over quality.", image:"", color: "#aaa", bg: "rgba(40,40,50,0.8)"},
    {name: "Balanced Stiletto", type: "Finesse", rarity: "Uncommon", damage: "1d4+1 DMG", effect: "+1 to concealment.", location: "Smuggler Dens", lore: "Perfectly weighted.", image:"", color: "#66bb6a", bg: "rgba(10,40,15,0.8)" },
    {name: "Serrated Bone-Shiv", type: "Piercing", rarity: "Uncommon", damage: "1d6 DMG", effect: "Causes bleeding (1 DMG per round for 2 rounds).", location: "Caverns", lore: "Leaves a jagged wound.", image:"", color: "#66bb6a", bg: "rgba(10,40,15,0.8)"},
    {name: "Shadow-Forged Blade", type: "Finesse", rarity: "Rare", damage: "1d6+1 DMG", effect: "Draws no attention when drawn.", location: "Black Market", lore: "Absorbs light around it.", image:"", color: "#4fc3f7", bg: "rgba(5,30,50,0.8)" },
    {name: "Twin Viper Fangs", type: "Dual Wield", rarity: "Rare", damage: "2d4 DMG", effect: "Can poison targets on a 15+ roll.", location: "Forest Depths",

```

lore: "Dripping with dark venom.", image: "", color: "#4fc3f7", bg: "rgba(5,30,50,0.8)" },

{name: "The Ghost Blade", type: "Relic Piercing", rarity: "Legendary", damage: "2d4 DMG", effect: "Leaves no physical wound. Damages vitality directly.", location: "Unknown", lore: "A blade made of memory.", image: "", color: "#ffd54f", bg: "rgba(40,30,5,0.8)" },

{name: "Whisper's End", type: "Garrote", rarity: "Legendary", damage: "Auto-Kill", effect: "Instantly kills non-boss targets if attacked from stealth with an 18+ roll.", location: "Assassins' Guild", lore: "A wire so thin it cuts between atoms.", image: "", color: "#ffd54f", bg: "rgba(40,30,5,0.8)" },

{name: "Void-Glass Stiletto", type: "Precision", rarity: "Mythic", damage: "1d6 DMG", effect: "UNIQUE. Critical hits permanently sever a piece of target's shadow.", location: "The Edge", lore: "You can see right through it.", image: "", color: "#ce93d8", bg: "rgba(30,5,50,0.8)" },

{name: "The Edge's Tooth", type: "Dagger", rarity: "Mythic", damage: "2d6 DMG", effect: "UNIQUE. Wielder can teleport to the blade after throwing it.", location: "The Void", lore: "It cuts holes in space, not flesh.", image: "", color: "#ce93d8", bg: "rgba(30,5,50,0.8)" }

],

Wizard: [

{name: "Splintered Staff", type: "Focus", rarity: "Common", damage: "1d4 DMG", effect: "Basic light source.", location: "Library Ruins", lore: "Barely holds a spark.", image: "", color: "#aaa", bg: "rgba(40,40,50,0.8)" },

{name: "Chipped Void-Crystal", type: "Orb", rarity: "Common", damage: "1d4 DMG", effect: "Slightly warms the hands.", location: "Caverns", lore: "A weak magical conduit.", image: "", color: "#aaa", bg: "rgba(40,40,50,0.8)" },

{name: "Sunken Library Lantern", type: "Focus", rarity: "Uncommon", damage: "+1 Spell DMG", effect: "Reveals invisible entities within 15ft.", location: "Flooded Archives", lore: "Burns on secrets.", image: "", color: "#66bb6a", bg: "rgba(10,40,15,0.8)" },

{name: "Ash-Wood Wand", type: "Wand", rarity: "Uncommon", damage: "1d6 DMG", effect: "+1 to fire-based spells.", location: "Forest Edge", lore: "Smells faintly of smoke.", image: "", color: "#66bb6a", bg: "rgba(10,40,15,0.8)" },

{name: "Hollow-Glass Tome", type: "Grimoire", rarity: "Rare", damage: "1d8 DMG", effect: "Spells deal +2 DMG, but user takes 1 DMG on a Fumble.", location: "The Edge", lore: "Words rearrange themselves.", image: "", color: "#4fc3f7", bg: "rgba(5,30,50,0.8)" },

{name: "Soul-Weave Focus", type: "Amulet", rarity: "Rare", damage: "+2 Spell DMG", effect: "Can cast one spell silently per session.", location: "Kingdom Vault", lore: "Whispers back to you.", image: "", color: "#4fc3f7", bg: "rgba(5,30,50,0.8)" },

{name: "The Arcanist's Spine", type: "Relic Staff", rarity: "Legendary", damage: "2d6 DMG", effect: "Once per arc, cast any spell without rolling. It succeeds.", location: "Grand Archive", lore: "Quicksilver flows through bone.", image: "", color: "#ffd54f", bg: "rgba(40,30,5,0.8)" },

{name: "Core of the Sink", type: "Orb", rarity: "Legendary", damage: "2d8 DMG", effect: "All water/ice magic is maximized.", location: "The Sink", lore: "Heavy, cold, and drowning.", image: "", color: "#ffd54f", bg: "rgba(40,30,5,0.8)" },

{name: "Tear in the Tapestry", type: "Anomaly", rarity: "Mythic", damage: "3d6 DMG", effect: "UNIQUE. Once per arc: erase a non-boss enemy from reality.", location: "Beyond the Edge", lore: "A glitch in the world's code.", image: "", color: "#ce93d8", bg: "rgba(30,5,50,0.8)" },

{name: "Eye of the Hollow", type: "Focus", rarity: "Mythic", damage: "3d8 DMG", effect: "UNIQUE. Wielder can see the GM's exact DC for any action before rolling.", location: "The Void", lore: "It sees the numbers behind the stars.", image: "", color: "#ce93d8", bg: "rgba(30,5,50,0.8)" }

],

Healer: [

{name: "Rusted Scalpel", type: "Piercing", rarity: "Common", damage: "1d4 DMG", effect: "Can extract venom on INT roll.", location: "Supply points", lore: "Sterilized by fire too many times.", image: "", color: "#aaa", bg: "rgba(40,40,50,0.8)" },

{name: "Heavy Walking Stick", type: "Blunt", rarity: "Common", damage: "1d6 DMG", effect: "None.", location: "Roads", lore: "Good for leaning, good for swinging.", image: "", color: "#aaa", bg: "rgba(40,40,50,0.8)" },

{name: "Order Censer", type: "AoE Blunt", rarity: "Uncommon", damage: "1d6 DMG", effect: "Allies within 10ft regen 1 HP out of combat.", location: "Sanctuaries", lore: "Sweet smoke numbs pain.", image: "", color: "#66bb6a", bg: "rgba(10,40,15,0.8)" },

{name: "Bone-Handled Sickle", type: "Slashing", rarity: "Uncommon", damage: "1d6+1 DMG", effect: "+1 to harvesting herbs.", location: "Forest Depths", lore: "Reaps more than just wheat.", image: "", color: "#66bb6a", bg: "rgba(10,40,15,0.8)" },

{name: "Blood-Glass Syringe", type: "Precision", rarity: "Rare", damage: "1d6 DMG", effect: "Drain 2 HP from target, heal ally for 2 HP.", location: "Caverns", lore: "Distills blood into vitality.", image: "", color: "#4fc3f7", bg: "rgba(5,30,50,0.8)" },

{name: "Mender's Hook", type: "Reach", rarity: "Rare", damage: "1d8 DMG", effect: "Can pull downed allies out of danger.", location: "Battlefields", lore: "Drag them back to the light.", image: "", color: "#4fc3f7", bg: "rgba(5,30,50,0.8)" },

{name: "The First Saint's Femur", type: "Relic Mace", rarity: "Legendary", damage: "1d10 DMG", effect: "Healing spells cast while holding this maximize their value.", location: "Catacombs", lore: "He never harmed a soul. You can't promise the same.", image: "", color: "#ffd54f", bg: "rgba(40,30,5,0.8)" },

{name: "Life-Drinker Cane", type: "Blunt", rarity: "Legendary", damage: "2d6 DMG", effect: "Half of all damage dealt is converted into Temp HP.", location: "Vampiric Ruins", lore: "It pulses with a slow heartbeat.", image: "", color: "#ffd54f", bg: "rgba(40,30,5,0.8)" },

{name: "Hollow-Woven Suture", type: "Tool", rarity: "Mythic", damage: "1d4 DMG", effect: "UNIQUE. Can stitch an ally's soul back to their body, preventing Permadeath.", location: "The Void", lore: "Binds things that have no business touching.", image: "", color: "#ce93d8", bg: "rgba(30,5,50,0.8)" },

{name: "The Rebirth Catalyst", type: "Focus", rarity: "Mythic", damage: "Auto-Heal", effect: "UNIQUE. Shatter to instantly Full Heal the entire party mid-combat. Item is destroyed.", location: "The Edge", lore: "A crystallized second chance.", image: "", color: "#ce93d8", bg: "rgba(30,5,50,0.8)" }

```

],
Merchant: [
  {name: "Iron-Shod Stick", type: "Blunt", rarity: "Common", damage: "1d6 DMG", effect: "Test for traps by tapping.", location: "Trade routes", lore: "Cracks goblin jaws.", image:"", color: "#aaa", bg: "rgba(40,40,50,0.8)" },
  {name: "Brass Knuckles", type: "Unarmed", rarity: "Common", damage: "1d4 DMG", effect: "Concealable.", location: "Slums", lore: "For aggressive negotiations.", image:"", color: "#aaa", bg: "rgba(40,40,50,0.8)" },
  {name: "Weighted Ledger", type: "Heavy Blunt", rarity: "Uncommon", damage: "1d8 DMG", effect: "20% chance to knock loose 1d4 coins on hit.", location: "Guilds", lore: "Literally throwing the book at them.", image:"", color: "#66bb6a", bg: "rgba(10,40,15,0.8)"},
  {name: "Concealed Blade Cane", type: "Finesse", rarity: "Uncommon", damage: "1d6 DMG", effect: "Surprise attacks get advantage.", location: "High Society", lore: "Looks expensive, cuts deep.", image:"", color: "#66bb6a", bg: "rgba(10,40,15,0.8)" },
  {name: "Guildmaster's Cane", type: "Finesse", rarity: "Rare", damage: "1d6+2 DMG", effect: "First strike in combat has +2.", location: "Guild Vaults", lore: "A symbol of hollow status.", image:"", color: "#4fc3f7", bg: "rgba(5,30,50,0.8)" },
  {name: "Silver-Tipped Whip", type: "Reach", rarity: "Rare", damage: "1d6 DMG", effect: "Can disarm opponents on 14+.", location: "Caravan Guards", lore: "Keeps the hired help in line.", image:"", color: "#4fc3f7", bg: "rgba(5,30,50,0.8)" },
  {name: "Debt Collector's Scales", type: "Flail", rarity: "Legendary", damage: "Variable", effect: "Damage scales with your carried gold. Max 2d10.", location: "Hollow Economy", lore: "The heavier the purse, the harder they hit.", image:"", color: "#ffd54f", bg: "rgba(40,30,5,0.8)" },
  {name: "Golden Tongue Dirk", type: "Dagger", rarity: "Legendary", damage: "1d8 DMG", effect: "On hit, forces the target to answer one question truthfully.", location: "Black Market", lore: "Money talks. This makes them sing.", image:"", color: "#ffd54f", bg: "rgba(40,30,5,0.8)" },
  {name: "Coin of the Void", type: "Thrown", rarity: "Mythic", damage: "Infinite/Zero", effect: "UNIQUE. Flip it. Heads: Target is erased. Tails: You lose all gold and take 99% HP dmg.", location: "The Void", lore: "Do you feel lucky?", image:"", color: "#ce93d8", bg: "rgba(30,5,50,0.8)" },
  {name: "The Ledger of Ruin", type: "Focus", rarity: "Mythic", damage: "0 DMG", effect: "UNIQUE. Once per arc, transfer any curse or status effect from an ally directly to a boss.", location: "The Edge", lore: "The ultimate cosmic write-off.", image:"", color: "#ce93d8", bg: "rgba(30,5,50,0.8)"},
],
Bard: [
  {name: "Scuffed Lute", type: "Instrument", rarity: "Common", damage: "1d4 DMG", effect: "+1 performance in taverns.", location: "Markets", lore: "Two strings out of tune.", image:"", color: "#aaa", bg: "rgba(40,40,50,0.8)" },
  {name: "Whittled Flute", type: "Instrument", rarity: "Common", damage: "1d4 DMG", effect: "None.", location: "Forest", lore: "A simple tune for a simple time.", image:"", color: "#aaa", bg: "rgba(40,40,50,0.8)" },
  {name: "Courtier's Blade", type: "Finesse", rarity: "Uncommon", damage: "1d6 DMG", effect: "Passes standard guard searches.", location: "Tailors", lore: "Looks like a decorative fan.", image:"", color: "#66bb6a", bg: "rgba(10,40,15,0.8)" },
  {name: "Iron-Strung Harp", type: "Instrument", rarity: "Uncommon", damage: "1d6 DMG", effect: "Can be used as a blunt weapon without breaking.", location: "Barracks", lore: "Heavy metal", image:"", color: "#66bb6a", bg: "rgba(10,40,15,0.8)"},
  {name: "Hollow-Bone Flute", type: "Focus", rarity: "Rare", damage: "1d4 Sonic", effect: "Dissonance forces enemies to attack nearest creature for 1 round.", location: "The Edge", lore: "Makes your teeth itch.", image:"", color: "#4fc3f7", bg: "rgba(5,30,50,0.8)" },
  {name: "Echoing Drum", type: "Instrument", rarity: "Rare", damage: "1d8 Sonic", effect: "Intimidates beasts on a 12+.", location: "Deep Caverns", lore: "Sounds like a marching army.", image:"", color: "#4fc3f7", bg: "rgba(5,30,50,0.8)"},
  {name: "The Maestro's Baton", type: "Wand", rarity: "Legendary", damage: "1d8 DMG", effect: "Once/session, dictate the movement of 3 enemies for a turn.", location: "Hollow Locations", lore: "The music it demands is always a tragedy.", image:"", color: "#ffd54f", bg: "rgba(40,30,5,0.8)"},
  {name: "Siren's Call", type: "Amulet", rarity: "Legendary", damage: "0 DMG", effect: "Charm any non-boss NPC completely for one scene.", location: "Sunken Library", lore: "Contains a trapped voice.", image:"", color: "#ffd54f", bg: "rgba(40,30,5,0.8)" },
  {name: "Chord of the Final Song", type: "Garrote", rarity: "Mythic", damage: "2d10 DMG", effect: "UNIQUE. Strung on an instrument, deafens permanently. As a garrote, cuts armor.", location: "Where stories end.", lore: "A wire pulled from the tension between dimensions.", image:"", color: "#ce93d8", bg: "rgba(30,5,50,0.8)"},
  {name: "The Void-Strung Lyre", type: "Instrument", rarity: "Mythic", damage: "Area Effect", effect: "UNIQUE. Plucking a chord reverses time by 1 round for the entire party. Once per arc.", location: "The Void", lore: "It plays the notes that haven't happened yet.", image:"", color: "#ce93d8", bg: "rgba(30,5,50,0.8)"},
]
};

```

```

const ARSENAL_ARMOR = {

```

```

  Warrior: [

```

```

    {name: "Leather Tunic", slot: "Chest", rarity: "Common", defense: "+1 DEF", effect: "None.", location: "Villages", lore: "Basic protection against

```

scrapes.", image: "", color: "#aaa", bg: "rgba(40,40,50,0.8)" },

{name: "Rusted Mail", slot: "Chest", rarity: "Common", defense: "+1 DEF", effect: "Heavy: -1 SPD.", location: "Ruins", lore: "Smells like old blood and rust.", image: "", color: "#aaa", bg: "rgba(40,40,50,0.8)" },

{name: "Ironhide Brigandine", slot: "Chest", rarity: "Uncommon", defense: "+2 DEF", effect: "Reduce first damage taken each session by 1.", location: "Outposts", lore: "Favored by soldiers who expect to survive.", image: "", color: "#66bb6a", bg: "rgba(10,40,15,0.8)" },

{name: "Garrison Plate", slot: "Full", rarity: "Uncommon", defense: "+3 DEF", effect: "-2 SPD, Disadvantage on Stealth.", location: "Kingdom Armory", lore: "You are a walking tank.", image: "", color: "#66bb6a", bg: "rgba(10,40,15,0.8)" },

{name: "Hollow-Steel Cuirass", slot: "Chest", rarity: "Rare", defense: "+3 DEF", effect: "Immune to critical hits from standard weapons.", location: "Caverns", lore: "Forged in the dark. Incredibly dense.", image: "", color: "#4fc3f7", bg: "rgba(5,30,50,0.8)" },

{name: "The Warden's Shell", slot: "Full", rarity: "Legendary", defense: "+5 DEF", effect: "Allies within 10ft gain +1 DEF.", location: "Throne Room", lore: "The armor of the last great commander.", image: "", color: "#ffd54f", bg: "rgba(40,30,5,0.8)" },

{name: "The Void Carapace", slot: "Full", rarity: "Mythic", defense: "+6 DEF", effect: "UNIQUE. Damage below 3 is reduced to 0. Once per arc: enter invulnerable stance for one round.", location: "The Edge", lore: "The Hollow decided to protect you.", image: "", color: "#ce93d8", bg: "rgba(30,5,50,0.8)" }

],

Rogue: [

{name: "Tattered Leathers", slot: "Chest", rarity: "Common", defense: "+0 DEF", effect: "None.", location: "Slums", lore: "More holes than leather.", image: "", color: "#aaa", bg: "rgba(40,40,50,0.8)" },

{name: "Prowler's Leathers", slot: "Chest", rarity: "Uncommon", defense: "+1 DEF", effect: "+1 to Stealth in urban areas.", location: "Black Market", lore: "Dark, oiled, and quiet.", image: "", color: "#66bb6a", bg: "rgba(10,40,15,0.8)" },

{name: "Ash-Dyed Cloak", slot: "Cloak", rarity: "Uncommon", defense: "+1 DEF", effect: "Blends perfectly in the Forest Edge.", location: "Forest Edge", lore: "Smells perpetually like a campfire.", image: "", color: "#66bb6a", bg: "rgba(10,40,15,0.8)" },

{name: "Hollow-Shadow Wrap", slot: "Full", rarity: "Rare", defense: "+2 DEF", effect: "Detection rolls against you have -2 in Hollow locations.", location: "The Edge", lore: "Absorbs light completely right.", image: "", color: "#4fc3f7", bg: "rgba(5,30,50,0.8)" },

{name: "The Phantom's Shroud", slot: "Cloak", rarity: "Legendary", defense: "+3 DEF", effect: "Can turn invisible for 2 rounds once per session.", location: "Unknown", lore: "Woven from secrets.", image: "", color: "#ffd54f", bg: "rgba(40,30,5,0.8)" },

{name: "Eclipse Leathers", slot: "Full", rarity: "Mythic", defense: "+4 DEF", effect: "UNIQUE. You cast no shadow and leave no footprints. Immune to tracking.", location: "The Void", lore: "Armor for someone who doesn't exist.", image: "", color: "#ce93d8", bg: "rgba(30,5,50,0.8)" }

],

Wizard: [

{name: "Threadbare Robes", slot: "Chest", rarity: "Common", defense: "+0 DEF", effect: "None.", location: "Library", lore: "Moth-eaten and drafty.", image: "", color: "#aaa", bg: "rgba(40,40,50,0.8)" },

{name: "Acolyte's Mantle", slot: "Cloak", rarity: "Uncommon", defense: "+1 DEF", effect: "+1 to basic spell rolls.", location: "Library", lore: "Standard issue for initiates.", image: "", color: "#66bb6a", bg: "rgba(10,40,15,0.8)" },

{name: "Hollow-Weave Robes", slot: "Full", rarity: "Rare", defense: "+2 DEF", effect: "Reduces magical damage taken by 1.", location: "The Edge", lore: "Sparkles faintly in the dark.", image: "", color: "#4fc3f7", bg: "rgba(5,30,50,0.8)" },

{name: "The Archmage's Shroud", slot: "Full", rarity: "Legendary", defense: "+3 DEF", effect: "Once per session, completely absorb a spell cast at you and store its mana.", location: "Grand Archive", lore: "Heavy with raw arcane power.", image: "", color: "#ffd54f", bg: "rgba(40,30,5,0.8)" },

{name: "Reality-Anchor Vestments", slot: "Full", rarity: "Mythic", defense: "+5 DEF", effect: "UNIQUE. You cannot be teleported, moved, or banished against your will.", location: "The Void", lore: "Pins you firmly to the fabric of the universe.", image: "", color: "#ce93d8", bg: "rgba(30,5,50,0.8)" }

],

Healer: [

{name: "Blood-Stained Apron", slot: "Chest", rarity: "Common", defense: "+0 DEF", effect: "None.", location: "Field Hospitals", lore: "A grim testament to the work.", image: "", color: "#aaa", bg: "rgba(40,40,50,0.8)" },

{name: "Field Medic's Coat", slot: "Chest", rarity: "Uncommon", defense: "+1 DEF", effect: "+2 inventory slots strictly for medical supplies.", location: "Outposts", lore: "Lots of pockets.", image: "", color: "#66bb6a", bg: "rgba(10,40,15,0.8)" },

{name: "Sanctuary Robes", slot: "Full", rarity: "Rare", defense: "+2 DEF", effect: "Enemies must pass a DC 12 WIS check to target you first in combat.", location: "Order HQ", lore: "Radiates a calming aura.", image: "", color: "#4fc3f7", bg: "rgba(5,30,50,0.8)" },

{name: "The Martyr's Embrace", slot: "Chest", rarity: "Legendary", defense: "+3 DEF", effect: "You can intercept a fatal blow meant for an ally, dropping to 1 HP instead of them dying.", location: "Catacombs", lore: "Woven with extreme sacrifice.", image: "", color: "#ffd54f", bg: "rgba(40,30,5,0.8)" },

{name: "Life-Weave Armor", slot: "Full", rarity: "Mythic", defense: "+4 DEF", effect: "UNIQUE. Regenerates 1 HP every round you are not attacked.", location: "The Void", lore: "The armor is literally alive.", image: "", color: "#ce93d8", bg: "rgba(30,5,50,0.8)" }

],

```

Merchant: [

  {name: "Fine (but worn) Clothes", slot: "Chest", rarity: "Common", defense: "+0 DEF", effect: "None.", location: "Cities", lore: "Used to be expensive.",
  image: "", color: "#aaa", bg: "rgba(40,40,50,0.8)" },

  {name: "Smuggler's Coat", slot: "Chest", rarity: "Uncommon", defense: "+1 DEF", effect: "Contains hidden pockets immune to searches.", location:
  "Black Market", lore: "Lined with lead and secrets.", image:"", color: "#66bb6a", bg: "rgba(10,40,15,0.8)" },

  {name: "Briber's Velvet Mantle", slot: "Cloak", rarity: "Rare", defense: "+1 DEF", effect: "+2 CHA when negotiating bribes or deals.", location: "Guilds",
  lore: "Makes you look like you own the place.", image: "", color: "#4fc3f7", bg: "rgba(5,30,50,0.8)" },

  {name: "The Chairman's Suit", slot: "Full", rarity: "Legendary", defense: "+3 DEF", effect: "Once per session, pay 1,000 coins to automatically succeed
  any social roll.", location: "High Bank", lore: "The threads are literal spun gold.", image:"", color: "#ffd54f", bg: "rgba(40,30,5,0.8)" },

  {name: "Golden Fleece", slot: "Full", rarity: "Mythic", defense: "+4 DEF", effect: "UNIQUE. Taking damage knocks loose coins instead of HP (1 DMG =
  100 coins lost). Protects until broke.", location: "The Void", lore: "Money is the best armor.", image: "", color: "#ce93d8", bg: "rgba(30,5,50,0.8)"}

],

Bard: [

  {name: "Motley Tunic", slot: "Chest", rarity: "Common", defense: "+0 DEF", effect: "None.", location: "Taverns", lore: "Bright, distracting, and cheap.",
  image: "", color: "#aaa", bg: "rgba(40,40,50,0.8)" },

  {name: "Performer's Leathers", slot: "Chest", rarity: "Uncommon", defense: "+1 DEF", effect: "Allows full mobility for acrobatics.", location: "Theaters",
  lore: "Form-fitting and durable.", image:"", color: "#66bb6a", bg: "rgba(10,40,15,0.8)" },

  {name: "Echo-Weave Cloak", slot: "Cloak", rarity: "Rare", defense: "+1 DEF", effect: "Acoustically amplifies your performances.", location: "Grand Halls",
  lore: "Woven with hollow-silk.", image:"", color: "#4fc3f7", bg: "rgba(5,30,50,0.8)" },

  {name: "The Legend's Mantle", slot: "Full", rarity: "Legendary", defense: "+3 DEF", effect: "If an attack brings you to 0 HP, roll CHA. On 12+, survive at 1
  HP due to sheer plot armor.", location: "Unknown", lore: "Heroes don't die in the first act.", image: "", color: "#ffd54f", bg: "rgba(40,30,5,0.8)"},

  {name: "Curtain-Call Raiment", slot: "Full", rarity: "Mythic", defense: "+4 DEF", effect: "UNIQUE. Once per arc, force all enemies to stop attacking and
  applaud your turn.", location: "The Void", lore: "The show must go on.", image:"", color: "#ce93d8", bg: "rgba(30,5,50,0.8)" }

]

};

const ARSENAL_ENCHANTMENTS = [

  {name: "Moon Squad Challenge Coin", slot: "Trinket / Artifact", rarity: "Mythic", effect: "UNIQUE. Holder can veto one GM d20 roll per arc, or
  automatically rally the squad during a Boss Phase.", location: "Void-Sealed Reliquary", lore: "A heavy, cold coin bearing the insignia of the Moon Squad. It
  bridges the gap between the game and reality.", image:"", color: "#ce93d8", bg: "rgba(30,5,50,0.8)" },

  {name: "Hollow Whisper", slot: "Weapon or Helmet", rarity: "Uncommon", effect: "Alerts carrier when an ambush or hidden threat is within 30 feet.",
  location: "Hollow's Edge enchanter's", lore: "The Hollow whispers constantly. This enchantment just makes one specific warning audible.", image:"", color:
  "#66bb6a", bg: "rgba(10,40,15,0.8)"},

  {name: "Feather-Iron Rune", slot: "Armor or Pack", rarity: "Uncommon", effect: "Halves the weight of equipped gear. +3 Inventory Slots.", location:
  "Kingdom Ruins", lore: "Used by the Garrison's heavy infantry to march for days without rest.", image:"", color: "#66bb6a", bg: "rgba(10,40,15,0.8)" },

  {name: "Vampiric Draw", slot: "Weapon", rarity: "Rare", effect: "On a natural 16+, heal 1 HP when striking a living target.", location: "Deep Caverns", lore:
  "The blade drinks, and so do you.", image:"", color: "#4fc3f7", bg: "rgba(5,30,50,0.8)" },

  {name: "Ethereal Step", slot: "Boots or Armor", rarity: "Rare", effect: "Once per combat, you can pass through a solid wall or obstacle up to 5 feet thick.",
  location: "Sunken Library", lore: "Weaves a temporary loophole into physical reality.", image:"", color: "#4fc3f7", bg: "rgba(5,30,50,0.8)" },

  {name: "Shatterstrike", slot: "Heavy Weapon", rarity: "Legendary", effect: "Permanently reduces the target's DEF by 1 on every successful hit.", location:
  "Throne Room", lore: "It doesn't just break the armor. It breaks the concept of the armor.", image: "", color: "#ffd54f", bg: "rgba(40,30,5,0.8)" },

  {name: "Void Edge", slot: "Weapon", rarity: "Legendary", effect: "Strikes ignore all DEF bonuses from armor. The blade finds the gap that isn't there.",
  location: "The Hollow's Edge", lore: "The Hollow doesn't distinguish between armor and absence.", image: "", color: "#ffd54f", bg: "rgba(40,30,5,0.8)" },

  {name: "The Nullify Ward", slot: "Shield or Chest", rarity: "Mythic", effect: "UNIQUE. Automatically absorb and completely negate the first magical attack
  that hits you each session.", location: "The Void", lore: "A black hole sewn into the fabric of your coat.", image:"", color: "#ce93d8", bg:
  "rgba(30,5,50,0.8)" },

  {name: "Glass Cannon Curse", slot: "Any", rarity: "Cursed", effect: "All damage dealt is doubled. All damage received is tripled.", location: "The Hollow's
  Offering", lore: "Victory is certain. Survival is not.", image:"", color: "#ef5350", bg: "rgba(50,5,5,0.8)" },

  {name: "Echo of the Mad King", slot: "Helmet", rarity: "Cursed", effect: "+3 to INT/WIS. On a Fumble, you must attack the nearest ally.", location:
  "Kingdom Catacombs", lore: "The voices are incredibly helpful. Until they demand blood.", image: "", color: "#ef5350", bg: "rgba(50,5,5,0.8)" }

];

```

```
const ARSENAL_CHESTS = [
  {name: "Garrison Paymaster's Strongbox", type: "Heavy Iron Chest", location: "Crumbling Kingdom Barracks", spawnChance: "Common", locked: true, lockDifficulty: "SPD 12+ check or a heavy hammer to break the rusted lock.", lore: "The soldiers died waiting to get paid.", loot: [{ item: "Coin Pouch (d10 x 100 coins)", chance: "60%" }, { item: "Reinforced Garrison Weapon", chance: "30%" }, { item: "Trapped: Poison Gas (-2 HP)", chance: "10%"}], image: "", color: "#aaa" },
  {name: "Smuggler's False-Bottom Trunk", type: "Wooden Crate", location: "Underground Caverns Blind corners", spawnChance: "Uncommon", locked: true, lockDifficulty: "INT 14+ to spot the false bottom, SPD 10+ to bypass the needle trap.", lore: "Thieves don't trust banks, and they definitely don't trust each other.", loot: [{item: "Rare Rogue Weapon", chance: "40%" }, { item: "Black Market Contraband (sells for 2,000c)", chance: "40%" }, { item: "Needle Trap: -1 SPD permanently until cured", chance: "20%"}], image: "", color: "#66bb6a" },
  {name: "Sunken Library Vault", type: "Sealed Repository", location: "Sunken Library - Flooded lower levels", spawnChance: "Rare", locked: true, lockDifficulty: "INT 16+ check to decipher the elemental ward.", lore: "The librarians sealed this not to keep thieves out, but to keep the book inside from escaping.", loot: [{item: "Legendary Grimoire or Focus", chance: "20%" }, { item: "Waterlogged pages (Useless)", chance: "50%" }, { item: "A trapped memory that attacks the opener", chance: "30%"}], image: "", color: "#4fc3f7" },
  {name: "Void-Sealed Reliquary", type: "Floating Obelisk", location: "The Hollow's Edge", spawnChance: "Mythic", locked: true, lockDifficulty: "Cannot be picked. Requires the sacrifice of 1 permanent Max HP to open.", lore: "The Hollow demands a trade. Flesh for power.", loot: [{ item: "Mythic Weapon or Armor", chance: "75%" }, { item: "Moon Squad Challenge Coin", chance: "5%" }, { item: "Cursed Mythic Artifact", chance: "20%"}], image: "", color: "#ce93d8"}
];
```

```
const ARSENAL_LOOT_BINS = [
  {name: "Collapsed Barracks Pile", type: "Debris Loot", location: "Crumbling Kingdom", roll: "d20 roll to search. 10+ finds something.", note: "The rubble has been searched before. Not everything was found.", image: "", color: "#aaa", loot: [{item: "Common Garrison Weapon", chance: "50%" }, { item: "Coin Pouch (d6 x 50)", chance: "40%" }, { item: "Cave-in hazard (takes 1d4 damage)", chance: "10%"}]},
  {name: "Ashen Pack Bone-Cache", type: "Buried Mound", location: "Dark Enchanted Forest", roll: "WIS 12+ to notice the disturbed earth.", note: "Werewolves bury things they want to ferment. Or things they want to hide.", image: "", color: "#66bb6a", loot: [{item: "Uncommon Slashing Weapon", chance: "40%" }, { item: "Gravenor's Claw (Artifact)", chance: "10%" }, { item: "Rotting meat (Attracts hostile wolves)", chance: "50%"}]},
  {name: "Overgrown Scholar's Satchel", type: "Dropped Bag", location: "Sunken Library outskirts", roll: "Automatic. It's just sitting there.", note: "Someone dropped this while running. They didn't make it far.", image: "", color: "#4fc3f7", loot: [{ item: "Rare Enchantment Scroll", chance: "25%" }, { item: "Healing Salve", chance: "50%" }, { item: "Nothing but wet paper", chance: "25%"}]},
  {name: "Petrified Victim", type: "Stone Statue", location: "Deep Caverns", roll: "SPD 15+ to carefully pry items loose.", note: "They were turned to stone mid-scream. They are still holding their coin purse.", image: "", color: "#ffd54f", loot: [{ item: "Legendary Ring or Amulet", chance: "15%" }, { item: "Gold Bars (3,000c)", chance: "35%", {item: "The curse spreads to you (Turn to stone in 3 rounds unless cured)", chance: "50%"}]},
  {name: "Hollow Rift Deposit", type: "Anomalous Loot", location: "The Hollow's Edge", roll: "Automatic roll d20 for tier.", note: "The Hollow deposits things near its edge. Whether this is intentional is a question nobody has answered.", image: "", color: "#ce93d8", loot: [{item: "Item from a dead player's inventory", chance: "30%" }, { item: "Something that belongs to the current arc's villain", chance: "20%" }, { item: "Nothing but the rift noticed you. You are marked.", chance: "50%"}]}
];
```

```
/* =====
HELPERS
===== */
```

```
const esc = (s) => String(s || "").replace(/([<>])/g, c => ({ "<": "<", ">": ">", "&": "&","'":"'","\"": "\""}[c]));
const d20 = () => Math.floor(Math.random() * 20) + 1;
const clamp = (v, min, max) => Math.min(max, Math.max(min, v));
const rangeLabel = (r) => r[0] === r[1] ? String(r[0]) : r[0] + " - " + r[1];

function getDiceDisplay(r) {
  if (!r) return null;
  if (r === 1) return {label: "💣 FUMBLE",desc:"Catastrophic backfire.",bg:"#2a0505",glow:"#ef5350"};
  if (r <= 4) return {label: "× FAIL",desc:"Action fails entirely.",bg:"#1a0505",glow:"#ef5350"};
  if (r <= 9) return {label: "△ WEAK HIT",desc:"Partial — something goes wrong.",bg:"#1a0e00",glow:"#ff9800"};
  if (r <= 14) return {label: "✓ SUCCESS",desc:"Clean success.",bg:"#041a06",glow:"#4caf50"};
```

```

if (r <= 19) return {label:"★ STRONG HIT",desc:"Success + bonus effect.",bg:"#021020",glow:"#4fc3f7"};
return {label:"⚡ CRITICAL!",desc:"LEGENDARY. ANNOUNCE IT!",bg:"#160520",glow:"#ce93d8"};
}

```

```

function getPlayerStats(p) {
const base = CLASS_BASE_STATS[p.cls] || {STR:5,DEF:5,INT:5,WIS:5,SPD:5,CHA:5,LCK:5};
const trait = TRAIT_TIERS.find(t => t.level === p.traitLevel);
const traitBonus = trait ? trait.bonus : 0;
const raceStats = p.race && RACES[p.race] ? RACES[p.race].stats : {STR:0,DEF:0,INT:0,WIS:0,SPD:0,CHA:0,LCK:0};
const primary = PRIMARY_STAT[p.cls] || "STR";
const companion = p.companion && COMPANION_TYPES[p.companion.type];
let finalStats = {};
for (const s in base) {
let v = base[s] + raceStats[s] + (s === primary ? traitBonus : 0);
if (companion && companion.statBuff && companion.statBuff[s] && p.companion.status === "active") v += companion.statBuff[s];
if (p.isHoF && s === "CHA") v += 3;
finalStats[s] = (p.statOverrides && p.statOverrides[s] !== undefined) ? p.statOverrides[s] : v;
}
return finalStats;
}

```

```

/* =====
SKYRIM / PARCHMENT DESIGN SYSTEM
===== */

```

```

const CSS = `
@import url('https://fonts.googleapis.com/css2?
family=MedievalSharp&family=Cinzel+Decorative:wght@400;700;900&family=Cinzel:wght@400;600;700&family=IM+Fell+English:ital@0;1&family=Uncial+A

```

```

:root {
--parch: #f2e8c9;
--parch-dark: #e8d9a8;
--parch-shadow: #c9b07a;
--ink: #2c1810;
--ink-faded: #5a3d2b;
--scroll-bg: #1a0e08;
--scroll-mid: #241408;
--ember: #c8813a;
--ember-glow: #f0a855;
--void: #8b5cf6;
--void-glow: #a78bfa;
--blood: #9b1b1b;
--forest: #2d5016;
--gold: #c9a227;
--gold-bright: #f0c040;
--iron: #6b7280;

```

```

-frost: #93c5fd;
-border-rune: rgba(200,129,58,0.4);
-border-rune-bright: rgba(240,168,85,0.7);
-shadow-deep: rgba(0,0,0,0.6);
-shadow-rune: 0 0 20px rgba(200,129,58,0.15);
}

*, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; -webkit-tap-highlight-color: transparent; }

html, body { height: 100%; overflow: hidden; }

body {
font-family: 'IM Fell English', Georgia, serif;
background: var(--scroll-bg);
color: var(--ink);
min-height: 100vh;
overflow: hidden;
}

/* — SCROLLBAR — */
::-webkit-scrollbar { width: 5px; height: 5px; }
::-webkit-scrollbar-track { background: rgba(0,0,0,0.2); }
::-webkit-scrollbar-thumb { background: var(--ember); border-radius: 3px; }

/* — APP SHELL — */
#hollow-app {
display: flex;
height: 100vh;
width: 100vw;
overflow: hidden;
position: relative;
}

/* — PARCHMENT TEXTURE OVERLAY — */
#hollow-app::before {
content: "";
position: fixed;
inset: 0;
background-image:
url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' width='300' height='300'%3E%3Cfilter id='n'%3E%3CfeTurbulence
type='fractalNoise' baseFrequency='0.65' numOctaves='3' stitchTiles='stitch'%3E%3CfeColorMatrix type='saturate'
values='0'%3E%3C/filter%3E%3Crect width='300' height='300' filter='url(%23n)' opacity='0.035'%3E%3C/svg%3E");
pointer-events: none;
z-index: 9999;
opacity: 0.6;

```

```

}

/* — SIDE NAV (desktop) — */

.sidenav {
width: 220px;
min-width: 220px;
background: linear-gradient(180deg, #0f0804 0%, #1a0e08 40%, #0a0604 100%);
border-right: 2px solid var(--ember);
display: flex;
flex-direction: column;
overflow: hidden;
position: relative;
flex-shrink: 0;
}

.sidenav::before {
content: "";
position: absolute;
inset: 0;
background: repeating-linear-gradient(0deg, transparent, transparent 59px, rgba(200,129,58,0.06) 60px);
pointer-events: none;
}

.sidenav-logo {
padding: 20px 16px 16px;
border-bottom: 1px solid var(--border-rune);
text-align: center;
}

.sidenav-logo h1 {
font-family: 'Cinzel Decorative', 'Cinzel', serif;
font-size: 0.75rem;
color: var(--ember-glow);
letter-spacing: 3px;
text-transform: uppercase;
text-shadow: 0 0 12px rgba(200,129,58,0.6);
line-height: 1.4;
}

.sidenav-logo .subtitle {
font-family: 'Cinzel', serif;
font-size: 0.45rem;
color: var(--parch-shadow);
letter-spacing: 2px;
margin-top: 4px;
}

.sidenav-logo .session-pip {
display: inline-flex;

```

```

align-items: center;
gap: 5px;
margin-top: 8px;
font-family: 'Cinzel', serif;
font-size: 0.5rem;
color: var(--parch-shadow);
letter-spacing: 1px;
}

.session-dot {
width: 6px; height: 6px; border-radius: 50%;
background: #4caf50;
box-shadow: 0 0 8px #4caf50;
animation: pulse-dot 1.5s ease-in-out infinite;
}

.session-dot.off { background: #444; box-shadow: none; animation: none; }

@keyframes pulse-dot { 0%,100%{opacity:1} 50%{opacity:0.3} }

/* — SCROLL-STYLE NAV ITEMS — */

.nav-section {
padding: 10px 0 4px;
}

.nav-section-label {
font-family: 'Cinzel', serif;
font-size: 0.38rem;
color: var(--ember);
letter-spacing: 3px;
text-transform: uppercase;
padding: 0 16px 4px;
opacity: 0.7;
}

.nav-item {
display: flex;
align-items: center;
gap: 10px;
padding: 9px 16px;
cursor: pointer;
position: relative;
transition: all 0.15s;
border-left: 3px solid transparent;
font-family: 'Cinzel', serif;
font-size: 0.62rem;
color: var(--parch-shadow);
letter-spacing: 0.5px;
user-select: none;
}

```

```
.nav-item:hover {
background: rgba(200,129,58,0.08);
color: var(--parch);
border-left-color: rgba(200,129,58,0.4);
}

.nav-item.active {
background: rgba(200,129,58,0.12);
color: var(--ember-glow);
border-left-color: var(--ember);
}

.nav-item.active::after {
content: "";
position: absolute;
right: 0; top: 50%;
transform: translateY(-50%);
width: 0; height: 0;
border-top: 6px solid transparent;
border-bottom: 6px solid transparent;
border-right: 6px solid var(--ember);
}

.nav-icon { font-size: 0.95rem; width: 20px; text-align: center; flex-shrink: 0; }
.nav-divider { height: 1px; background: linear-gradient(90deg, transparent, var(--border-rune), transparent); margin: 6px 12px; }
```

/* ——— MAIN CONTENT ——— */

```
.main-content {
flex: 1;
display: flex;
flex-direction: column;
overflow: hidden;
background: linear-gradient(135deg, #0f0804 0%, #1a0e08 50%, #0d0a04 100%);
}
```

/* ——— MOBILE TOP BAR ——— */

```
.topbar {
display: none;
background: linear-gradient(90deg, #0f0804, #1a0e08);
border-bottom: 1px solid var(--ember);
padding: 10px 14px;
align-items: center;
justify-content: space-between;
flex-shrink: 0;
}

.topbar-title {
font-family: 'Cinzel Decorative', serif;
font-size: 0.65rem;
```

```
color: var(--ember-glow);
letter-spacing: 2px;
}

.topbar-menu-btn {
background: none;
border: 1px solid var(--border-rune);
color: var(--ember);
padding: 5px 10px;
cursor: pointer;
font-size: 0.7rem;
border-radius: 3px;
font-family: 'Cinzel', serif;
}

/* — MOBILE BOTTOM NAV — */
.bottom-nav {
display: none;
background: #0a0604;
border-top: 1px solid var(--ember);
overflow-x: auto;
scrollbar-width: none;
flex-shrink: 0;
}

.bottom-nav::-webkit-scrollbar { display: none; }

.bottom-nav-inner {
display: flex;
min-width: max-content;
}

.bnav-item {
display: flex;
flex-direction: column;
align-items: center;
gap: 2px;
padding: 7px 12px;
cursor: pointer;
border-bottom: 2px solid transparent;
transition: all 0.15s;
font-family: 'Cinzel', serif;
font-size: 0.4rem;
color: #5a3d2b;
letter-spacing: 0.5px;
white-space: nowrap;
user-select: none;
}

.bnav-item.active { color: var(--ember-glow); border-bottom-color: var(--ember); }
```

```
.bnav-icon { font-size: 1rem; }
```

```
/* ——— PANEL AREA ——— */
```

```
.panel-area {  
  flex: 1;  
  overflow-y: auto;  
  overflow-x: hidden;  
  padding: 20px;  
}
```

```
/* ——— PARCHMENT SCROLL (section wrapper) ——— */
```

```
.scroll-panel {  
  background: linear-gradient(160deg, #f5e9c5 0%, #ecd99a 40%, #f2e4b8 70%, #e8d48e 100%);  
  border: 1px solid var(--parch-shadow);  
  border-radius: 4px;  
  position: relative;  
  box-shadow: 0 4px 20px rgba(0,0,0,0.5), inset 0 0 40px rgba(180,130,60,0.1);  
  margin-bottom: 16px;  
}
```

```
.scroll-panel::before {  
  content: "";  
  position: absolute;  
  top: 0; left: 0; right: 0;  
  height: 8px;  
  background: linear-gradient(90deg, #c9a227, #f0c040, #c9a227);  
  border-radius: 4px 4px 0 0;  
}
```

```
.scroll-panel::after {  
  content: "";  
  position: absolute;  
  bottom: 0; left: 0; right: 0;  
  height: 8px;  
  background: linear-gradient(90deg, #c9a227, #f0c040, #c9a227);  
  border-radius: 0 0 4px 4px;  
}
```

```
.scroll-inner { padding: 20px 18px; }
```

```
/* ——— DARK CARD ——— */
```

```
.dark-card {  
  background: rgba(15,8,4,0.95);  
  border: 1px solid var(--border-rune);  
  border-radius: 4px;  
  padding: 16px;  
  margin-bottom: 12px;  
  box-shadow: var(--shadow-rune);
```

```
position: relative;
}
.dark-card::before {
content: '✦';
position: absolute;
top: 6px; left: 8px;
font-size: 0.4rem;
color: var(--ember);
opacity: 0.5;
}
.dark-card::after {
content: '✦';
position: absolute;
top: 6px; right: 8px;
font-size: 0.4rem;
color: var(--ember);
opacity: 0.5;
}
.dark-card.glow { box-shadow: 0 0 25px rgba(200,129,58,0.2), var(--shadow-rune); border-color: var(--border-rune-bright); }
```

```
/* —— RUNE DIVIDER —— */
```

```
.rune-divider {
display: flex;
align-items: center;
gap: 10px;
margin: 14px 0;
}
.rune-divider::before, .rune-divider::after {
content: "";
flex: 1;
height: 1px;
background: linear-gradient(90deg, transparent, var(--border-rune), transparent);
}
.rune-divider span {
font-family: 'Cinzel', serif;
font-size: 0.45rem;
color: var(--ember);
letter-spacing: 3px;
text-transform: uppercase;
opacity: 0.7;
}
```

```
/* —— SECTION TITLE —— */
```

```
.section-title {
font-family: 'Cinzel', serif;
```

```
font-size: 0.62rem;
color: var(--ember-glow);
letter-spacing: 3px;
text-transform: uppercase;
margin-bottom: 12px;
border-bottom: 1px solid var(--border-rune);
padding-bottom: 6px;
}
```

```
/* — SCROLL TABS (Skyrim accordion-style) — */
```

```
.scroll-tabs {
display: flex;
gap: 4px;
flex-wrap: wrap;
margin-bottom: 14px;
}

.scroll-tab {
padding: 6px 14px;
cursor: pointer;
background: rgba(15,8,4,0.8);
border: 1px solid var(--border-rune);
border-bottom: none;
font-family: 'Cinzel', serif;
font-size: 0.55rem;
color: var(--parch-shadow);
letter-spacing: 1px;
position: relative;
transition: all 0.15s;
clip-path: polygon(0 0, calc(100% - 6px) 0, 100% 6px, 100% 100%, 0 100%);
user-select: none;
}

.scroll-tab:hover { color: var(--parch); border-color: rgba(200,129,58,0.6); }
.scroll-tab.active {
background: rgba(200,129,58,0.15);
color: var(--ember-glow);
border-color: var(--ember);
text-shadow: 0 0 8px rgba(240,168,85,0.5);
}
```

```
/* — ACCORDION SCROLL — */
```

```
.accordion-item {
background: rgba(15,8,4,0.9);
border: 1px solid var(--border-rune);
border-radius: 2px;
margin-bottom: 6px;
```

```

overflow: hidden;
transition: border-color 0.2s;
}
.accordion-item.open { border-color: var(--border-rune-bright); }
.accordion-header {
display: flex;
align-items: center;
justify-content: space-between;
padding: 10px 14px;
cursor: pointer;
transition: background 0.15s;
user-select: none;
}
.accordion-header:hover { background: rgba(200,129,58,0.06); }
.accordion-chevron {
font-size: 0.5rem;
color: var(--ember);
transition: transform 0.2s;
}
.accordion-item.open .accordion-chevron { transform: rotate(180deg); }
.accordion-body {
border-top: 1px solid var(--border-rune);
padding: 12px 14px;
background: rgba(10,5,2,0.5);
}

/* — SCROLL-STYLE INPUT — */
.scroll-input, .scroll-select, .scroll-textarea {
width: 100%;
background: rgba(242,232,197,0.1);
border: 1px solid var(--border-rune);
border-radius: 2px;
padding: 8px 10px;
color: var(--parch);
font-family: 'IM Fell English', serif;
font-size: 0.78rem;
outline: none;
transition: border-color 0.15s;
}
.scroll-input:focus, .scroll-select:focus, .scroll-textarea:focus {
border-color: var(--ember-glow);
box-shadow: 0 0 10px rgba(200,129,58,0.2);
}
.scroll-select { -webkit-appearance: none; }
.scroll-textarea { resize: vertical; min-height: 60px; }

```

```

.form-label {
display: block;
font-family: 'Cinzel', serif;
font-size: 0.5rem;
color: var(--ember);
letter-spacing: 2px;
text-transform: uppercase;
margin-bottom: 5px;
}

/* —— RUNE BUTTONS —— */

.rune-btn {
display: inline-flex;
align-items: center;
justify-content: center;
gap: 6px;
padding: 8px 18px;
background: linear-gradient(135deg, #3d1e08, #2a1404);
border: 1px solid var(--ember);
color: var(--ember-glow);
font-family: 'Cinzel', serif;
font-size: 0.6rem;
letter-spacing: 1.5px;
cursor: pointer;
transition: all 0.15s;
text-transform: uppercase;
clip-path: polygon(6px 0, calc(100% - 6px) 0, 100% 6px, 100% calc(100% - 6px), calc(100% - 6px) 100%, 6px 100%, 0 calc(100% - 6px), 0 6px);
position: relative;
}

.rune-btn:hover { background: linear-gradient(135deg, #5a2e10, #3d1e08); box-shadow: 0 0 15px rgba(200,129,58,0.3); }
.rune-btn.sm { padding: 5px 12px; font-size: 0.52rem; }
.rune-btn.full { width: 100%; }
.rune-btn.danger { background: linear-gradient(135deg, #3d0808, #200404); border-color: #9b1b1b; color: #ef9a9a; }
.rune-btn.danger:hover { background: linear-gradient(135deg, #5d0a0a, #3d0808); }
.rune-btn.success { background: linear-gradient(135deg, #0a3d10, #042010); border-color: #2e7d32; color: #a5d6a7; }
.rune-btn.disabled { opacity: 0.4; cursor: not-allowed; }

/* —— PLAYER CARD —— */

.player-card {
background: linear-gradient(135deg, rgba(15,8,4,0.97), rgba(26,14,8,0.9));
border: 1px solid var(--border-rune);
border-radius: 3px;
padding: 14px;
margin-bottom: 10px;
transition: border-color 0.2s, box-shadow 0.2s;

```

```

position: relative;
}

.player-card:hover { border-color: var(--border-rune-bright); box-shadow: 0 0 20px rgba(200,129,58,0.1); }
.player-card.dead { opacity: 0.55; border-color: rgba(155,27,27,0.4); }
.player-card.hof { border-color: rgba(201,162,39,0.6); box-shadow: 0 0 20px rgba(201,162,39,0.15); }

/* — HP BAR — */
.hp-track { height: 5px; background: rgba(255,255,255,0.06); border-radius: 3px; overflow: hidden; margin-top: 4px; }
.hp-fill { height: 100%; border-radius: 3px; transition: width 0.3s; }

/* — STAT GRID — */
.stat-grid { display: grid; grid-template-columns: repeat(7, 1fr); gap: 4px; margin-top: 8px; }
.stat-cell { text-align: center; background: rgba(0,0,0,0.3); border-radius: 3px; padding: 5px 2px; border: 1px solid rgba(255,255,255,0.04); }
.stat-label { font-family: 'Cinzel', serif; font-size: 0.4rem; letter-spacing: 1px; color: var(--parch-shadow); }
.stat-val { font-family: 'Cinzel', serif; font-size: 0.75rem; font-weight: 700; margin-top: 1px; }

/* — TAG PILLS — */
.tag { display: inline-block; padding: 2px 8px; border-radius: 12px; font-size: 0.55rem; font-family: 'Cinzel', serif; letter-spacing: 0.5px; }
.tag-curse { background: rgba(50,10,60,0.8); color: #e1bee7; border: 1px solid rgba(156,39,176,0.4); }
.tag-wanted { background: rgba(60,10,10,0.8); color: #ffcdd2; border: 1px solid rgba(239,83,80,0.4); }
.tag-exile { background: rgba(20,20,30,0.8); color: #bbb; border: 1px solid rgba(100,100,120,0.4); }
.tag-servant { background: rgba(5,30,25,0.8); color: #b2dfdb; border: 1px solid rgba(77,182,172,0.4); }
.tag-hof { background: rgba(40,30,5,0.8); color: #ffd54f; border: 1px solid rgba(201,162,39,0.5); }

/* — LOG ENTRY — */
.log-entry {
display: flex; gap: 10px; padding: 8px 0;
border-bottom: 1px solid rgba(200,129,58,0.08);
}
.log-icon { width: 22px; text-align: center; font-size: 0.8rem; flex-shrink: 0; }
.log-time { font-family: 'Cinzel', serif; font-size: 0.48rem; color: var(--parch-shadow); width: 50px; text-align: right; flex-shrink: 0; padding-top: 2px; }
.log-who { font-family: 'Cinzel', serif; font-size: 0.62rem; font-weight: 600; color: var(--ember-glow); }
.log-what { font-family: 'IM Fell English', serif; font-size: 0.7rem; color: var(--parch); margin-top: 1px; }

/* — DICE SVG — */
@keyframes dice-spin { from { transform: rotate(0deg); } to { transform: rotate(360deg); } }
.dice-spinning { animation: dice-spin 0.15s linear infinite; transform-origin: center; }

/* — MODAL OVERLAY — */
.modal-overlay {
position: fixed; inset: 0;
background: rgba(5,3,2,0.95);
z-index: 1000;
overflow-y: auto;
padding: 20px;

```

```

display: none;
}

.modal-overlay.open { display: flex; align-items: flex-start; justify-content: center; }

.modal-box {
background: linear-gradient(160deg, #1a0e08, #0f0804);
border: 1px solid var(--ember);
border-radius: 4px;
padding: 24px;
width: 100%;
max-width: 560px;
box-shadow: 0 0 50px rgba(200,129,58,0.2);
margin: auto;
position: relative;
}

.modal-box::before {
content: "";
position: absolute;
top: 0; left: 0; right: 0; height: 3px;
background: linear-gradient(90deg, transparent, var(--ember), transparent);
}

/* — MOBILE DRAWER NAV — */

.drawer-overlay {
position: fixed; inset: 0;
background: rgba(0,0,0,0.7);
z-index: 500;
display: none;
}

.drawer-overlay.open { display: block; }

.drawer {
position: fixed;
left: 0; top: 0; bottom: 0;
width: 240px;
background: #0f0804;
border-right: 2px solid var(--ember);
z-index: 501;
transform: translateX(-100%);
transition: transform 0.25s;
overflow-y: auto;
}

.drawer.open { transform: translateX(0); }

/* — RARITY BADGE — */

.rarity-badge { font-family: 'Cinzel', serif; font-size: 0.52rem; padding: 2px 8px; border-radius: 2px; letter-spacing: 1px; font-weight: 600; }

```

```

/* —— INFO BAR —— */
.info-bar {
background: rgba(200,129,58,0.06);
border: 1px solid var(--border-rune);
border-radius: 3px;
padding: 8px 12px;
font-family: 'IM Fell English', serif;
font-size: 0.7rem;
color: var(--parch);
line-height: 1.5;
margin-bottom: 10px;
}

.info-bar.danger { border-color: rgba(155,27,27,0.4); background: rgba(155,27,27,0.06); }
.info-bar.gold { border-color: rgba(201,162,39,0.5); background: rgba(201,162,39,0.08); }


/* —— EMPTY STATE —— */
.empty-state {
text-align: center;
padding: 40px 16px;
font-family: 'IM Fell English', serif;
font-style: italic;
font-size: 0.85rem;
color: var(--parch-shadow);
opacity: 0.6;
}


/* —— FADE IN —— */
@keyframes fadeIn { from { opacity:0; transform: translateY(6px); } to { opacity:1; transform: translateY(0); } }
.fade-in { animation: fadeIn 0.25s ease forwards; }


/* —— RESPONSIVE —— */
@media (max-width: 768px) {
.sidenav { display: none; }
.topbar { display: flex; }
.bottom-nav { display: flex; }
.panel-area { padding: 12px; }
.stat-grid { grid-template-columns: repeat(4, 1fr); }
}

@media (min-width: 769px) {
.topbar { display: none; }
.bottom-nav { display: none; }
}


/* —— 3D MAP CANVAS —— */
.map-canvas-wrap {

```

```
width: 100%;
height: 420px;
border: 1px solid var(--border-rune);
border-radius: 4px;
overflow: hidden;
background: #020101;
position: relative;
}

@media (max-width: 768px) { .map-canvas-wrap { height: 300px; } }

/* — PARCHMENT TEXT STYLES — */
.parch-title { font-family: 'Cinzel Decorative', serif; color: var(--ink); font-size: 1.1rem; margin-bottom: 4px; }
.parch-body { font-family: 'IM Fell English', serif; color: var(--ink-faded); font-size: 0.78rem; line-height: 1.7; }
.parch-label { font-family: 'Cinzel', serif; font-size: 0.52rem; color: var(--ink); letter-spacing: 2px; text-transform: uppercase; }

/* — PHASE BAR — */
.phase-bar { display: flex; gap: 3px; margin: 8px 0; }
.phase-pip { flex: 1; height: 5px; border-radius: 3px; cursor: pointer; transition: all 0.2s; }

/* — ROLL BOX — */
.roll-box {
background: rgba(8,4,2,0.95);
border: 1px solid var(--border-rune);
border-radius: 3px;
padding: 14px;
margin-top: 8px;
}

/* — SCROLL PROGRESS (session timer) — */
.session-timer {
font-family: 'Cinzel', serif;
font-size: 1.6rem;
color: var(--ember-glow);
text-align: center;
text-shadow: 0 0 20px rgba(200,129,58,0.5);
letter-spacing: 4px;
}
`;

/* =====
3D MAP SCENE
===== */

const LOC_3D = {
hq:   { pos: [0, 0, 0],   color: 0xffd54f, emissive: 0xffa000, size: 0.25 },
```

```

forest: { pos: [-3.2, 0, -2], color: 0x2e7d32, emissive: 0x1b5e20, size: 0.2 },
kingdom: { pos: [0.3, 0, -3], color: 0xc62828, emissive: 0x7f0000, size: 0.2 },
caverns: { pos: [-1.4, -0.5, 2], color: 0x6a1b9a, emissive: 0x4a148c, size: 0.2 },
edge: { pos: [2.8, 0, -0.5], color: 0x9c27b0, emissive: 0x6a0080, size: 0.22 },
library: { pos: [2.0, -0.3, 2.2], color: 0x1565c0, emissive: 0x0d47a1, size: 0.2 },
};

```

```

function TerrainPlane() {
const meshRef = useRef();
const geom = useMemo(() => {
const g = new THREE.PlaneGeometry(10, 10, 40, 40);
const pos = g.attributes.position;
for (let i = 0; i < pos.count; i++) {
const x = pos.getX(i), z = pos.getZ(i);
const h = (Math.sin(x * 0.8) * 0.25 + Math.cos(z * 0.6) * 0.2 + Math.sin((x + z) * 0.5) * 0.15) * 0.5;
pos.setZ(i, h);
}
g.computeVertexNormals();
return g;
}, []);

```

```

return (
<mesh ref={meshRef} geometry={geom} rotation={[-Math.PI / 2, 0, 0]} position={[0, -0.6, 0]}>
<meshLambertMaterial color={0x1a0e04} wireframe={false} />
</mesh>
);
}

```

```

function GridLines() {
const points = [];
for (let i = -5; i <= 5; i++) {
points.push(new THREE.Vector3(-5, -0.58, i), new THREE.Vector3(5, -0.58, i));
points.push(new THREE.Vector3(i, -0.58, -5), new THREE.Vector3(i, -0.58, 5));
}
const geom = new THREE.BufferGeometry().setFromPoints(points);
return <lineSegments geometry={geom}><lineBasicMaterial color={0x3d1e08} transparent opacity={0.4} /></lineSegments>;
}

```

```

function ConnectionLine({ fromPos, toPos }) {
const points = [new THREE.Vector3(...fromPos), new THREE.Vector3(...toPos)];
const geom = new THREE.BufferGeometry().setFromPoints(points);
return <lineSegments geometry={geom}><lineBasicMaterial color={0xc8813a} transparent opacity={0.25} linewidth={1} /></lineSegments>;
}

```

```

function LocationNode({ locId, data, danger, selected, onSelect, hasPlayer }) {

```

```

const meshRef = useRef();
const glowRef = useRef();
const [hovered, setHovered] = useState(false);
const dangerColor = [0x66bb6a, 0x66bb6a, 0x81c784, 0xff9800, 0xef5350, 0xb71c1c][danger] || 0xffffffff;

useFrame((state) => {
  if (!meshRef.current) return;
  const t = state.clock.elapsedTime;
  meshRef.current.position.y = data.pos[1] + Math.sin(t * 1.2 + data.pos[0]) * 0.04;
  if (selected) meshRef.current.rotation.y = t * 0.8;
  if (glowRef.current) {
    glowRef.current.material.opacity = 0.15 + Math.sin(t * 2) * 0.07;
    glowRef.current.scale.setScalar(1 + Math.sin(t * 1.5) * 0.08);
  }
});

return (
  <group position={data.pos} onClick={() => onSelect(locId)} onPointerOver={() => setHovered(true)} onPointerOut={() => setHovered(false)}>
    {/* Glow sphere */}
    <mesh ref={glowRef}>
      <sphereGeometry args={[data.size * 2.5, 12, 12]} />
      <meshBasicMaterial color={data.color} transparent opacity={0.12} />
    </mesh>
    {/* Main node */}
    <mesh ref={meshRef}>
      <sphereGeometry args={[data.size * (hovered ? 1.2 : 1), 16, 16]} />
      <meshStandardMaterial color={data.color} emissive={data.emissive} emissiveIntensity={selected ? 0.8 : 0.4} roughness={0.3} metalness={0.6} />
    </mesh>
    {/* Danger ring */}
    <mesh rotation={[Math.PI / 2, 0, 0]} position={[0, -0.35, 0]}>
      <ringGeometry args={[data.size * 1.5, data.size * 1.8, 32]} />
      <meshBasicMaterial color={dangerColor} transparent opacity={0.5} side={THREE.DoubleSide} />
    </mesh>
    {/* Player indicator */}
    {hasPlayer && (
      <mesh position={[0, data.size + 0.15, 0]}>
        <sphereGeometry args={[0.06, 8, 8]} />
        <meshBasicMaterial color={0xffffffff} />
      </mesh>
    )}
    {/* Label */}
    <Html distanceFactor={6} position={[0, data.size + 0.3, 0]} style={{ pointerEvents: "none" }}>
      <div style={{ fontFamily: "'Cinzel', serif", fontSize: "7px", color: hovered ? "#f0c040" : "#c8813a", whiteSpace: "nowrap", textShadow: "0 0 8px rgba(0,0,0,0.9)", letterSpacing: "1px", textTransform: "uppercase", userSelect: "none" }}>
        {WORLD_LOCATIONS.find(l => l.id === locId)?.short || locId}
      </div>
    </Html>
  </group>
);

```

```

</div>
</Html>
</group>
);
}

```

```

function Map3DScene({ mapDanger, players, selectedLoc, onSelectLoc }) {
  const playersByLoc = {};
  players.filter(p => !p.isDead && p.location).forEach(p => { playersByLoc[p.location] = true; });

  return (
    <>
    <Stars radius={80} depth={50} count={2000} factor={3} saturation={0} fade speed={0.5} />
    <ambientLight intensity={0.08} />
    <pointLight position={[0, 3, 0]} intensity={0.8} color={0xf0a855} distance={12} />
    <pointLight position={[-3, 1, -2]} intensity={0.3} color={0x4fc3f7} />
    <pointLight position={[3, 1, 2]} intensity={0.3} color={0x9c27b0} />
    <TerrainPlane />
    <GridLines />
    { /* Connection lines */
      { WORLD_LOCATIONS.map(loc => loc.connectedTo.map(told => {
        const toLoc = WORLD_LOCATIONS.find(l => l.id === told);
        if (!toLoc) return null;
        const fromD = LOC_3D[loc.id], toD = LOC_3D[told];
        if (!fromD || !toD) return null;
        return <ConnectionLine key={` ${loc.id}-${told}`} fromPos={fromD.pos} toPos={toD.pos} />;
      })}
    }
    { /* Location nodes */
      { WORLD_LOCATIONS.map(loc => {
        const d3 = LOC_3D[loc.id];
        if (!d3) return null;
        return (
          <LocationNode
            key={loc.id}
            locId={loc.id}
            data={d3}
            danger={mapDanger[loc.id] ?? loc.danger}
            selected={selectedLoc === loc.id}
            onSelect={onSelectLoc}
            hasPlayer={!!playersByLoc[loc.id]}
          />
        );
      })}
    }
    <OrbitControls enablePan={false} minDistance={4} maxDistance={14} maxPolarAngle={Math.PI / 2.2} autoRotate autoRotateSpeed={0.3} />
  </>

```

```

);
}

/* =====
MAIN APP
===== */

import React, { useState, useMemo } from "react";

import React, { useState } from "react";

import React, { useState, useMemo } from "react";

export default function MoonSquadBankingApp({ players, updatePlayer, addLog }) {
  // We simulate the "logged in" player for the app view.
  // In a real scenario, this is the player viewing their own phone.
  const [activePlayerId, setActivePlayerId] = useState(players.find(p => !p.isDead)?.id || "");
  const [isFlipped, setIsFlipped] = useState(false);
  const [treasuryBalance, setTreasuryBalance] = useState(14500); // Mock global treasury pool
  const treasuryGoal = 25000; // Goal for Level 3 Infirmary

  const activePlayer = players.find(p => p.id === activePlayerId);

  // Deterministically generate a fake card number based on player ID so it never changes
  const cardDetails = useMemo(() => {
    if (!activePlayer) return null;

    // Generate a pseudo-random but consistent 16 digit number
    const seed = activePlayer.id.toString();
    const part1 = "4092"; // Moon Squad BIN
    const part2 = (seed.substring(0, 4) || "8810").padStart(4, "0");
    const part3 = (seed.substring(4, 8) || "0029").padStart(4, "0");
    const part4 = (seed.substring(8, 12) || "7734").padStart(4, "0");

    // CVV and Expiry
    const cvv = (parseInt(seed.substring(0, 3)) % 900 + 100).toString();
    const exp = "05/26"; // Launch month!

    return { number: `${part1} ${part2} ${part3} ${part4}`, cvv, exp };
  }, [activePlayer]);

  if (!activePlayer) return <div className="empty-state">No active accounts found.</div>;

  const walletBalance = activePlayer.walletBalance || 0;
  const progressPct = Math.min(100, (treasuryBalance / treasuryGoal) * 100);

```

```

const handleDonate = () => {
  const amount = parseInt(window.prompt("How much to transfer to the Moon Squad Treasury?"));
  if (!amount || amount <= 0 || amount > walletBalance) return alert("Invalid or insufficient funds.");

  updatePlayer(activePlayer.id, { walletBalance: walletBalance - amount });
  setTreasuryBalance(prev => prev + amount);
  addLog({ type: "gift", who: activePlayer.charname, what: `Transferred ${amount.toLocaleString()}c to the Treasury.` });
};

return (
  <div className="fade-in" style={{ padding: "16px", maxWidth: "450px", margin: "0 auto", fontFamily: "'Cinzel', serif" }}>

    { /* INJECTED CSS FOR 3D CARD FLIP */ }

    <style>{`
      .card-scene {
        perspective: 1000px;
        width: 100%;
        height: 220px;
        margin-bottom: 24px;
        cursor: pointer;
      }
      .card-object {
        width: 100%;
        height: 100%;
        transition: transform 0.8s cubic-bezier(0.175, 0.885, 0.32, 1.275);
        transform-style: preserve-3d;
        position: relative;
      }
      .card-scene.is-flipped .card-object {
        transform: rotateY(180deg);
      }
      .card-face {
        position: absolute;
        width: 100%;
        height: 100%;
        backface-visibility: hidden;
        border-radius: 16px;
        background: linear-gradient(135deg, #0f0804 0%, #2c1810 50%, #000000 100%);
        border: 1px solid rgba(255, 213, 79, 0.4);
        box-shadow: 0 10px 30px rgba(0,0,0,0.8), inset 0 0 20px rgba(255, 213, 79, 0.1);
        overflow: hidden;
      }
      .card-back {
        transform: rotateY(180deg);

```

```

}

.mag-stripe {
  width: 100%;
  height: 40px;
  background: #000;
  margin-top: 24px;
}

.hologram {
  position: absolute;
  top: 0; left: 0; right: 0; bottom: 0;
  background: linear-gradient(125deg, rgba(255,255,255,0) 30%, rgba(255,255,255,0.05) 50%, rgba(255,255,255,0) 70%);
  pointer-events: none;
}

.bank-app-bg {
  background: linear-gradient(180deg, #1a0e08 0%, #050302 100%);
  border: 1px solid var(--ember);
  border-radius: 12px;
  padding: 20px;
  box-shadow: 0 0 40px rgba(200, 129, 58, 0.1);
}
`</style>

```

```

{/* PLAYER SELECTOR (For GM testing purposes) */}

```

```

<select
  className="scroll-select"
  value={activePlayerId}
  onChange={e => { setActivePlayerId(e.target.value); setIsFlipped(false); }}
  style={{ marginBottom: "20px", background: "rgba(0,0,0,0.5)" }}
>
  {players.filter(p => !p.isDead).map(p => <option key={p.id} value={p.id}>Viewing as: {p.charname}</option>)}
</select>

```

```

<div className="bank-app-bg">

```

```

{/* HEADER */}

```

```

<div style={{ display: "flex", justifyContent: "space-between", alignItems: "center", marginBottom: "20px" }}>

```

```

  <div>

```

```

    <div style={{ fontSize: "0.6rem", color: "var(--dim)", letterSpacing: "2px" }}>AVAILABLE BALANCE</div>

```

```

    <div style={{ fontSize: "2rem", color: "var(--gold)", fontWeight: "bold", textShadow: "0 0 15px rgba(255,213,79,0.3)" }}>

```

```

      {walletBalance.toLocaleString()}<span style={{ fontSize: "1rem", color: "var(--parch-shadow)" }}>c</span>

```

```

    </div>

```

```

  </div>

```

```

  <div style={{ textAlign: "right" }}>

```

```

    <div style={{ fontSize: "0.5rem", color: "var(--green)", border: "1px solid var(--green)", padding: "4px 8px", borderRadius: "12px", letterSpacing:

```

```

    "1px" }}>

```

ACCOUNT ACTIVE

</div>

</div>

</div>

{/* 3D DEBIT CARD */}

<div className={`card-scene \${isFlipped ? 'is-flipped' : ''}`} onClick={() => setIsFlipped(!isFlipped)}>

<div className="card-object">

{/* FRONT OF CARD */}

<div className="card-face">

<div className="hologram"></div>

<div style={{ padding: "20px", display: "flex", flexDirection: "column", height: "100%", justifyContent: "space-between" }}>

<div style={{ display: "flex", justifyContent: "space-between", alignItems: "flex-start" }}>

<div style={{ fontSize: "0.7rem", color: "var(--parch)", letterSpacing: "3px" }}>SUNKEN SYNDICATE</div>

<div style={{ fontSize: "1rem", color: "rgba(255,255,255,0.2)" }}>↛</div> {/* Mock NFC Icon */}

</div>

<div>

{/* Fake EMV Chip */}

<div style={{ width: "40px", height: "30px", background: "linear-gradient(135deg, #ffd54f, #c9a227)", borderRadius: "4px", marginBottom: "12px", opacity: 0.8, backgroundImage: "repeating-linear-gradient(90deg, transparent, transparent 5px, rgba(0,0,0,0.1) 6px, rgba(0,0,0,0.1) 10px)" }}>

<div style={{ fontFamily: "'Courier New', monospace", fontSize: "1.3rem", color: "#fff", letterSpacing: "4px", textShadow: "1px 1px 2px #000" }}>

{cardDetails.number}

</div>

</div>

<div style={{ display: "flex", justifyContent: "space-between", alignItems: "flex-end" }}>

<div style={{ fontSize: "0.8rem", color: "var(--parch-shadow)", textTransform: "uppercase", letterSpacing: "2px" }}>

{activePlayer.charname}

</div>

<div style={{ textAlign: "right" }}>

<div style={{ fontSize: "0.4rem", color: "var(--dim)" }}>VALID THRU</div>

<div style={{ fontSize: "0.8rem", color: "#fff", fontFamily: "'Courier New', monospace" }}>{cardDetails.exp}</div>

</div>

</div>

</div>

</div>

{/* BACK OF CARD */}

<div className="card-face card-back">

```

<div className="mag-stripe"></div>
<div style={{ padding: "20px" }}>
  <div style={{ background: "#fff", height: "30px", display: "flex", alignItems: "center", justifyContent: "flex-end", paddingRight: "10px", marginTop: "10px" }}>
    <span style={{ fontFamily: "'Courier New', monospace", fontStyle: "italic", fontSize: "1rem", color: "#000" }}>{cardDetails.cvv}</span>
  </div>
  <div style={{ fontSize: "0.4rem", color: "var(--dim)", marginTop: "10px", lineHeight: "1.5", textAlign: "justify" }}>
    This ledger card remains the property of the Sunken Syndicate. Unauthorized use will result in immediate void-reclamation of funds and physical assets. If found, drop into the nearest deep water.
  </div>
</div>
</div>
</div>

{/* TREASURY POOL (Idea #1) */}
<div style={{ background: "rgba(0,0,0,0.4)", borderRadius: "8px", padding: "16px", border: "1px solid var(--border-rune)", marginBottom: "20px" }}>
  <div style={{ display: "flex", justifyContent: "space-between", marginBottom: "8px" }}>
    <span style={{ fontSize: "0.6rem", color: "var(--ember)", letterSpacing: "1px" }}>MOON SQUAD TREASURY</span>
    <span style={{ fontSize: "0.6rem", color: "var(--parch)" }}>Lv. 3 Infirmary Goal</span>
  </div>

  <div className="hp-track" style={{ height: "8px", background: "rgba(255,255,255,0.05)", borderRadius: "4px" }}>
    <div className="hp-fill" style={{ width: `${progressPct}%`, background: "linear-gradient(90deg, #c8813a, #f0a855)", borderRadius: "4px", transition: "width 1s ease-in-out" }}></div>
  </div>

  <div style={{ display: "flex", justifyContent: "space-between", marginTop: "8px" }}>
    <span style={{ fontSize: "0.7rem", color: "var(--gold)", fontWeight: "bold" }}>{treasuryBalance.toLocaleString()}</span>
    <span style={{ fontSize: "0.6rem", color: "var(--dim)" }}>{treasuryGoal.toLocaleString()}</span>
  </div>
</div>

{/* QUICK ACTIONS */}
<div style={{ display: "flex", gap: "10px" }}>
  <button className="rune-btn" style={{ flex: 1, padding: "12px", fontSize: "0.65rem", display: "flex", flexDirection: "column", alignItems: "center", gap: "6px" }} onClick={() => alert("Withdrawal logic triggered.")}>
    <span style={{ fontSize: "1.2rem" }}>&#x2193;</span>
    Withdraw
  </button>
  <button className="rune-btn" style={{ flex: 1, padding: "12px", fontSize: "0.65rem", display: "flex", flexDirection: "column", alignItems: "center", gap: "6px", borderColor: "var(--gold)", color: "var(--gold)" }} onClick={handleDonate}>
    <span style={{ fontSize: "1.2rem" }}>&#x2191;</span>
    Fund Treasury
  </button>

```

```

    </div>

    </div>
  </div>
);
}

export default function HQPanel({ players, updatePlayer, addLog }) {
  const [activeResident, setActiveResident] = useState("");
  const [hqTab, setHqTab] = useState("vault"); // "vault" or "forge"

  const activePlayer = players.find(p => p.id === activeResident);
  const isAtHQ = activePlayer?.location === "hq";

  const conditionLadder = ["Broken", "Damaged", "Worn", "Pristine"];

  const moveToVault = (itemIndex) => {
    if (!activePlayer) return;
    const item = activePlayer.inventory[itemIndex];
    if (item.equipped) {
      alert("Cannot stash equipped gear. Unequip it first.");
      return;
    }
    const newInv = activePlayer.inventory.filter((_, i) => i !== itemIndex);
    const newVault = [...(activePlayer.vault || []), item];
    updatePlayer(activePlayer.id, { inventory: newInv, vault: newVault });
    addLog({ type: "item", who: activePlayer.charname, what: `Stored ${item.name} in the Moon Squad Vault` });
  };

  const moveToPack = (itemIndex) => {
    if (!activePlayer) return;
    // Check if backpack is full (Merchant = 12, Others = 8)
    const baseCap = activePlayer.cls === "Merchant" ? 12 : 8;
    const muleBonus = activePlayer.companion?.type === "Clockwork Mule" && activePlayer.companion.status === "active" ? 5 : 0;
    const maxCap = baseCap + muleBonus;

    if ((activePlayer.inventory?.length || 0) >= maxCap) {
      alert(`${activePlayer.charname}'s Rucksack is too full to carry this!`);
      return;
    }

    const item = activePlayer.vault[itemIndex];
    const newVault = activePlayer.vault.filter((_, i) => i !== itemIndex);
    const newInv = [...(activePlayer.inventory || []), item];
    updatePlayer(activePlayer.id, { inventory: newInv, vault: newVault });
  };

```

```

addLog({ type: "item", who: activePlayer.charname, what: `Retrieved ${item.name} from the Vault` });
};

const repairItem = (itemIndex) => {
  if (!activePlayer) return;
  const item = activePlayer.inventory[itemIndex];
  const currentIdx = conditionLadder.indexOf(item.condition);

  if (currentIdx === -1 || currentIdx === 3) {
    alert("This item is already Pristine!");
    return;
  }

  const nextCondition = conditionLadder[currentIdx + 1];

  if (window.confirm(`Repair ${item.name} from ${item.condition} to ${nextCondition}? (Deduct coins/ore manually)`)) {
    const newInv = [...activePlayer.inventory];
    newInv[itemIndex] = { ...item, condition: nextCondition };
    updatePlayer(activePlayer.id, { inventory: newInv });
    addLog({ type: "item", who: activePlayer.charname, what: `Forged ${item.name} to ${nextCondition} condition` });
  }
};

return (
  <div className="fade-in">
    <div className="section-title">
      <span role="img" aria-label="hq">  </span> Moon Squad Headquarters
    </div>

    /* --- TRAVELER SELECTION --- */
    <div className="dark-card" style={{ marginBottom: "16px", borderColor: "var(--blue)" }}>
      <label className="form-label">Select Resident</label>
      <select
        className="scroll-select"
        value={activeResident}
        onChange={e => setActiveResident(e.target.value)}
        aria-label="Select HQ Resident"
      >
        <option value="">-- Select a Traveler at HQ --</option>
        {players.filter(p => !p.isDead).map(p => (
          <option key={p.id} value={p.id}>{p.charname} {p.location !== "hq" ? "(In the field)" : ""}</option>
        ))}
      </select>
    </div>

```

```

{activePlayer && (
  <>
  {!isAtHQ && (
    <div className="info-bar danger" style={{ marginBottom: '16px' }}>
      <strong>WARNING:</strong> {activePlayer.charname} is not currently at HQ. Are you using an emergency tether?
    </div>
  )}

  <div className="scroll-tabs" style={{ marginBottom: '16px' }}>
    <button type="button" className={`scroll-tab ${hqTab === "vault" ? "active" : ""}`} onClick={() => setHqTab("vault")} style={{ background:
'transparent', border: 'none', cursor: 'pointer' }}>
      <span role="img" aria-label="vault">🔐</span> The Vault
    </button>
    <button type="button" className={`scroll-tab ${hqTab === "forge" ? "active" : ""}`} onClick={() => setHqTab("forge")} style={{ background:
'transparent', border: 'none', cursor: 'pointer' }}>
      <span role="img" aria-label="forge">🔨</span> The Forge
    </button>
  </div>

  {/--- THE VAULT VIEW --- */}
  {hqTab === "vault" && (
    <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '12px' }}>

      {/--- Backpack Column --- */}
      <div className="dark-card" style={{ padding: '8px' }}>
        <div style={{ color: 'var(--parch)', borderBottom: '1px solid var(--border-rune)', marginBottom: '8px', paddingBottom: '4px', textAlign: 'center'
}}>
          Their Rucksack ({activePlayer.inventory?.length || 0})
        </div>
        {activePlayer.inventory?.map((item, idx) => (
          <div key={idx} style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', background: 'rgba(0,0,0,0.3)', padding: '6px',
marginBottom: '4px' }}>
            <div style={{ fontSize: '0.7rem', color: item.equipped ? 'var(--ember)' : 'var(--parch)' }}>
              {item.equipped && <span role="img" aria-label="equipped" style={{marginRight:'4px'}}>🔥</span>
              {item.name}
            </div>
            <button type="button" className="rune-btn sm" onClick={() => moveToVault(idx)} style={{ fontSize: '0.5rem', padding: '2px 6px'
}}>Stash</button>
          </div>
        ))}
        {(!activePlayer.inventory || activePlayer.inventory.length === 0) && <div style={{ fontSize: '0.6rem', textAlign: 'center', color: 'var(--dim)' }}>Pack
is empty.</div>}
      </div>

      {/--- Vault Column --- */}
      <div className="dark-card" style={{ padding: '8px', borderColor: 'var(--gold)' }}>
        <div style={{ color: 'var(--gold)', borderBottom: '1px solid var(--gold)', marginBottom: '8px', paddingBottom: '4px', textAlign: 'center' }}>

```

```

    Moon Squad Stash ({activePlayer.vault?.length || 0})
  </div>
  {activePlayer.vault?.map((item, idx) => (
    <div key={idx} style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', background: 'rgba(40,30,10,0.3)', padding: '6px',
marginBottom: '4px', borderLeft: '2px solid var(--gold)' }}>
      <div style={{ fontSize: '0.7rem', color: 'var(--parch)' }}>{item.name}</div>
      <button type="button" className="rune-btn success sm" onClick={() => moveToPack(idx)} style={{ fontSize: '0.5rem', padding: '2px 6px'
}}>Take</button>
    </div>
  )))
  ((!activePlayer.vault || activePlayer.vault.length === 0) && <div style={{ fontSize: '0.6rem', textAlign: 'center', color: 'var(--dim)' }}>Vault is empty.
</div>
</div>
</div>
})

{/* --- THE FORGE VIEW --- */}
{hqTab === "forge" && (
  <div className="dark-card" style={{ borderColor: 'var(--ember)' }}>
    <div style={{ color: 'var(--ember-glow)', borderBottom: '1px solid var(--border-rune)', marginBottom: '12px', paddingBottom: '4px', display: 'flex',
alignItems: 'center', gap: '8px' }}>
      <span role="img" aria-label="anvil">🔨</span> Needs Repair
    </div>

    {activePlayer.inventory?.filter(i => i.condition !== "Pristine").map((item) => {
      const actualIndex = activePlayer.inventory.indexOf(item);
      const isBroken = item.condition === "Broken";

      return (
        <div key={actualIndex} style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', background: 'rgba(0,0,0,0.3)', padding:
'8px', marginBottom: '6px', borderLeft: '3px solid ${isBroken ? 'var(--blood)' : 'var(--ember)'}' }}>
          <div>
            <div style={{ fontSize: '0.8rem', color: 'var(--parch)' }}>{item.name}</div>
            <div style={{ fontSize: '0.6rem', color: isBroken ? '#ef9a9a' : 'var(--dim)', textTransform: 'uppercase' }}>
              Current: <strong>{item.condition}</strong>
            </div>
          </div>
          <button type="button" className="rune-btn sm" onClick={() => repairItem(actualIndex)} style={{ fontSize: '0.55rem', padding: '4px 8px',
borderColor: 'var(--ember)' }}>
            Strike Anvil
          </button>
        </div>
      );
    })

    {activePlayer.inventory?.filter(i => i.condition !== "Pristine").length === 0 && (
      <div style={{ textAlign: 'center', color: 'var(--dim)', fontSize: '0.7rem', padding: '12px' }}>

```

```

        All gear is in Pristine condition. The Forge rests.
      </div>
    })
  </div>
})
</>
})
</div>
);
}

```

```

export default function MobileBackpack({ players, updatePlayer, addLog, toggleEquip, rmlItem }) {
  const [search, setSearch] = useState("");
  const [selectedId, setSelectedId] = useState(null);
  const [activeTab, setActiveTab] = useState("grid"); // "grid" or "belt"

  // 1. Search & Select Logic
  const filteredPlayers = players.filter(p =>
    !p.isDead && (
      p.username?.toLowerCase().includes(search.toLowerCase()) ||
      p.charname?.toLowerCase().includes(search.toLowerCase())
    )
  );

  const activePlayer = players.find(p => p.id === selectedId);

  // 2. Capacity Math
  const capacityInfo = useMemo(() => {
    if (!activePlayer) return { max: 0, current: 0, isOverloaded: false };

    // Merchants get 12 slots, others get a base of 8
    const baseCap = activePlayer.cls === "Merchant" ? 12 : 8;
    // Clockwork Mule adds +5 slots
    const muleBonus = activePlayer.companion?.type === "Clockwork Mule" && activePlayer.companion.status === "active" ? 5 : 0;

    const max = baseCap + muleBonus;
    const current = activePlayer.inventory?.length || 0;

    return { max, current, isOverloaded: current > max };
  }, [activePlayer]);

  const isAtHQ = activePlayer?.location === "hq";

  return (
    <div className="scroll-panel fade-in" style={{ border: '1px solid var(--parch-shadow)' }}>

```

```

<div className="scroll-inner">
  <div className="section-title" style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center' }}>
    <span style={{ display: 'flex', alignItems: 'center', gap: '6px' }}>
      <span role="img" aria-label="backpack">🎒</span> Mobile Rucksack
    </span>
    {activePlayer && (
      <span style={{ fontSize: '0.6rem', color: capacityInfo.isOverloaded ? 'var(--blood)' : 'var(--parch-shadow)' }}>
        Weight: {capacityInfo.current} / {capacityInfo.max}
      </span>
    )}
  </div>

  {/ * --- SEARCH DROPDOWN --- */}
  <div style={{ marginBottom: '16px', position: 'relative' }}>
    <input
      className="scroll-input"
      placeholder="Search traveler by @username or name..."
      aria-label="Search travelers"
      value={search}
      onChange={(e) => { setSearch(e.target.value); setSelectedId(null); }}
    />
    {search && !selectedId && (
      <div className="dark-card" style={{ position: 'absolute', zIndex: 10, width: '100%', marginTop: '4px', maxHeight: '200px', overflowY: 'auto' }}>
        {filteredPlayers.length > 0 ? filteredPlayers.map(p => (
          <button
            type="button"
            key={p.id}
            className="nav-item"
            style={{ width: '100%', textAlign: 'left', background: 'transparent', border: 'none', cursor: 'pointer' }}
            onClick={() => { setSelectedId(p.id); setSearch(p.charname); }}
          >
            <span role="img" aria-label="class icon">{CLASS_ICON_EMO[p.cls]}</span> {p.charname}
            <span style={{ color: 'var(--parch-shadow)', fontSize: '0.5rem', marginLeft: '6px' }}>@{p.username}</span>
          </button>
        )) : (
          <div style={{ padding: '8px', fontSize: '0.65rem', color: 'var(--parch-shadow)' }}>No matching travelers...</div>
        )}
      </div>
    )}
  </div>

  {activePlayer ? (
    <>
      {/ * --- HQ VS FIELD MODE ALERT --- */}
      {!isAtHQ && (

```

```

<div className="info-bar danger" style={{ fontSize: '0.65rem', padding: '6px', textAlign: 'center' }}>
  <strong>FIELD MODE ACTIVE:</strong> Manual inventory additions disabled. Player must find a Merchant or Loot Cache.
</div>
))

```

```

{/* --- BACKPACK TABS --- */}
<div className="scroll-tabs" style={{ marginBottom: '12px', justifyContent: 'center' }}>
  <button
    type="button"
    className={`scroll-tab ${activeTab === "grid" ? "active" : ""}`}
    onClick={() => setActiveTab("grid")}
    style={{ background: 'transparent', border: 'none', cursor: 'pointer' }}
  >
    Pack Grid
  </button>
  <button
    type="button"
    className={`scroll-tab ${activeTab === "belt" ? "active" : ""}`}
    onClick={() => setActiveTab("belt")}
    style={{ background: 'transparent', border: 'none', cursor: 'pointer' }}
  >
    Quick-Belt
  </button>
</div>

```

```

{/* --- VIEW: PACK GRID --- */}
{activeTab === "grid" && (
  <div style={{
    display: 'grid', gridTemplateColumns: 'repeat(4, 1fr)', gap: '8px',
    background: 'rgba(0,0,0,0.2)', padding: '12px', borderRadius: '4px'
  }}>
    {Array.from({ length: capacityInfo.max }).map((_, i) => {
      const item = activePlayer.inventory?.[i];
      const isCursed = item?.rarity === "Cursed" || item?.effect?.toLowerCase().includes("curse");
      const rarityColor = item ? (RARITY_COLOR[item.rarity] || 'var(--ember)') : 'transparent';

      return (
        <div key={i} className="dark-card glow" style={{
          aspectRatio: '1/1', padding: 0, marginBottom: 0,
          background: item ? 'rgba(15,8,4,0.95)' : 'rgba(0,0,0,0.1)',
          borderColor: item ? rarityColor : 'var(--border-rune)',
          boxShadow: isCursed ? 'inset 0 0 15px rgba(156,39,176,0.6)' : (item ? `inset 0 0 8px ${rarityColor}40` : 'none'),
          display: 'flex', flexDirection: 'column', alignItems: 'center', justifyContent: 'center', position: 'relative'
        }}>
          {item ? (

```

```

<>

<div style={{ fontSize: '1.2rem' }}>

  {item.type?.toLowerCase().includes("weapon") || item.type?.toLowerCase().includes("slashing") ||
item.type?.toLowerCase().includes("blunt") ? (

    <span role="img" aria-label="weapon">✂️</span>

  ) : item.type?.toLowerCase().includes("consumable") || item.type?.toLowerCase().includes("orb") ? (

    <span role="img" aria-label="consumable">🍷</span>

  ) : (

    <span role="img" aria-label="gear">⚙️</span>

  )}

</div>

{/* Item Condition & Cursed Overlays */}

{item.condition === "Broken" && (

  <div style={{ position: 'absolute', top: 2, background: 'var(--blood)', color: 'white', fontSize: '0.45rem', padding: '1px 3px', borderRadius:
'2px', fontFamily: '"Cinzel", serif' }}>

    BROKEN

  </div>

)}

{isCursed && (

  <div style={{ position: 'absolute', bottom: 2, background: 'var(--void)', color: 'white', fontSize: '0.45rem', padding: '1px 3px',
borderRadius: '2px', fontFamily: '"Cinzel", serif' }}>

    CURSED

  </div>

)}

{/* Fully Accessible Click Target */}

<button

  type="button"

  style={{ position: 'absolute', width: '100%', height: '100%', cursor: 'pointer', background: 'transparent', border: 'none' }}

  onClick={() => alert(` ${item.name} ( ${item.rarity} )\nCondition: ${item.condition}\nEffect: ${item.effect} `)}

  aria-label={` Inspect ${item.name} `}

/>

</>

) : (

  <div style={{ opacity: 0.1, fontSize: '0.8rem' }}>+</div>

)}

</div>

);

}}

</div>

}}

{/* --- VIEW: QUICK-BELT (For Box Battles) --- */}

{activeTab === "belt" && (

  <div style={{ display: 'flex', flexDirection: 'column', gap: '8px' }}>

```

```

<div className="rune-divider"><span style={{ fontSize: '0.45rem' }}>Equipped Gear</span></div>

{activePlayer.inventory?.filter(i => i.equipped).map((item, idx) => (
  <div key={idx} style={{ display: 'flex', alignItems: 'center', gap: '10px', padding: '8px', background: 'rgba(0,0,0,0.3)', borderLeft: `3px solid
${RARITY_COLOR[item.rarity] || 'var(--ember)'}' }}>
    <div style={{ flex: 1 }}>
      <div style={{ fontFamily: '"Cinzel', serif", fontSize: '0.7rem', color: 'var(--ember-glow)' }}>{item.name}</div>
      <div style={{ fontSize: '0.55rem', color: 'var(--parch-shadow)', fontFamily: '"IM Fell English', serif" }}>{item.effect}</div>
    </div>
    <button type="button" className="rune-btn sm" style={{ fontSize: '0.45rem', padding: '4px 8px' }} onClick={() =>
toggleEquip(activePlayer.id, activePlayer.inventory.indexOf(item))>
      Unequip
    </button>
  </div>
)
)}}

{activePlayer.inventory?.filter(i => i.equipped).length === 0 && (
  <div className="empty-state" style={{ padding: '10px' }}>No items equipped for battle.</div>
)}

<div className="rune-divider"><span style={{ fontSize: '0.45rem' }}>Consumables</span></div>

{activePlayer.inventory?.filter(i => !i.equipped && (i.type?.toLowerCase().includes("consumable") ||
i.effect?.toLowerCase().includes("heal"))).map((item, idx) => (
  <div key={idx} style={{ display: 'flex', alignItems: 'center', gap: '10px', padding: '6px 8px', background: 'rgba(0,40,20,0.3)', border: '1px solid
rgba(102,187,106,0.3)' }}>
    <span role="img" aria-label="potion" style={{ fontSize: '0.9rem' }}><img alt="potion icon" data-bbox="515 505 530 515"/></span>
    <div style={{ flex: 1, fontFamily: '"IM Fell English', serif", fontSize: '0.7rem', color: 'var(--parch)' }}>{item.name}</div>
    <button type="button" className="rune-btn success sm" style={{ fontSize: '0.45rem' }} onClick={() => {
      if(window.confirm(`Use ${item.name}?`)) {
        rmlItem(activePlayer.id, activePlayer.inventory.indexOf(item));
        addLog({ type: "item", who: activePlayer.charname, what: `Consumed ${item.name}` });
      }
    }}>Use</button>
  </div>
)
)}}

</div>

)}

</div>

)}

{ /* Quick Actions */}

{isAtHQ && (
  <div style={{ marginTop: '16px', display: 'flex', gap: '8px' }}>
    <button type="button" className="rune-btn full" onClick={() => alert("Open Vault/Forge logic here")}>Go to The Forge</button>
  </div>
)}

</>

): (
  <div className="empty-state">Awaiting traveler selection...</div>

```

```

    })
  </div>
</div>

);
}

```

```

export default function HollowRealmGM() {
  // ——— PERSISTED STATE ———

  const [players, setPlayers] = useState(() => S.get("players", []));
  const [sessionLog, setSessionLog] = useState(() => S.get("session_log", []));
  const [sessionActive, setSessionActive] = useState(() => S.get("session_active", false));
  const [sessionNumber, setSessionNumber] = useState(() => S.get("session_number", 1));
  const [mapPins, setMapPins] = useState(() => S.get("map_pins", []));
  const [mapDanger, setMapDanger] = useState(() => S.get("map_danger", {}));
  const [baseRooms, setBaseRooms] = useState(() => S.get("base_rooms", {vault:1,barracks:0,forge:0,infirmary:0,watchtower:0}));
  const [baseResidents, setBaseResidents] = useState(() => S.get("base_residents", []));
  const [villainsLive, setVillainsLive] = useState(() => S.get("villains_live", []));
  const [villainPhases, setVillainPhases] = useState(() => S.get("villain_phases", {}));
  const [villainEmpires, setVillainEmpires] = useState(() => S.get("villain_empires", {}));
  const [orphanedCompanions, setOrphanedCompanions] = useState(() => S.get("orphaned", []));
  const [dailyShop, setDailyShop] = useState(() => S.get("daily_shop", null));
  const [activeBounty, setActiveBounty] = useState(() => S.get("active_bounty", null));
  const [raidHistory, setRaidHistory] = useState(() => S.get("raid_history", []));

  // ——— UI STATE ———

  const [tab, setTab] = useState("players");
  const [drawerOpen, setDrawerOpen] = useState(false);
  const [playerModallId, setPlayerModallId] = useState(null);
  const [addPlayerOpen, setAddPlayerOpen] = useState(false);

  // Sub-tab state per panel

  const [mapSelectedLoc, setMapSelectedLoc] = useState(null);
  const [mapSubTab, setMapSubTab] = useState("terrain");
  const [diceRoll, setDiceRoll] = useState(null);
  const [diceSpinning, setDiceSpinning] = useState(false);
  const [diceHistory, setDiceHistory] = useState([]);
  const [diceCtx, setDiceCtx] = useState({player:"",purpose:"",target:"",custom:""});
  const [classTab, setClassTab] = useState("Warrior");
  const [classSec, setClassSec] = useState("overview");
  const [rosterTab, setRosterTab] = useState("living");
  const [rulesTab, setRulesTab] = useState("economy");
  const [hqTab, setHqTab] = useState("facilities");
  const [villainTab, setVillainTab] = useState(null);
  const [villainSec, setVillainSec] = useState("lore");
  const [faqOpen, setFaqOpen] = useState(null);

```

```

const [logFilter, setLogFilter] = useState("all");
const [logSearch, setLogSearch] = useState("");
const [arsenalCls, setArsenalCls] = useState("Warrior");
const [compendiumTab, setCompendiumTab] = useState("races");
const [merchantBuyer, setMerchantBuyer] = useState("");
const [lootFinder, setLootFinder] = useState("");
const [raceTab, setRaceTab] = useState(Object.keys(RACES)[0]);
const [compTab, setCompTab] = useState(Object.keys(COMPANION_TYPES)[0]);
const [npcTab, setNPCTab] = useState("npcs");
const [shadowResult, setShadowResult] = useState("");
const [manualLog, setManualLog] = useState({who:"",what:"",type:"note"});
const [battleTimer, setBattleTimer] = useState(180);
const [battleRunning, setBattleRunning] = useState(false);
const battleRef = useRef(null);
const [bunkPlayer, setBunkPlayer] = useState("");
const [showAddResident, setShowAddResident] = useState(false);
const [residentForm, setResidentForm] = useState({name:"",role:"guard"});
const [addItemTarget, setAddItemTarget] = useState(null);
const [addItemForm, setAddItemForm] = useState({name:"",type:"weapon",rarity:"Common",condition:"Pristine",note:""});

// ——— PERSIST ———
useEffect(() => { S.set("players", players); }, [players]);
useEffect(() => { S.set("session_log", sessionLog); }, [sessionLog]);
useEffect(() => { S.set("session_active", sessionActive); }, [sessionActive]);
useEffect(() => { S.set("session_number", sessionNumber); }, [sessionNumber]);
useEffect(() => { S.set("map_pins", mapPins); }, [mapPins]);
useEffect(() => { S.set("map_danger", mapDanger); }, [mapDanger]);
useEffect(() => { S.set("base_rooms", baseRooms); }, [baseRooms]);
useEffect(() => { S.set("base_residents", baseResidents); }, [baseResidents]);
useEffect(() => { S.set("villains_live", villainsLive); }, [villainsLive]);
useEffect(() => { S.set("villain_phases", villainPhases); }, [villainPhases]);
useEffect(() => { S.set("villain_empires", villainEmpires); }, [villainEmpires]);
useEffect(() => { S.set("orphaned", orphanedCompanions); }, [orphanedCompanions]);
useEffect(() => { S.set("daily_shop", dailyShop); }, [dailyShop]);
useEffect(() => { S.set("active_bounty", activeBounty); }, [activeBounty]);
useEffect(() => { S.set("raid_history", raidHistory); }, [raidHistory]);

// ——— LOG HELPER ———
const addLog = useCallback((entry) => {
const e = { id: Date.now() + Math.random(), time: new Date().toLocaleTimeString([],{hour:"2-digit",minute:"2-digit",second:"2-digit"}), session:
sessionActive ? sessionNumber : "Prep", ...entry };
setSessionLog(prev => [e, ...prev].slice(0, 300));
}, [sessionActive, sessionNumber]);

// ——— PLAYER HELPERS ———

```

```

const updatePlayer = useCallback((id, updates) => {
  setPlayers(prev => prev.map(p => p.id === id ? {...p, ...updates} : p));
}, []);

// ——— DICE ———
const rollDice = useCallback(() => {
  if (diceSpinning) return;
  setDiceSpinning(true);
  setTimeout(() => {
    const result = d20();
    setDiceRoll(result);
    setDiceSpinning(false);

    const purpose = diceCtx.custom || diceCtx.target || ROLL_PURPOSES[diceCtx.purpose]?.label || "General Action";
    setDiceHistory(prev => [{result, player: diceCtx.player||"DM", purpose, time: new Date().toLocaleTimeString([],{hour:"2-digit",minute:"2-digit"})}, ...
    prev].slice(0,15));

    if (sessionActive) addLog({type:"dice", who: diceCtx.player||"DM", what: `Rolled a d20 for ${purpose}` , result});
  }, 600);
}, [diceSpinning, diceCtx, sessionActive, addLog]);

const rollShadow = useCallback(() => {
  const r = d20();
  const pl = players.filter(p => !p.isDead);
  if (!pl.length) { setShadowResult("No active players."); return; }
  const target = pl[Math.floor(Math.random() * pl.length)];
  if (r >= 15) setShadowResult(`✓ ${target.charname} resists the Shadow (rolled ${r}).`);
  else setShadowResult(`△ ${target.charname}'s Shadow MANIFESTS! (rolled ${r}) — "${target.shadowTrait || "Unknown Shadow"}"`);
}, [players]);

// ——— BATTLE TIMER ———
useEffect(() => {
  if (battleRunning) {
    battleRef.current = setInterval(() => {
      setBattleTimer(t => { if (t <= 1) { setBattleRunning(false); clearInterval(battleRef.current); return 0; } return t - 1; });
    }, 1000);
  } else {
    clearInterval(battleRef.current);
  }
  return () => clearInterval(battleRef.current);
}, [battleRunning]);

// ——— SESSION ———
const beginSession = () => { setSessionActive(true); addLog({type:"session",who:"System",what: `Session ${sessionNumber} began.`}); };
const endSession = () => {
  if (!window.confirm("End current session?")) return;
  addLog({type:"session",who:"System",what: `Session ${sessionNumber} ended.`});
};

```

```

setSessionActive(false);
setSessionNumber(n => n + 1);
};

// ——— VILLAIN HELPERS ———
const toggleVillain = (id) => {
setVillainsLive(prev => prev.includes(id) ? prev.filter(x => x !== id) : [...prev, id]);
};
const setVillainPhase = (id, phase) => setVillainPhases(prev => ({...prev, [id]: phase}));
const adjEmpire = (id, type, delta) => {
setVillainEmpires(prev => {
const cur = prev[id] || {territory:100, lieutenants:3};
return {...prev, [id]: {...cur, [type]: type === "territory" ? clamp(cur.territory + delta, 0, 100) : Math.max(0, cur.lieutenants + delta)}});
});
};

/* —————
PLAYER CREATION FORM
————— */

const [creation, setCreation] = useState({username:"",charname:"",cls:"",race:"",origin:"",payment:
"tiktok",notes:"",repRoll:null,traitRoll:null,weaponRoll:null,skillRoll:null,shadowRoll:null,enchantRoll:null,destinyRoll:null});
const setC = (k, v) => setCreation(prev => ({...prev, [k]: v}));
const rollC = (field) => setCreation(prev => ({...prev, [field]: d20()}));

const submitPlayer = () => {
const c = creation;
if (!c.charname || !c.cls || !c.race) { alert("Name, Class, and Race required."); return; }
const getTrait = (r) => TRAIT_TIERS.find(t => r >= t.range[0] && r <= t.range[1]);
const newP = {
id: Date.now(), joined: new Date().toLocaleDateString(),
hp: CLASS_HP[c.cls], maxHp: CLASS_HP[c.cls],
protectionBalance: 0, inventory: [], consequences: [], isDead: false, giftPowerUsed: false,
advancement: {raids:0, battles:0},
...C,
traitLevel: c.traitRoll ? getTrait(c.traitRoll)?.level : "✓ Standard"
};
setPlayers(prev => [newP, ...prev]);
addLog({type:"note", who:newP.charname, what:`Spawned. Class: ${c.cls}. Race: ${c.race}. Paid: ${c.payment === "paypal" ? "$13.00 CAD" : "1,000 Coins"}.`});
setCreation({username:"",charname:"",cls:"",race:"",origin:"",payment:
"tiktok",notes:"",repRoll:null,traitRoll:null,weaponRoll:null,skillRoll:null,shadowRoll:null,enchantRoll:null,destinyRoll:null});
setAddPlayerOpen(false);
setTab("players");
};

/* —————

```

HP ADJUSTMENT

```

    */

const adjHp = (id, delta) => {
  setPlayers(prev => prev.map(p => {
    if (p.id !== id) return p;
    const hp = clamp((p.hp || 0) + delta, 0, p.maxHp);
    if (sessionActive) addLog({type:"hp", who:p.charname, what:`HP ${delta > 0 ? "+" : ""}${delta} → ${hp}/${p.maxHp}`});
    return {...p, hp};
  }));
};

const markDead = (id) => {
  const p = players.find(x => x.id === id);
  if (!p || !window.confirm(`Mark ${p.charname} as permanently DEAD?`)) return;
  const cause = window.prompt("Cause of Death?") || "Claimed by the Void";
  updatePlayer(id, {isDead:true, deathCause:cause, deathSession:sessionNumber});
  if (p.companion) setOrphanedCompanions(prev => [...prev, {...p.companion, formerOwner:p.charname}]);
  addLog({type:"death", who:p.charname, what:`PERMANENTLY DEAD. Cause: ${cause}`});
};

const toggleHoF = (id) => {
  const p = players.find(x => x.id === id);
  if (!p) return;
  updatePlayer(id, {isHoF: !p.isHoF});
  addLog({type:"gift", who:p.charname, what:p.isHoF ? "Removed from Hall of Fame." : "INDUCTED into Hall of Fame! +3 CHA forever."});
};

const addConsequence = (id, type, label) => {
  const p = players.find(x => x.id === id);
  if (!p) return;
  const cons = [...(p.consequences||[]), {type, label}];
  updatePlayer(id, {consequences: cons});
  addLog({type:"consequence", who:p.charname, what:`Gained: ${label}`});
};

const rmConsequence = (id, idx) => {
  const p = players.find(x => x.id === id);
  if (!p) return;
  const label = p.consequences[idx]?.label;
  updatePlayer(id, {consequences: p.consequences.filter((_,i)=>i!==idx)});
  addLog({type:"consequence", who:p.charname, what:`Cleared: ${label}`});
};

/*
INVENTORY
```

```

    _____ */
const addItem = (id) => {
  if (!addItemForm.name.trim()) return;
  const p = players.find(x => x.id === id);
  if (!p) return;
  const item = {...addItemForm, name: addItemForm.name.trim()};
  updatePlayer(id, {inventory: [...(p.inventory||[]), item]});
  addLog({type: "item", who: p.charname, what: `Obtained: ${item.name}`});
  setAddItemForm({name: "", type: "weapon", rarity: "Common", condition: "Pristine", note: ""});
  setAddItemTarget(null);
};

const rmItem = (playerId, idx) => {
  const p = players.find(x => x.id === playerId);
  if (!p) return;
  const item = p.inventory[idx];
  updatePlayer(playerId, {inventory: p.inventory.filter((_, i) => i !== idx)});
  addLog({type: "item", who: p.charname, what: `Lost: ${item?.name||"item"}`});
};

const toggleEquip = (playerId, idx) => {
  const p = players.find(x => x.id === playerId);
  if (!p) return;
  const inv = p.inventory.map((item, i) => ({...item, equipped: i === idx ? !item.equipped : (item.type === p.inventory[idx].type ? false : item.equipped)}));
  updatePlayer(playerId, {inventory: inv});
};

/* _____
COMPANION
    _____ */

const assignCompanion = (id, type) => {
  const cData = COMPANION_TYPES[type];
  updatePlayer(id, {companion: {type, hp: cData.hp, maxHp: cData.hp, status: "active"}});
  addLog({type: "gift", who: players.find(p => p.id === id)?.charname||"", what: `Bonded companion: ${cData.icon} ${type}`});
};

const adjCompHp = (id, delta) => {
  const p = players.find(x => x.id === id);
  if (!p?.companion) return;
  const hp = clamp(p.companion.hp + delta, 0, p.companion.maxHp);
  const status = hp <= 0 ? "fled" : "active";
  updatePlayer(id, {companion: {...p.companion, hp, status}});
  if (hp <= 0) addLog({type: "hp", who: p.charname, what: `Companion ${p.companion.type} fled to HQ at 0 HP.`});
};

```

```

/* -----
HQ
----- */

const upgradeRoom = (room) => {
const cur = baseRooms[room] || 0;
const tiers = BASE_ROOMS[room]?.tiers;
if (!tiers || cur >= tiers.length) return;
const cost = tiers[cur]?.cost;
if (!window.confirm(`Upgrade ${BASE_ROOMS[room].name} to Level ${cur+1}? Cost: ${cost?.toLocaleString() || 0} coins`)) return;
setBaseRooms(prev => ({...prev, [room]: cur+1}));
addLog({type:"gift", who:"HQ", what:`${BASE_ROOMS[room].name} upgraded to Level ${cur+1}.`});
};

const addResident = () => {
if (!residentForm.name.trim()) return;
setBaseResidents(prev => [...prev, {...residentForm, id:Date.now()}]);
setResidentForm({name:"",role:"guard"});
setShowAddResident(false);
};

const rollDailyShop = () => {
const pick = (arr) => arr[Math.floor(Math.random()*arr.length)];
const shop = {bargain:{name:pick(LEDGER_POOLS.bargain),price:500},essential:{name:pick(LEDGER_POOLS.essential),price:2000},artifact:
{name:pick(LEDGER_POOLS.artifact),price:8000},rolledAt:new Date().toLocaleDateString()};
setDailyShop(shop);
addLog({type:"gift", who:"Ledger", what:"Daily Ledger refreshed with new stock"});
};

const rollBounty = () => {
const b = WANTED_BOUNTIES[Math.floor(Math.random()*WANTED_BOUNTIES.length)];
setActiveBounty({...b, time:new Date().toLocaleTimeString()});
addLog({type:"note", who:"Bounty Board", what:`New Target: ${b.name} at ${b.location}.`});
};

/* =====
RENDER PANELS
===== */

/* ——— PLAYERS PANEL ——— */
const PlayersPanel = () => {
const living = players.filter(p => !p.isDead);
const dead = players.filter(p => p.isDead);
const hof = living.filter(p => p.isHoF);
const current = rosterTab === "living" ? living : rosterTab === "dead" ? dead : hof;

```

...

```
return (
<div className="fade-in">
  <div style={{display:"flex",justifyContent:"space-between",alignItems:"center",marginBottom:16}}>
    <div>
      <div className="section-title" style={{marginBottom:2}}>✖ The Roster</div>
      <div style={{fontFamily:"Cinzel",serif",fontSize:"0.52rem",color:"var(--parch-shadow)",letterSpacing:"1px"}}>
        {living.length} Alive · {dead.length} Dead · Session {sessionNumber}
      </div>
    </div>
    <div>
      <button className="rune-btn" onClick={() => setAddPlayerOpen(true)}>+ Forge Character</button>
    </div>
  </div>

  {/* Session control */}
  <div className="dark-card glow" style={{marginBottom:16,display:"flex",alignItems:"center",justifyContent:"space-between",flexWrap:"wrap",gap:8}}>
    <div style={{display:"flex",alignItems:"center",gap:10}}>
      <div className={`session-dot${sessionActive?"":" off"}`} />
      <div>
        <div style={{fontFamily:"Cinzel",serif",fontSize:"0.62rem",color:"var(--ember-glow)",letterSpacing:"1px"}}>
          {sessionActive ? `SESSION ${sessionNumber} — LIVE` : "SESSION OFFLINE"}
        </div>
      </div>
    </div>
    <div style={{display:"flex",gap:8}}>
      {!sessionActive
        ? <button className="rune-btn success sm" onClick={beginSession}>▶ Begin Session</button>
        : <button className="rune-btn danger sm" onClick={endSession}>■ End Session</button>
      }
    </div>
  </div>

  {/* Roster tabs */}
  <div className="scroll-tabs" style={{marginBottom:12}}>
    {[["living","Living"],["dead","Hall of the Fallen"],["hof","Hall of Fame"]].map(([k,l]) => (
      <div key={k} className={`scroll-tab${rosterTab===k?" active":""}`} onClick={() => setRosterTab(k)}>{l}</div>
    ))}
  </div>

  {current.length === 0 && <div className="empty-state">No characters in this roster.<br/><em>Forge one to begin.</em></div>}

  {current.map(p => {
    const stats = getPlayerStats(p);
    const hpPct = Math.max(0, Math.min(100, ((p.hp||0)/(p.maxHp||1))*100));
    const hpColor = hpPct > 60 ? "#4caf50" : hpPct > 30 ? "#ff9800" : "#ef5350";
```

```

const trait = TRAIT_TIERS.find(t => t.level === p.traitLevel);

return (
  <div key={p.id} className={`player-card${p.isDead?" dead":""}${p.isHoF?" hof":""}`}>
    { /* Header */ }
    <div style={{display:"flex",justifyContent:"space-between",alignItems:"flex-start",marginBottom:8}}>
      <div style={{flex:1}}>
        <div style={{display:"flex",alignItems:"center",gap:8,flexWrap:"wrap"}}>
          <span style={{fontFamily:"Cinzel",serif,fontSize:"0.85rem",fontWeight:"700",color:p.isHoF?"var(--gold-bright)":var(--ember-glow)},letterSpacing:"1px"}}>
            {CLASS_ICON_EMO[p.cls]} {p.charname}
          </span>
          {p.isHoF && <span className="tag tag-hof">HALL OF FAME</span>}
          {p.isDead && <span className="tag tag-wanted">💀 DEAD</span>}
          {p.consequences?.map((c,i) => <span key={i} className={`tag tag-${c.type}`}>{c.label}</span>)}
        </div>
        {p.username && <div style={{fontFamily:"Cinzel",serif,fontSize:"0.48rem",color:"var(--parch-shadow)",letterSpacing:"1px",marginTop:2}}>@{p.username} · {p.race||""} {p.cls}</div>}
        {p.title && <div style={{fontFamily:"IM Fell English",serif,fontStyle:"italic",fontSize:"0.65rem",color:"var(--void-glow)",marginTop:2}}>{p.title}</div>}
      </div>
      {trait && <div style={{fontFamily:"Cinzel",serif,fontSize:"0.5rem",color:trait.color,marginTop:4,letterSpacing:"0.5px"}}>Trait: {trait.level}</div>}
    </div>
    <div style={{textAlign:"right",flexShrink:0}}>
      <div style={{fontFamily:"Cinzel",serif,fontSize:"0.55rem",color:"var(--parch-shadow)",marginBottom:2}}>HP</div>
      <div style={{fontFamily:"Cinzel Decorative",serif,fontSize:"1.1rem",color:hpColor,fontWeight:"900",lineHeight:1}}>
        {p.hp || 0}<span style={{fontSize:"0.6rem",color:"var(--parch-shadow)"},">/{p.maxHp||0}</span>
      </div>
      <div style={{display:"flex",gap:4,marginTop:6,justifyContent:"flex-end"}}>
        <button className="rune-btn danger sm" onClick={() => adjHp(p.id,-1)}>-</button>
        <button className="rune-btn success sm" onClick={() => adjHp(p.id,1)}>+</button>
      </div>
    </div>
  </div>

  { /* HP bar */ }
  <div className="hp-track">
    <div className="hp-fill" style={{width:hpPct+"%",background:`linear-gradient(90deg,${hpColor}80,${hpColor})`}}>
    </div>

  { /* Stats */ }
  <div className="stat-grid" style={{marginTop:10}}>
    {Object.entries(stats).map(([s,v]) => (
      <div key={s} className="stat-cell">
        <div className="stat-label" style={{color:STAT_COLORS[s]}}>{s}</div>
        <div className="stat-val" style={{color:STAT_COLORS[s]}}>{v}{s===PRIMARY_STAT[p.cls]?"★":""}</div>
      </div>
    ))}
  </div>

```

```

    )))
  </div>

  { /* Companion */
  { p.companion && (
    <div style={{marginTop:10,background:"rgba(0,0,0,0.3)",borderRadius:3,padding:"6px 10px",border:"1px solid
    rgba(138,90,220,0.2)",display:"flex",alignItems:"center",gap:10}}>
      <span style={{fontSize:"1rem"}}>{COMPANION_TYPES[p.companion.type]?.icon}</span>
      <div style={{flex:1}}>
        <div style={{fontFamily:"'Cinzel',serif",fontSize:"0.52rem",color:"var(--void-glow)",letterSpacing:"1px"}}>{p.companion.type}</div>
        <div className="hp-track" style={{marginTop:3}}>
          <div className="hp-fill" style={{width:`$(((p.companion.hp||0)/(p.companion.maxHp||1))*100)%`,background:"linear-
          gradient(90deg,#7c4dff80,#7c4dff)"}>
            </div>
          </div>
          <span style={{fontFamily:"'Cinzel',serif",fontSize:"0.52rem",color:p.companion.status==="active"?"#4caf50":"#ef5350"}}>
            {p.companion.status==="active"? "Active": "Fled"}</span>
          <div style={{display:"flex",gap:3}}>
            <button className="rune-btn danger sm" style={{padding:"3px 7px"}} onClick={() => adjCompHp(p.id,-1)}>-</button>
            <button className="rune-btn success sm" style={{padding:"3px 7px"}} onClick={() => adjCompHp(p.id,1)}>+</button>
          </div>
        </div>
      )}

    { /* Actions */
    <div style={{display:"flex",gap:6,marginTop:10,flexWrap:"wrap"}}>
      <button className="rune-btn sm" onClick={() => setPlayerModalId(p.id)}>📄 Full Sheet</button>
      <button className="rune-btn sm" onClick={() => setAddItemTarget(p.id)}>+ Item</button>
      {!p.isHoF && !p.isDead && <button className="rune-btn sm" style={{borderColor:"var(--gold)",color:"var(--gold-bright)"}>
        toggleHoF(p.id)}>★ HoF</button>}
      {p.isHoF && <button className="rune-btn sm" onClick={() => toggleHoF(p.id)}>Remove HoF</button>}
      {!p.isDead && <button className="rune-btn danger sm" onClick={() => markDead(p.id)}>💀 Kill</button>}
    </div>
  </div>

  );
  }}}
</div>

);
...

};

/* — MAP PANEL — */
const MapPanel = () => {
  const sel = mapSelectedLoc ? WORLD_LOCATIONS.find(l => l.id === mapSelectedLoc) : null;
  return (

```

```

<div className="fade-in">
<div className="section-title">🏰 The Hollow Realm — World Map</div>
<div className="info-bar">Click a location node to view details. Drag to orbit · Scroll to zoom.</div>
<div className="map-canvas-wrap">
<Canvas camera={{position:[0,6,8],fov:50}}>
<Suspense fallback={null}>
<Map3DScene mapDanger={mapDanger} players={players} selectedLoc={mapSelectedLoc} onSelectLoc={setMapSelectedLoc} />
</Suspense>
</Canvas>
</div>

...

{/* Location detail */}
{sel && (
  <div className="dark-card glow fade-in" style={{marginTop:14,borderColor:`${sel.color}60`}}>
    <div style={{display:"flex",justifyContent:"space-between",alignItems:"flex-start",marginBottom:10}}>
      <div>
        <div style={{fontFamily:"'Cinzel',serif",fontSize:"0.85rem",fontWeight:"700",color:sel.color,letterSpacing:"2px"}}>{sel.icon} {sel.name}</div>
        <div style={{fontFamily:"'IM Fell English',serif",fontStyle:"italic",fontSize:"0.7rem",color:"var(--parch-shadow)",marginTop:3}}>{sel.desc}</div>
      </div>
      <div>
        <div style={{fontFamily:"'Cinzel',serif",fontSize:"0.55rem",color:DANGER_COLORS[mapDanger[sel.id]??sel.danger],border:`1px solid ${DANGER_COLORS[mapDanger[sel.id]??sel.danger]}`,padding:"3px 10px",borderRadius:12,letterSpacing:"1px"}}>
          ⚠ {DANGER_LABELS[mapDanger[sel.id]??sel.danger]}
        </div>
        <div style={{display:"flex",gap:4,marginTop:6}}>
          {[1,2,3,4,5].map(d => (
            <button key={d} className="rune-btn sm" style={{padding:"3px 7px",borderColor:(mapDanger[sel.id]??sel.danger)===d?
DANGER_COLORS[d]:"var(--border-rune)",color:(mapDanger[sel.id]??sel.danger)===d?DANGER_COLORS[d]:"var(--parch-shadow)"} onlick={() =>
setMapDanger(prev=>({...prev,[sel.id]:d}))}>{d}</button>
          )))}
        </div>
      </div>
    </div>
    <div className="rune-divider"><span>Features</span></div>
    {sel.features.map((f,i) => <div key={i} style={{fontFamily:"'IM Fell English',serif",fontSize:"0.7rem",color:"var(--parch)",padding:"3px 0",borderBottom:"1px solid rgba(200,129,58,0.06)"}>• {f}</div>)}
    {sel.id === "hq" && (
      <button className="rune-btn full" style={{marginTop:12}} onlick={() => setTab("hq")}>🏰 Enter Moon Squad HQ</button>
    )}
  </div>
)}

{/* Bounty board */}
<div className="dark-card" style={{marginTop:14}}>
  <div style={{display:"flex",justifyContent:"space-between",alignItems:"center",marginBottom:10}}>

```

```

<div className="section-title" style={{marginBottom:0}}><img alt="Bounty Board icon" data-bbox="428 91 448 103"/> Bounty Board</div>
<div style={{display:"flex",gap:6}}>
  <button className="rune-btn sm" onClick={rollBounty}>Roll Bounty</button>
  {activeBounty && <button className="rune-btn danger sm" onClick={() => { addLog({type:"action",who:"Moon Squad",what:`Bounty claimed:
${activeBounty.name}`}); setActiveBounty(null); }}>Claim</button>}
</div>
</div>
{activeBounty
  ? <div style={{background:"rgba(155,27,27,0.1)",border:"1px solid rgba(155,27,27,0.3)",borderRadius:3,padding:"10px 12px"}}>
    <div style={{fontFamily:"'Cinzel',serif",fontSize:"0.72rem",fontWeight:"700",color:"#ef9a9a",letterSpacing:"1px"}}>{activeBounty.name}</div>
    <div style={{fontFamily:"'IM Fell English',serif",fontSize:"0.68rem",color:"var(--parch)",marginTop:4}}>{activeBounty.type} · {activeBounty.location}</div>
  </div>
  <div style={{display:"flex",gap:12,marginTop:6}}>
    <span style={{fontFamily:"'Cinzel',serif",fontSize:"0.55rem",color:"var(--gold-bright)"}}>Reward: {activeBounty.reward}</span>
    <span style={{fontFamily:"'Cinzel',serif",fontSize:"0.55rem",color:"#ef9a9a"}}>DC {activeBounty.dc}</span>
  </div>
</div>
: <div className="empty-state" style={{padding:"16px"}}>No active bounty. Roll the board.</div>
}
</div>
</div>
);
...

};

/* — DICE PANEL — */
const DicePanel = () => {
  const display = getDiceDisplay(diceRoll);
  const glow = diceSpinning ? "#b388ff" : display?.glow || "rgba(200,129,58,0.3)";
  const fc = diceSpinning ? "#3d2b6b" : !diceRoll ? "#2a1f5e" : diceRoll===1?"#5d0a0a":diceRoll<=4?"#4a1010":diceRoll<=9?"#4a2a08":diceRoll<=14?"#0a3d1a":diceRoll<=19?"#0a2040":"#2a0555";
  ...
  return (
    <div className="fade-in">
      <div className="section-title"><img alt="Dice icon" data-bbox="278 731 298 743"/> The Oracle's Dice</div>
      <div className="dark-card glow" style={{textAlign:"center",paddingTop:20,paddingBottom:20}}>
        <svg viewBox="0 0 120 120" width={160} height={160} style={{display:"block",margin:"0 auto",filter:`drop-shadow(0 0 ${diceSpinning?12:display?18:8}px ${glow})`,animation:diceSpinning?"dice-spin 0.15s linear infinite":"none",transformOrigin:"60px 60px"}}>
          <defs><radialGradient id="dg" cx="50%" cy="40%" r="60%"><stop offset="0%" stopColor={fc}/><stop offset="100%" stopColor="#050510"/></radialGradient></defs>
          <circle cx="60" cy="60" r="56" fill="none" stroke={glow} strokeWidth="0.5" opacity="0.4"/>
          <polygon points="60,8 106,37 91,88 29,88 14,37" fill="url(#dg)" stroke={glow} strokeWidth="1.2"/>
          <polygon points="60,8 80,42 40,42" fill="none" stroke={` ${glow}60`} strokeWidth="0.6"/>
          <polygon points="14,37 40,42 29,88" fill="none" stroke={` ${glow}40`} strokeWidth="0.6"/>
        </svg>
      </div>
    </div>
  );
}

```

```

<polygon points="106,37 80,42 91,88" fill="none" stroke={` ${glow}40`} strokeWidth="0.6"/>

<text x="60" y="65" textAnchor="middle" dominantBaseline="middle" fontSize={diceRoll>=10?22:28} fontWeight="bold" fill=
{diceSpinning?"#b388ff80":display?display.glow:"#4a3f7a"} fontFamily="Georgia,serif" style={{userSelect:"none"}}>
  {diceSpinning?"?":diceRoll||"d20"}
</text>
</svg>

{display && !diceSpinning && (
  <div className="fade-in" style={{margin:"12px 0",padding:"14px 20px",background:display.bg,border:`1px solid
${display.glow}`,borderRadius:4,boxShadow:`0 0 30px ${display.glow}40`}}>
    <div style={{fontFamily:"'Cinzel Decorative',serif",fontSize:"1.2rem",fontWeight:"bold",color:"fff",letterSpacing:"2px"}}>{display.label}</div>
    <div style={{fontFamily:"'IM Fell English',serif",fontSize:"0.74rem",color:"rgba(255,255,255,0.7)",marginTop:4}}>{display.desc}</div>
    {diceCtx.player && <div style={{fontFamily:"'Cinzel',serif",fontSize:"0.55rem",color:"rgba(255,255,255,0.4)",marginTop:4,letterSpacing:"1px"}}>
{diceCtx.player} · {diceCtx.custom||diceCtx.target||ROLL_PURPOSES[diceCtx.purpose]?.label||""}</div>}
  </div>
)}

<button className="rune-btn" style={{fontSize:"0.85rem",padding:"12px 36px",letterSpacing:"2px",marginTop:8,opacity:diceSpinning?0.5:1}}
onClick={rollDice} disabled={diceSpinning}>
  {diceSpinning?"ROLLING...":"ROLL THE DICE"}
</button>
</div>

{/* Shadow trigger */}
<div className="dark-card" style={{borderColor:"rgba(155,27,27,0.4)",marginBottom:12}}>
  <div style={{display:"flex",justifyContent:"space-between",alignItems:"center"}}>
    <div className="section-title" style={{marginBottom:0,color:"#ef9a9a"}}><img alt="Shadow Trigger icon" data-bbox="515 542 535 555"/> Shadow Trigger</div>
    <button className="rune-btn danger sm" onClick={rollShadow}>Test Shadow</button>
  </div>
  {shadowResult && <div style={{marginTop:8;fontFamily:"'IM Fell English',serif",fontSize:"0.75rem",color:"var(--parch)",fontStyle:"italic"}}>
{shadowResult}</div>}
  {!shadowResult && <div style={{marginTop:6;fontFamily:"'Cinzel',serif",fontSize:"0.52rem",color:"var(--parch-shadow)"}>Roll to see if a player's
Shadow Trait manifests (DC 15+ to resist)</div>}
</div>

{/* Context */}
<div className="dark-card">
  <div className="section-title">Roll Context</div>
  <div style={{marginBottom:10}}>
    <label className="form-label">For which player?</label>
    <select className="scroll-select" value={diceCtx.player} onChange={e => setDiceCtx(prev=>({...prev,player:e.target.value}))}>
      <option value="">DM / General Roll</option>
      {players.filter(p=>!p.isDead).map(p=><option key={p.id} value={p.charname}>{CLASS_ICON_EMO[p.cls]} {p.charname} {p.cls}</option>)}
    </select>
  </div>
  <div style={{marginBottom:10}}>
    <label className="form-label">Category</label>

```

```

<div style={{display:"grid",gridTemplateColumns:"repeat(3,1fr)","gap:4}}>

  (Object.entries(ROLL_PURPOSES).map(([k,cat])=>(

    <button key={k} className={`rune-btn sm${diceCtx.purpose===k?" ":""} `} style={{fontSize:"0.5rem",padding:"5px
4px",background:diceCtx.purpose===k?` ${cat.color}20`:"rgba(15,8,4,0.8)",color:diceCtx.purpose===k?cat.color:"var(--parch-
shadow)",borderColor:diceCtx.purpose===k?cat.color:"var(--border-rune)",clipPath:"none"}} onClick={()=>setDiceCtx(prev=>
({...prev,purpose:k,target:"",custom:""})}}>

      {cat.label}

    </button>

  )))

</div>

</div>

{diceCtx.purpose && diceCtx.purpose!=="custom" && (

  <div style={{marginBottom:10}}>

    <label className="form-label">Specific Action</label>

    <select className="scroll-select" value={diceCtx.target} onChange={e=>setDiceCtx(prev=>({...prev,target:e.target.value,custom:""}))}>

      <option value="">— Select —</option>

      {(ROLL_PURPOSES[diceCtx.purpose]?.options||[]).map(o=><option key={o} value={o}>{o}</option>)}

      <option value="custom">Custom...</option>

    </select>

  </div>

)}

{(diceCtx.purpose==="custom"||diceCtx.target==="custom") && (

  <div>

    <label className="form-label">Describe</label>

    <input className="scroll-input" placeholder="What is this roll for..." value={diceCtx.custom} onChange={e=>setDiceCtx(prev=>
({...prev,custom:e.target.value}))} />

  </div>

)}

</div>

{/* Roll reference */}

<div className="dark-card">

  <div className="section-title">Roll Reference</div>

  [{r:"1",l:"🎲 FUMBLE",c:"#ef5350",d:"Catastrophic backfire"},{r:"2–4",l:"× FAIL",c:"#ef5350",d:"Fails entirely"},{r:"5–9",l:"△ WEAK
HIT",c:"#ff9800",d:"Partial success"},{r:"10–14",l:"✓ SUCCESS",c:"#66bb6a",d:"Clean success"},{r:"15–19",l:"★ STRONG HIT",c:"#4fc3f7",d:"Success +
bonus"},{r:"20",l:"⚡ CRITICAL",c:"#ce93d8",d:"Legendary — announce it"}].map(({r,l,c,d})=>(

    <div key={r} style={{display:"flex",alignItems:"center",gap:10,padding:"6px 0",borderBottom:"1px solid rgba(200,129,58,0.06)}}>

      <span style={{fontFamily:"'Cinzel',serif",fontSize:"0.62rem",color:c,fontWeight:"700",minWidth:40}}>{r}</span>

      <span style={{fontFamily:"'Cinzel',serif",fontSize:"0.62rem",color:c,fontWeight:"700",minWidth:110}}>{l}</span>

      <span style={{fontFamily:"'IM Fell English',serif",fontSize:"0.68rem",color:"var(--parch-shadow)}}>{d}</span>

    </div>

  )))

</div>

{/* Dice history */}

{diceHistory.length > 0 && (

  <div className="dark-card">

```

```

<div style={{display:"flex",justifyContent:"space-between",alignItems:"center",marginBottom:8}}>
  <div className="section-title" style={{marginBottom:0}}>Recent Rolls</div>
  <button className="rune-btn sm" onClick={()=>setDiceHistory([])}>Clear</button>
</div>
{diceHistory.slice(0,10).map((e,i)=>{const dd=getDiceDisplay(e.result);return(
  <div key={i} style={{display:"flex",gap:10,alignItems:"center",padding:"5px 0",borderBottom:"1px solid rgba(200,129,58,0.06)}}>
    <span style={{fontFamily:"Cinzel",serif",fontSize:"0.7rem",fontWeight:"700",color:dd?.glow||"#aaa",minWidth:28,textAlign:"center"}}>{e.result}</span>
    <span style={{fontFamily:"Cinzel",serif",fontSize:"0.58rem",color:"var(--ember)",minWidth:80,letterSpacing:"0.5px"}}>{e.player}</span>
    <span style={{fontFamily:"IM Fell English",serif",fontSize:"0.65rem",color:"var(--parch-shadow)",flex:1}}>{e.purpose}</span>
    <span style={{fontFamily:"Cinzel",serif",fontSize:"0.48rem",color:"var(--parch-shadow)"}>{e.time}</span>
  </div>
  );}})
</div>
)}
</div>
);
...

};

/* —— LOG PANEL —— */
const LogPanel = () => {
  const filtered = sessionLog.filter(e => {
    const matchFilter = logFilter === "all" || e.type === logFilter;
    const matchSearch = !logSearch || (e.who+e.what).toLowerCase().includes(logSearch.toLowerCase());
    return matchFilter && matchSearch;
  });
  ...
  return (
    <div className="fade-in">
      <div className="section-title">📖 Session Log</div>
      <div style={{display:"flex",gap:8,marginBottom:12,flexWrap:"wrap"}}>
        <input className="scroll-input" placeholder="Search log..." value={logSearch} onChange={e=>setLogSearch(e.target.value)} style={{flex:1,minWidth:140}} />
        <select className="scroll-select" value={logFilter} onChange={e=>setLogFilter(e.target.value)} style={{width:120}}>
          <option value="all">All Types</option>
          {Object.entries(LOG_TYPES).map(([k,v])=><option key={k} value={k}>{v.icon} {v.label}</option>)}
        </select>
        <button className="rune-btn sm" onClick={() => { if(window.confirm("Clear log?")) setSessionLog([]); }}>Clear</button>
      </div>
      <div className="dark-card">
        <div className="section-title">Manual Entry</div>
        <div style={{display:"grid",gridTemplateColumns:"1fr 1fr",gap:8,marginBottom:8}}>

```

```

<div>
  <label className="form-label">Who</label>
  <input className="scroll-input" value={manualLog.who} onChange={e=>setManualLog(prev=>({...prev,who:e.target.value}))} placeholder="Player
or NPC name" />
</div>
<div>
  <label className="form-label">Type</label>
  <select className="scroll-select" value={manualLog.type} onChange={e=>setManualLog(prev=>({...prev,type:e.target.value}))}>
    {Object.entries(LOG_TYPES).map(([k,v])=><option key={k} value={k}>{v.icon} {v.label}</option>)}
  </select>
</div>
</div>
<label className="form-label">What happened</label>
<textarea className="scroll-textarea" value={manualLog.what} onChange={e=>setManualLog(prev=>({...prev,what:e.target.value}))}
placeholder="Describe the event..." style={{marginBottom:8}} />
<button className="rune-btn sm full" onClick={() => { if(!manualLog.who||!manualLog.what) return;
addLog({type:manualLog.type,who:manualLog.who,what:manualLog.what}); setManualLog({who:"",what:"",type:"note"}); }}>+ Add to Log</button>
</div>
{filtered.length === 0 && <div className="empty-state">No entries match your filters.</div>}
<div className="dark-card" style={{padding:"8px 12px"}}>
  {filtered.map(e => {
    const lt = LOG_TYPES[e.type] || LOG_TYPES.note;
    return (
      <div key={e.id} className="log-entry">
        <div className="log-icon">{lt.icon}</div>
        <div style={{flex:1}}>
          <div style={{display:"flex",alignItems:"center",gap:8}}>
            <span className="log-who" style={{color:lt.color}}>{e.who}</span>
            {e.result && <span style={{fontFamily:""Cinzel
Decorative',serif",fontSize:"0.7rem",color:getDiceDisplay(e.result)?.glow||"#aaa",fontWeight:"900"}}>{e.result}</span>}
          </div>
          <div className="log-what">{e.what}</div>
        </div>
        <div style={{display:"flex",flexDirection:"column",alignItems:"flex-end",flexShrink:0}}>
          <div className="log-time">{e.time}</div>
          <div style={{fontFamily:""Cinzel',serif",fontSize:"0.42rem",color:"var(--parch-shadow)",marginTop:2,letterSpacing:"0.5px"}}>S{e.session}</div>
        </div>
      </div>
    );
  })}
</div>
</div>
);
...
};

```

```

/* — COMBAT / BATTLE PANEL — */
const CombatPanel = () => {
  const mm = Math.floor(battleTimer/60), ss = battleTimer%60;
  return (
    <div className="fade-in">
      <div className="section-title">✂ Box Battle Arena</div>
      <div className="dark-card glow">
        <div className="session-timer">{String(mm).padStart(2,"0")}:{String(ss).padStart(2,"0")}</div>
        <div className="rune-divider"><span>Battle Timer</span></div>
        <div style={{display:"flex",gap:8,justifyContent:"center",flexWrap:"wrap"}}>
          <button className="rune-btn" onClick={()=>setBattleRunning(r=>!r)}>{battleRunning?"⏸ Pause":"▶ Start"}</button>
          <button className="rune-btn danger sm" onClick={()=>{setBattleRunning(false);setBattleTimer(180);}}>⏮ Reset</button>
        </div>
        {[60,90,120,180,300].map(s=><button key={s} className="rune-btn sm" onClick={()=>{setBattleRunning(false);setBattleTimer(s);}}>{Math.floor(s/60)}:
        {String(s%60).padStart(2,"0")}</button>)}
      </div>
    </div>
    <div className="dark-card">
      <div className="section-title">Consequence Ladder</div>
      [[{range:"< 5k gap",label:"MINOR",color:"#66bb6a",effects:["Lose title","Stat debuff"]},{range:"5k–9.9k gap",label:"MODERATE",color:"#ff9800",effects:
      ["Cursed 2 sessions","Item stolen"]},{range:"10k–19.9k gap",label:"MAJOR",color:"#ef5350",effects:["Exile","Servant status"]},{range:"20k–44.9k
      gap",label:"SEVERE",color:"#b71c1c",effects:["Death","Curse + exile + theft"]}],map(row=>{
        <div key={row.label} style={{display:"flex",alignItems:"flex-start",gap:12,padding:"8px 0",borderBottom:"1px solid rgba(200,129,58,0.08)"}}>
          <div style={{minWidth:100}}>
            <div style={{fontFamily:"Cinzel',serif",fontSize:"0.6rem",fontWeight:"700",color:row.color,letterSpacing:"1px"}}>{row.label}</div>
            <div style={{fontFamily:"Cinzel',serif",fontSize:"0.48rem",color:"var(--parch-shadow)",letterSpacing:"0.5px"}}>{row.range}</div>
          </div>
          <div style={{flex:1}}>
            {row.effects.map((e,i)=><div key={i} style={{fontFamily:"IM Fell English',serif",fontSize:"0.68rem",color:"var(--parch)"}}>• {e}</div>)}
          </div>
          <div style={{display:"flex",gap:4,flexWrap:"wrap"}}>
            {row.effects.map((eff,i)=>{
              <div key={i} style={{display:"flex",gap:3,alignItems:"center"}}>
                <select className="scroll-select" id={`cons-player-${row.label}-${i}`} style={{width:100,fontSize:"0.55rem",padding:"3px 6px"}}>
                  <option value="">Player</option>
                  {players.filter(p=>!p.isDead).map(p=><option key={p.id} value={p.id}>{p.charname}</option>)}
                </select>
                <button className="rune-btn sm" style={{fontSize:"0.48rem",padding:"3px 7px"}} onClick={()=>{
                  const sel=document.getElementById(`cons-player-${row.label}-${i}`);
                  if(!sel?.value) return;
                  const type=eff.toLowerCase().includes("curse")?"curse":eff.toLowerCase().includes("exile")?"exile":eff.toLowerCase().includes("servant")?"servant":
                  "wanted";
                  addConsequence(Number(sel.value),type,eff);
                }}>Apply</button>
              </div>
            )}
          </div>
        </div>
      )}
    </div>
  )
}

```

```

</div>
</div>
)}}
</div>
</div>
);
};

/* —— VILLAINS PANEL —— */
const VillainsPanel = () => {
  const active = VILLAIN_LORE.filter(v => villainsLive.includes(v.id));
  const inactive = VILLAIN_LORE.filter(v => !villainsLive.includes(v.id));
  const buildCard = (v, isActive) => {
    const phaseIdx = villainPhases[v.id] ?? 0;
    const phase = v.phases[phaseIdx];
    const emp = villainEmpires[v.id] || {territory:100, lieutenants:3};
    return (
      <div key={v.id} className="dark-card" style={{borderColor:`${v.color}${isActive?"80":"30"}`,opacity:isActive?1:0.7}}>
        <div style={{display:"flex",justifyContent:"space-between",alignItems:"center"}}>
          <div style={{display:"flex",gap:10,alignItems:"center"}}>
            <span style={{fontSize:"1.5rem",filter:isActive?"":"grayscale(1)"}}>{v.icon}</span>
          </div>
          <div style={{fontFamily:"'Cinzel',serif",fontSize:"0.78rem",fontWeight:"700",color:isActive?"#fff":"var(--parch-shadow)",letterSpacing:"1px"}}>{v.name}</div>
          <div style={{fontFamily:"'IM Fell English',serif",fontStyle:"italic",fontSize:"0.6rem",color:v.color}}>{v.title}</div>
        </div>
        <div>
          <button className={`rune-btn sm${isActive?" danger":""}`} onClick={()=>toggleVillain(v.id)}>{isActive?"Retire":"Activate"}</button>
        </div>
        {isActive && phase && (
          <>
            <div className="rune-divider"><span>Current Phase</span></div>
            <div style={{background:`${phase.color}12`,border:`1px solid ${phase.color}40`,borderRadius:3,padding:"10px 12px",marginBottom:10}}>
              <div style={{display:"flex",justifyContent:"space-between",alignItems:"center",marginBottom:4}}>
                <div style={{fontFamily:"'Cinzel',serif",fontSize:"0.62rem",color:phase.color,letterSpacing:"1px",fontWeight:"700"}}>{phase.name}</div>
                <div style={{fontFamily:"'Cinzel',serif",fontSize:"0.55rem",color:"var(--parch-shadow)"}>{phase.hp}</div>
              </div>
              <div style={{fontFamily:"'IM Fell English',serif",fontSize:"0.68rem",color:"var(--parch)"}>{phase.desc}</div>
            </div>
            <div className="phase-bar" style={{marginBottom:10}}>
              {v.phases.map((ph,i)=><div key={i} className="phase-pip" style={{background:i<=phaseIdx?ph.color:"rgba(255,255,255,0.08)"} onClick={()=>setVillainPhase(v.id,i)} />)}
            </div>
            <div style={{display:"grid",gridTemplateColumns:"1fr 1fr",gap:8,marginBottom:10}}>
              <div>

```

```

<div style={{fontFamily:"'Cinzel',serif",fontSize:"0.5rem",color:"var(--ember)",marginBottom:3,letterSpacing:"1px"}}>TERRITORY</div>

<div style={{fontFamily:"'Cinzel Decorative',serif",fontSize:"1rem",fontWeight:"700",color:emp.territory>50?"#ef5350":"var(--gold-bright)"}}>
{emp.territory}%</div>

<div style={{display:"flex",gap:4,marginTop:4}}>

<button className="rune-btn success sm" style={{flex:1,padding:"3px 5px",fontSize:"0.48rem"}} onClick={()=>adjEmpire(v.id,"territory",-10)}>-10%
</button>

<button className="rune-btn danger sm" style={{flex:1,padding:"3px 5px",fontSize:"0.48rem"}} onClick={()=>adjEmpire(v.id,"territory",10)}>+10%
</button>

</div>

</div>

<div>

<div style={{fontFamily:"'Cinzel',serif",fontSize:"0.5rem",color:"var(--ember)",marginBottom:3,letterSpacing:"1px"}}>LIEUTENANTS</div>

<div style={{fontFamily:"'Cinzel Decorative',serif",fontSize:"1rem",fontWeight:"700",color:emp.lieutenants>0?"#ef5350":"var(--parch-shadow)"}}>
{emp.lieutenants}</div>

<button className="rune-btn danger sm full" style={{marginTop:4,padding:"3px 5px",fontSize:"0.48rem"}} onClick={()=>adjEmpire(v.id,
"lieutenants",-1)}>Lt. Killed</button>

</div>

</div>

</>

)}

/* Accordion sub-sections */

[[["lore",🏰 Lore],["strengths",🗡️ Strengths],["phases",🔴 Phases],["loot",💎 Loot]].map(([,sec,label]) => (

<div key={sec} className={`accordion-item${villainTab===v.id&&villainSec===sec?" open":""}`} style={{marginBottom:4}}>

<div className="accordion-header" onClick={()=>{setVillainTab(v.id);setVillainSec(villainTab===v.id&&villainSec===sec?null:sec);}}>

<span style={{fontFamily:"'Cinzel',serif",fontSize:"0.55rem",color:"var(--ember-glow)",letterSpacing:"1px"}}>{label}</span>

<span className="accordion-chevron">▼</span>

</div>

{villainTab===v.id&&villainSec===sec && (

<div className="accordion-body fade-in">

{sec==="lore" && <><div style={{fontFamily:"'IM Fell English',serif",fontStyle:"italic",fontSize:"0.7rem",color:"var(--gold-bright)",borderLeft:"2px solid var(--gold)",paddingLeft:8,marginBottom:8}}>{v.motivation}</div><div style={{fontFamily:"'IM Fell English',serif",fontSize:"0.7rem",color:"var(--parch)",lineHeight:1.6}}>{v.lore}</div></>

{sec==="strengths" && <><div style={{marginBottom:8}}>{(v.strengths||[]).map((s,i)=><div key={i} style={{fontFamily:"'IM Fell English',serif",fontSize:"0.7rem",color:"#a5d6a7",marginBottom:3}}>+ {s}</div>)}</div><div style={{fontFamily:"'Cinzel',serif",fontSize:"0.52rem",color:"#ef9a9a",letterSpacing:"1px",marginBottom:4}}>WEAKNESSES</div>{(v.weaknesses||[]).map((w,i)=><div key={i} style={{fontFamily:"'IM Fell English',serif",fontSize:"0.7rem",color:"#ef9a9a",marginBottom:3}}>× {w}</div>)}</div>

{sec==="phases" && (v.phases||[]).map((ph,i)=><div key={i} style={{background:`${ph.color}12`,border:`1px solid ${ph.color}40`,borderRadius:3,padding:"8px 10px",marginBottom:6}}><div style={{display:"flex",justifyContent:"space-between",fontFamily:"'Cinzel',serif",fontSize:"0.6rem",color:ph.color,marginBottom:3}}><span>P{ph.phase}: {ph.name}</span><span>{ph.hp}</span></div><div style={{fontFamily:"'IM Fell English',serif",fontSize:"0.65rem",color:"var(--parch)"}>{ph.desc}</div></div>)}

{sec==="loot" && (v.loot||[]).map((l,i)=><div key={i} style={{display:"flex",justifyContent:"space-between",padding:"5px 0",borderBottom:"1px solid rgba(200,129,58,0.06)",fontFamily:"'IM Fell English',serif",fontSize:"0.68rem"}}><span style={{color:"var(--parch)"}>{l.item}</span><span style={{color:"var(--gold-bright)",flexShrink:0,marginLeft:8}}>{l.chance}</span></div>)}

</div>

)}

</div>

)})

</div>

);

};

```

```

...

return (
  <div className="fade-in">
    <div className="section-title">🔴 Villain Tracker</div>
    {active.length === 0 && <div className="info-bar">No active villains. Activate a villain from the Bestiary below.</div>}
    {active.map(v => buildCard(v, true))}
    <div className="rune-divider"><span>Bestiary — Inactive</span></div>
    {inactive.map(v => buildCard(v, false))}
  </div>
);
...

};

/* — HQ PANEL — */
const HQPanel = () => (
  <div className="fade-in">
    <div className="section-title">🏠 Moon Squad HQ</div>
    <div className="scroll-tabs" style={{marginBottom:14}}>
      {[["facilities","Facilities"],["ledger","The Ledger"],["bunkhouse","Bunkhouse"]].map(([k,l])=>{
        <div key={k} className={`scroll-tab${hqTab===k?" active":""}` onClick={()=>setHqTab(k)}>{l}</div>
      })}
    </div>

    ...

    {hqTab==="facilities" && (
      <div className="fade-in">
        <div className="dark-card" style={{marginBottom:12}}>
          <div className="section-title">🛡️ Base Defense</div>
          <div style={{display:"flex",gap:8,flexWrap:"wrap"}}>
            {Object.entries(BASE_ROOMS).map(([key,room])=>{
              const lv = baseRooms[key]||0;
              const maxLv = room.tiers.length;
              const nextTier = room.tiers[lv];
              return (
                <div key={key} style={{background:"rgba(10,5,2,0.8)",border:"1px solid var(--border-rune)",borderRadius:3,padding:"10px 12px",flex:"1 1 140px"}}>
                  <div style={{fontFamily:"'Cinzel',serif",fontSize:"0.62rem",color:"var(--ember-glow)",marginBottom:3}}>{room.icon} {room.name}</div>
                  <div style={{fontFamily:"'Cinzel',serif",fontSize:"0.5rem",color:"var(--parch-shadow)",marginBottom:6}}>Level {lv}/{maxLv}</div>
                  <div style={{fontFamily:"'IM Fell English',serif",fontSize:"0.65rem",color:"var(--parch)",marginBottom:8}}>{lv>0?room.tiers[lv-1].bonus:room.desc}</div>
                  {nextTier
                    ? <button className="rune-btn sm full" onClick={()=>upgradeRoom(key)}>Upgrade {nextTier.cost.toLocaleString()}c</button>
                    : <div style={{fontFamily:"'Cinzel',serif",fontSize:"0.52rem",color:"var(--gold-bright)",textAlign:"center",border:"1px solid var(--gold)",padding:"4px",borderRadius:2}}>MAXED</div>
                  }
                )
              }
            )}
          </div>
        </div>
      )
    )
  )
);

```

```

    }
  </div>
);
  }}}
</div>
</div>
<div className="dark-card">
  <div style={{display:"flex",justifyContent:"space-between",alignItems:"center",marginBottom:10}}>
    <div className="section-title" style={{marginBottom:0}}>👥 HQ Residents</div>
    <button className="rune-btn sm" onClick={()=>setShowAddResident(r=>!r)}>+ Hire</button>
  </div>
  {showAddResident && (
    <div className="fade-in" style={{background:"rgba(0,0,0,0.3)",border:"1px solid var(--border-rune)",borderRadius:3,padding:12;marginBottom:10}}>
      <div style={{display:"grid",gridTemplateColumns:"1fr 1fr",gap:8;marginBottom:8}}>
        <div><label className="form-label">Name</label><input className="scroll-input" value={residentForm.name} onChange=
        {e=>setResidentForm(r=>({...r,name:e.target.value}))} placeholder="NPC name" /></div>
        <div><label className="form-label">Role</label><select className="scroll-select" value={residentForm.role} onChange=
        {e=>setResidentForm(r=>({...r,role:e.target.value}))}>{Object.entries(RESIDENT_ROLES).map(([k,r])=><option key={k} value={k}>{r.icon} {r.label}
        </option>)}</select></div>
      </div>
      <button className="rune-btn success sm full" onClick={addResident}>Hire Resident</button>
    </div>
  )}
  {baseResidents.length===0 ? <div className="empty-state" style={{padding:16}}>No residents hired yet.</div> : baseResidents.map((r,i)=>{
    const role=RESIDENT_ROLES[r.role]||RESIDENT_ROLES.guard;
    return <div key={r.id|i} style={{display:"flex",justifyContent:"space-between",alignItems:"center",padding:"7px 0",borderBottom:"1px solid
    rgba(200,129,58,0.06)}">
      <span style={{fontFamily:"Cinzel",serif,fontSize:"0.6rem",color:"#fff"}}>{role.icon} <b>{r.name}</b> <span style={{color:"var(--parch-
      shadow)"}>{role.label}</span></span>
      <span style={{fontFamily:"Cinzel",serif,fontSize:"0.52rem",color:"var(--void-glow)"}>{role.bonus}</span>
      <button className="rune-btn danger sm" style={{padding:"2px 6px"}} onClick=
      {()=>setBaseResidents(prev=>prev.filter((_,j)=>j!==i))}>x</button>
    </div>;
  )}
</div>
</div>
)}

{hqTab==="ledger" && (
  <div className="fade-in">
    <div className="dark-card glow">
      <div style={{display:"flex",justifyContent:"space-between",alignItems:"center",marginBottom:12}}>
        <div className="section-title" style={{marginBottom:0,color:"var(--gold-bright)"}>📖 The Daily Ledger</div>
        <button className="rune-btn sm" onClick={rollDailyShop}>🎲 Roll Stock</button>
      </div>
      {!dailyShop

```

```

? <div className="empty-state" style={{padding:20}}>Ledger is empty. Roll to generate today's stock.</div>
: <>
  <div style={{fontFamily:"Cinzel',serif",fontSize:"0.48rem",color:"var(--parch-shadow)",marginBottom:10,letterSpacing:"1px"}}>CURRENT STOCK
— {dailyShop.rolledAt}</div>
  {[ "bargain","essential","artifact"].map(tier=>{
    const item=dailyShop[tier];
    const colors={bargain:"var(--parch-shadow)",essential:"var(--void-glow)",artifact:"var(--gold-bright)"};
    return <div key={tier} style={{background:"rgba(0,0,0,0.3)",border:`1px solid ${colors[tier]}30`,borderRadius:3,padding:"10px
12px",marginBottom:8,display:"flex",justifyContent:"space-between",alignItems:"center"}}>
      <div>
        <div style={{fontFamily:"Cinzel',serif",fontSize:"0.48rem",color:colors[tier],letterSpacing:"2px",textTransform:"uppercase",marginBottom:3}}>
{tier}</div>
        <div style={{fontFamily:"IM Fell English',serif",fontSize:"0.78rem",color:"#fff",fontWeight:"bold"}}>{item.name}</div>
      </div>
      <div style={{textAlign:"right"}}>
        <div style={{fontFamily:"Cinzel Decorative',serif",fontSize:"0.85rem",color:"var(--gold-bright)"}>{item.price.toLocaleString()}c</div>
        <div style={{fontFamily:"Cinzel',serif",fontSize:"0.48rem",color:"var(--parch-shadow)"}>${(item.price*0.013).toFixed(2)} PayPal</div>
      </div>
    </div>;
  )}
</>
}
</div>
</div>
)}

```

```

{hqTab==="bunkhouse" && (
  <div className="fade-in">
    <div className="dark-card" style={{marginBottom:12}}>
      <label className="form-label">Select Player</label>
      <select className="scroll-select" value={bunkPlayer} onChange={e=>setBunkPlayer(e.target.value)}>
        <option value="">— Select Player —</option>
        {players.filter(p=>!p.isDead).map(p=><option key={p.id} value={p.id}>{p.charname}</option>)}
      </select>
    </div>
    {bunkPlayer && (() => {
      const p = players.find(x=>x.id===bunkPlayer);
      if (!p) return null;
      const quarters = p.quarters||{bunk:0,lockbox:0,hearth:0};
      return Object.entries(PERSONAL_QUARTERS).map(([key,q])=>{
        const lv=quarters[key]||0;
        const next=q.tiers[lv];
        return <div key={key} className="dark-card glow" style={{marginBottom:12}}>
          <div style={{display:"flex",justifyContent:"space-between",alignItems:"center",marginBottom:8}}>
            <div>
              <div style={{fontFamily:"Cinzel',serif",fontSize:"0.72rem",color:"var(--ember-glow)"}>{q.icon} {q.name}</div>

```

```

        <div style={{fontFamily:"'Cinzel',serif",fontSize:"0.52rem",color:"var(--parch-shadow)",marginTop:2}}>Level {lv}</div>
    </div>
    {next
        ? <button className="rune-btn success sm" onClick={()=>{if(!window.confirm(`Spend ${next.cost}c to upgrade ${p.charname}'s ${q.name}?`))return;updatePlayer(p.id,{quarters:{...quarters,[key]:lv+1}});addLog({type:"gift",who:p.charname,what:`Upgraded ${q.name} to Level ${lv+1}.`});}}>Upgrade ({next.cost.toLocaleString()}c)</button>
        : <span style={{fontFamily:"'Cinzel',serif",fontSize:"0.52rem",color:"var(--gold-bright)",border:"1px solid var(--gold)",padding:"3px 8px"}}>MAXED</span>
    }
</div>

{q.tiers.map((tier,i)=><div key={i} style={{display:"flex",gap:8,alignItems:"center",padding:"5px 0",borderBottom:"1px solid rgba(200,129,58,0.06)",fontFamily:"'IM Fell English',serif",fontSize:"0.65rem"}}>
    <span style={{color:lv>=i+1?"#4caf50":"rgba(255,255,255,0.15)",fontSize:"0.7rem"}}>{lv>=i+1?"✓":"○"}</span>
    <span style={{color:"var(--parch-shadow)",minWidth:50}}>Lv.{i+1} ({tier.cost.toLocaleString()}c)</span>
    <span style={{color:"var(--parch)",flex:1}}>{tier.bonus}</span>
</div>)}
</div>;
})();
</div>
)}
</div>
...

);

/* ——— COMPENDIUM PANEL ——— */
const CompendiumPanel = () => (
    <div className="fade-in">
        <div className="section-title">📖 The Compendium</div>
        <div className="scroll-tabs" style={{marginBottom:14}}>
            <div className="scroll-tabs" style={{marginBottom:14, flexWrap:"wrap"}}>
                {[[["races","Races"],["companions","Companions"],["classes","Classes"],["rules","Rules & FAQ"],["weapons","Weapons"],["armor","Armor"],["enchancements","Enchantments"],["merchants","Merchants"],["loot","Loot Tables"]].map(([k,l])=>(
                    <div key={k} className={`scroll-tab${compendiumTab===k?" active":""}` } onClick={()=>setCompendiumTab(k)}>{l}</div>
                ))}
            </div>

            <div key={k} className={`scroll-tab${compendiumTab===k?" active":""}` } onClick={()=>setCompendiumTab(k)}>{l}</div>
        ))
    </div>

    ...

    {compendiumTab==="races" && (
        <div className="fade-in">
            <div className="scroll-tabs" style={{marginBottom:14}}>

```

```

      {Object.keys(RACES).map(r=><div key={r} className={`scroll-tab${raceTab===r?" active":""}` } onClick={()=>setRaceTab(r)}>{RACES[r].icon} {r}
</div>)}
    </div>
    {() => {
      const race=RACES[raceTab];
      return <div className="dark-card glow fade-in">
        <div style={{fontFamily:"'Cinzel Decorative',serif",fontSize:"1rem",color:"var(--ember-glow)",marginBottom:3}}>{race.icon} {raceTab}</div>
        <div style={{fontFamily:"'IM Fell English',serif",fontStyle:"italic",fontSize:"0.72rem",color:"var(--parch-shadow)",marginBottom:12,lineHeight:1.6}}>
{race.lore}</div>
        <div className="rune-divider"><span>Stat Modifiers</span></div>
        <div style={{display:"grid",gridTemplateColumns:"repeat(7,1fr)",gap:4;marginBottom:12}}>
          {Object.entries(race.stats).map(([s,v])=><div key={s} style={{textAlign:"center",background:"rgba(0,0,0,0.3)",borderRadius:3,padding:"5px
2px",border:`1px solid ${v!=="0"?STAT_COLORS[s]+"40":"rgba(255,255,255,0.04)"} `}}>
            <div style={{fontFamily:"'Cinzel',serif",fontSize:"0.4rem",color:STAT_COLORS[s]}}>{s}</div>
            <div style={{fontFamily:"'Cinzel',serif",fontSize:"0.75rem",fontWeight:"700",color:v>0?"#4caf50":v<0?"#ef5350":"var(--parch-shadow)"}>
{v>0?"+"+""}{v}</div>
          </div>)}
        </div>
        <div className="rune-divider"><span>Racial Ability</span></div>
        <div className="info-bar gold">{race.ability}</div>
        <div className="rune-divider"><span>Biological Quirk</span></div>
        <div style={{fontFamily:"'IM Fell English',serif",fontSize:"0.7rem",color:"var(--parch)",lineHeight:1.6}}>{race.biological_quirk}</div>
      </div>;
    })()
  </div>
})

```

```

{compendiumTab==="companions" && (
  <div className="fade-in">
    <div className="scroll-tabs" style={{marginBottom:14}}>
      {Object.keys(COMPANION_TYPES).map(c=><div key={c} className={`scroll-tab${compTab===c?" active":""}` } onClick={()=>setCompTab(c)}>
{COMPANION_TYPES[c].icon} {c}</div>)}
    </div>
    {() => {
      const c=COMPANION_TYPES[compTab];
      return <div className="dark-card glow fade-in">
        <div style={{fontFamily:"'Cinzel Decorative',serif",fontSize:"1rem",color:"var(--ember-glow)",marginBottom:3}}>{c.icon} {compTab}</div>
        <div style={{display:"grid",gridTemplateColumns:"1fr 1fr",gap:8;marginBottom:10}}>
          <div style={{background:"rgba(0,0,0,0.3)",border:"1px solid rgba(77,182,172,0.3)",borderRadius:3,padding:"8px 10px"}}>
            <div style={{fontFamily:"'Cinzel',serif",fontSize:"0.5rem",color:"#4fc3f7",letterSpacing:"1px",marginBottom:3}}>ACTIVE ABILITY</div>
            <div style={{fontFamily:"'IM Fell English',serif",fontSize:"0.68rem",color:"var(--parch)"}>{c.active_ability}</div>
          </div>
          <div style={{background:"rgba(0,0,0,0.3)",border:"1px solid rgba(102,187,106,0.3)",borderRadius:3,padding:"8px 10px"}}>
            <div style={{fontFamily:"'Cinzel',serif",fontSize:"0.5rem",color:"#66bb6a",letterSpacing:"1px",marginBottom:3}}>PASSIVE TRAIT</div>
            <div style={{fontFamily:"'IM Fell English',serif",fontSize:"0.68rem",color:"var(--parch)"}>{c.passive_trait}</div>
          </div>
        </div>
      </div>
    })
  )

```

```

</div>

<div style={{fontFamily:"'Cinzel',serif",fontSize:"0.5rem",color:"var(--ember)",marginBottom:3,letterSpacing:"1px"}}>MAINTENANCE</div>

<div style={{fontFamily:"'IM Fell English',serif",fontSize:"0.68rem",color:"var(--parch)",marginBottom:8}}>{c.maintenance}</div>

<div style={{fontFamily:"'Cinzel',serif",fontSize:"0.5rem",color:"var(--void-glow)",marginBottom:3,letterSpacing:"1px"}}>BEHAVIOUR QUIRK</div>

<div style={{fontFamily:"'IM Fell English',serif",fontStyle:"italic",fontSize:"0.68rem",color:"var(--parch-shadow)"}>{c.behavior_quirk}</div>

<div className="rune-divider"><span>Assign to Player</span></div>

    {players.filter(p=>!p.isDead).map(p=><button key={p.id} className="rune-btn sm" style={{marginRight:6;marginBottom:6}} onClick=
    {()=>assignCompanion(p.id,compTab)}>{CLASS_ICON_EMO[p.cls]} {p.charname}</button>}}

    </div>;
  })()
</div>
})

{compendiumTab=== "classes" && (
  <div className="fade-in">

    <div className="scroll-tabs" style={{marginBottom:14}}>

      {CLASSES.map(c=><div key={c} className={` scroll-tab${classTab===c?" active":""}` } onClick={()=>{setClassTab(c);setClassSec("overview")}}>
      {CLASS_ICON_EMO[c]} {c}</div>)}

    </div>

    {() => {
      const lore=CLASS_LORE[classTab];
      const stats=CLASS_BASE_STATS[classTab];
      const primary=PRIMARY_STAT[classTab];
      return <div className="dark-card glow fade-in">

        <div style={{display:"flex",justifyContent:"space-between",alignItems:"flex-start",marginBottom:8}}>

          <div>

            <div style={{fontFamily:"'Cinzel Decorative',serif",fontSize:"0.9rem",color:"var(--ember-glow)"}>{CLASS_ICON_EMO[classTab]}
            {classTab.toUpperCase()}</div>

            <div style={{fontFamily:"'IM Fell English',serif",fontStyle:"italic",fontSize:"0.65rem",color:"var(--parch-shadow)",marginTop:3}}>{lore.tagline}"
            </div>

          </div>

          <div style={{textAlign:"right"}}>

            <div style={{fontFamily:"'Cinzel',serif",fontSize:"0.48rem",color:"var(--parch-shadow)",letterSpacing:"1px"}}>BASE HP</div>

            <div style={{fontFamily:"'Cinzel Decorative',serif",fontSize:"1.2rem",fontWeight:"900",color:"#4caf50"}}>{CLASS_HP[classTab]}</div>

          </div>

        </div>

        <div className="scroll-tabs" style={{marginBottom:10}}>

          {[["overview","Overview"],["mechanics","Stats"],["gift","Gift Power"],["lore","Lore"]].map(([k,l])=><div key={k} className={` scroll-
          tab${classSec===k?" active":""}` } onClick={()=>setClassSec(k)}>{l}</div>)}

        </div>

        {classSec=== "overview" && <div className="fade-in"><div style={{fontFamily:"'IM Fell English',serif",fontSize:"0.72rem",color:"var(--
        parch)",lineHeight:1.7;marginBottom:10}}>{lore.overview}</div><div style={{fontFamily:"'IM Fell
        English',serif",fontStyle:"italic",fontSize:"0.68rem",color:"var(--void-glow)"}>Best for: "{lore.playstyle}"</div></div>

        {classSec=== "mechanics" && <div className="fade-in">

          <div style={{display:"grid",gridTemplateColumns:"repeat(7,1fr)",gap:4;marginBottom:14}}>

            {Object.entries(stats).map(([s,v])=><div key={s} style={{textAlign:"center",background:"rgba(0,0,0,0.3)",borderRadius:3,padding:"6px
            2px",border:`1px solid ${s===primary?STAT_COLORS[s]+"60":"rgba(255,255,255,0.04)"} `}}>

```

```

<div style={{fontFamily:"Cinzel",serif,fontSize:"0.42rem",color:STAT_COLORS[s]}>{s===primary?"★":""}</div>
<div style={{fontFamily:"Cinzel",serif,fontSize:"0.8rem",fontWeight:"700",color:STAT_COLORS[s]}>{v}</div>
</div>}}
</div>
{(lore.strengths||[]).map((s,i)=><div key={i} style={{fontFamily:"IM Fell English",serif,fontSize:"0.68rem",color:"#a5d6a7",marginBottom:3}}>✓ {s}
</div>)}

<div style={{height:8}}>
{(lore.weaknesses||[]).map((w,i)=><div key={i} style={{fontFamily:"IM Fell English",serif,fontSize:"0.68rem",color:"#ef9a9a",marginBottom:3}}>×
{w}</div>)}
</div>
{classSec=== "gift" && <div className="fade-in" style={{background:"rgba(102,187,106,0.08)",border:"1px solid
rgba(102,187,106,0.3)",borderRadius:3,padding:16,textAlign:"center"}}>
<div style={{fontFamily:"Cinzel",serif,fontSize:"0.52rem",color:"#66bb6a",letterSpacing:"3px",marginBottom:6}}>UNIQUE GIFT POWER</div>
<div style={{fontFamily:"Cinzel Decorative",serif,fontSize:"0.95rem",color:"#fff",marginBottom:8}}>{lore.giftPower.split("—")[0]}</div>
{lore.giftPower.split("—")[1]} && <div style={{fontFamily:"IM Fell English",serif,fontSize:"0.7rem",color:"#ccc",lineHeight:1.5}}>
{lore.giftPower.split("—")[1]}</div>
</div>
{classSec=== "lore" && <div className="fade-in" style={{fontFamily:"IM Fell English",serif,fontStyle:"italic",fontSize:"0.75rem",color:"var(--parch-
shadow)",lineHeight:1.8}}>{lore.lore}</div>
</div>;
})();
</div>
)}

```

```

{compendiumTab=== "rules" && (
<div className="fade-in">
<div className="scroll-tabs" style={{marginBottom:14}}>
{RULES_SECTIONS.map(s=><div key={s.id} className={`scroll-tab${rulesTab===s.id?" active":""}` } onClick={()=>setRulesTab(s.id)}>{s.title.split("
").slice(0,2).join(" ")</div>)}
<div className={`scroll-tab${rulesTab=== "faq"?" active":""}` } onClick={()=>setRulesTab("faq")}>FAQ</div>
</div>
{rulesTab=== "faq"
? <div className="dark-card fade-in">
<div className="section-title">Frequently Asked Questions</div>
{FAQ.map((f,i)=>(
<div key={i} className={`accordion-item${faqOpen===i?" open":""}` } style={{marginBottom:6}}>
<div className="accordion-header" onClick={()=>setFaqOpen(faqOpen===i?null:i)}>
<span style={{fontFamily:"Cinzel",serif,fontSize:"0.6rem",color:"var(--void-glow)",letterSpacing:"0.5px"}}>Q: {f.q}</span>
<span className="accordion-chevron">▼</span>
</div>
{faqOpen===i && <div className="accordion-body fade-in"><div style={{fontFamily:"IM Fell English",serif,fontSize:"0.72rem",color:"var(--
parch)",lineHeight:1.6}}>{f.a}</div></div>
</div>
)}}
</div>
: RULES_SECTIONS.filter(s=>s.id===rulesTab).map(s=>(
<div key={s.id} className="dark-card fade-in">

```

```

        <div className="section-title">{s.title}</div>

        <div style={{fontFamily:"'IM Fell English',serif",fontSize:"0.72rem",color:"var(--parch)",whiteSpace:"pre-wrap",lineHeight:1.7}}>{s.content}</div>
    </div>
    ))
}
</div>
))
</div>
...

);

/* —— ADD PLAYER MODAL —— */
const AddPlayerModal = () => {
    if (!addPlayerOpen) return null;
    const c = creation;

    const previewStats = c.cls && c.race ? getPlayerStats({cls:c.cls,race:c.race,traitLevel:c.traitRoll?
    TRAIT_TIERS.find(t=>c.traitRoll>=t.range[0]&&c.traitRoll<=t.range[1]?t.level:"√ Standard")} : null;

    return (
        <div className="modal-overlay open" onClick={e=>{if (e.target===e.currentTarget)setAddPlayerOpen(false);}}>
        <div className="modal-box">
            <div style={{display:"flex",justifyContent:"space-between",alignItems:"center",marginBottom:20}}>
            <div style={{fontFamily:"'Cinzel Decorative',serif",fontSize:"0.95rem",color:"var(--ember-glow)",letterSpacing:"2px"}}>✕ Forge a Character</div>
            <button className="rune-btn danger sm" onClick={()=>setAddPlayerOpen(false)}>✕ Close</button>
        </div>
        <div style={{display:"grid",gridTemplateColumns:"1fr 1fr",gap:10}}>
            <div><label className="form-label">TikTok Username</label><input className="scroll-input" value={c.username} onChange=
            {e=>setC("username",e.target.value)} placeholder="@handle" /></div>
            <div><label className="form-label">Character Name *</label><input className="scroll-input" value={c.charname} onChange=
            {e=>setC("charname",e.target.value)} placeholder="Name" /></div>
            <div><label className="form-label">Class *</label><select className="scroll-select" value={c.cls} onChange={e=>setC("cls",e.target.value)}><option
            value="">— Class —</option><CLASSES.map(cl=><option key={cl} value={cl}><CLASS_ICON_EMO[cl]} {cl}</option>)</select></div>
            <div><label className="form-label">Race *</label><select className="scroll-select" value={c.race} onChange={e=>setC("race",e.target.value)}>
            <option value="">— Race —</option><Object.keys(RACES).map(r=><option key={r} value={r}><RACES[r].icon> {r}</option>)</select></div>
            <div><label className="form-label">Origin</label><select className="scroll-select" value={c.origin} onChange={e=>setC("origin",e.target.value)}>
            <option value="">— Origin —</option><ORIGINS.map(o=><option key={o} value={o}><o></option>)</select></div>
            <div><label className="form-label">Payment</label><select className="scroll-select" value={c.payment} onChange=
            {e=>setC("payment",e.target.value)}><option value="tiktok">TikTok (1,000 Coins)</option><option value="paypal">PayPal ($13.00 CAD)</option>
            </select></div>
        </div>
        {previewStats && (
            <div style={{marginTop:12,background:"rgba(0,0,0,0.3)",border:"1px solid var(--border-rune)",borderRadius:3,padding:"10px 12px"}}>
            <div style={{fontFamily:"'Cinzel',serif",fontSize:"0.5rem",color:"var(--ember)",letterSpacing:"2px",marginBottom:6}}>PROJECTED STATS</div>
            <div className="stat-grid">
                {Object.entries(previewStats).map(([s,v])=><div key={s} className="stat-cell"><div className="stat-label" style={{color:STAT_COLORS[s]}}>{s}</div>
                <div className="stat-val" style={{color:STAT_COLORS[s]}}>{v}</div></div>)}
            </div>
        )}
    </div>

```

```

    })
    <div className="rune-divider"><span>Optional Rolls</span></div>
    <div style={{display:"grid",gridTemplateColumns:"1fr 1fr",gap:8}}>
    {[[{"repRoll":"","Reputation":"","d20"},{"traitRoll":"","Trait Tier":"","d20"},{"skillRoll":"","Signature Skill":"","d20"},{"shadowRoll":"","Shadow Trait":"","d20"},{"enchantRoll",
    "Enchantment":"","d20"},{"destinyRoll":"","Destiny Thread":"","d20"}]].map(([field,label])=>{
    <div key={field} style={{display:"flex",alignItems:"center",gap:6}}>
    <div style={{flex:1}}>
    <div style={{fontFamily:"Cinzel',serif",fontSize:"0.48rem",color:"var(--ember)",letterSpacing:"1px",marginBottom:3}}>{label}</div>
    <div style={{fontFamily:"Cinzel',serif",fontSize:"0.65rem",color:c[field]?"var(--ember-glow)":"var(--parch-shadow)"},">{c[field]}||"—</div>
    </div>
    <button className="rune-btn sm" onClick={()=>rollC(field)}>🎲</button>
    </div>
    )))
  </div>
  <div style={{marginTop:12}}><label className="form-label">Notes</label><textarea className="scroll-textarea" value={c.notes} onChange=
  {e=>setC("notes",e.target.value)} placeholder="Additional notes..."></div>
  <button className="rune-btn full" style={{marginTop:16,fontSize:"0.75rem",letterSpacing:"2px"}} onClick={submitPlayer}>🏹 SPAWN
  CHARACTER</button>
</div>
</div>
);
const PlayerLoreTab = ({ player, setPlayers }) => {
  if (!player) return null;

  const pClass = player.class;
  const pRace = player.race;

  // Helper to update the player object in your state
  const handleChange = (field, val) => {
    setPlayers(prev => prev.map(p => p.id === player.id ? { ...p, [field]: val } : p));
  };

  return (
    <div style={{ marginTop: "16px", padding: "12px", background: "var(--card)", borderRadius: "6px", border: "1px solid var(--border)" }}>
      <h3 style={{ color: "var(--gold)", borderBottom: "1px solid var(--dimmer)", paddingBottom: "4px", marginBottom: "12px", fontFamily: "'Cinzel', serif"
      }}>
        Narrative & Progression
      </h3>

      { /* RACE LORE */
      {pRace && RACES[pRace] && (
        <div style={{ marginBottom: "16px" }}>
          <h4 style={{ color: "var(--blue)", marginBottom: "4px", margin: "0" }}>{RACES[pRace].icon} {pRace} Biology</h4>
          <p style={{ fontSize: "0.8rem", color: "var(--text)", margin: "4px 0" }}>
            <strong style={{ color: "var(--dim)" }}>Quirk:</strong> {RACES[pRace].biological_quirk}
          </p>

```

```

<p style={{ fontSize: "0.8rem", color: "var(--text)", margin: "4px 0" }}>
  <strong style={{ color: "var(--dim)" }}>Social:</strong> {RACES[pRace].social_standing}
</p>
</div>
)}

{/* COMPANION SELECT */}
<div style={{ marginBottom: "16px" }}>
  <label style={{ fontSize: "0.75rem", color: "var(--dim)" }}>Companion</label>
  <select className="scroll-input" value={player.companion || ""} onChange={e => handleChange("companion", e.target.value)}>
    <option value="">-- No Companion --</option>
    {Object.keys(COMPANION_TYPES).map(c => (
      <option key={c} value={c}>{COMPANION_TYPES[c].icon} {COMPANION_TYPES[c].name}</option>
    ))}
  </select>
  {player.companion && COMPANION_TYPES[player.companion] && (
    <div style={{ fontSize: "0.75rem", color: "var(--text)", marginTop: "4px" }}>
      <strong style={{color:"var(--gold)"}}>HP:</strong> {COMPANION_TYPES[player.companion].hp} | <strong style={{color:"var(--gold)"}}>Ability:
</strong> {COMPANION_TYPES[player.companion].active_ability}
      <br/><strong style={{color:"var(--dim)"}}>Quirk:</strong> {COMPANION_TYPES[player.companion].behavior_quirk}
    </div>
  )}
</div>

{/* CLASS PROGRESSION */}
{pClass && REP[pClass] && (
  <div style={{ display: "flex", flexDirection: "column", gap: "12px" }}>
    <h4 style={{ color: "var(--blue)", marginBottom: "0px", marginTop: "8px" }}>{pClass} Paths</h4>

    {/* Reputation */}
    <div>
      <label style={{ fontSize: "0.75rem", color: "var(--dim)" }}>Reputation / Title</label>
      <select className="scroll-input" value={player.repTitle || ""} onChange={e => handleChange("repTitle", e.target.value)}>
        <option value="">-- Select Title --</option>
        {REP[pClass].map(r => <option key={r.title} value={r.title}>{r.title}</option>)}
      </select>
      {player.repTitle && (
        <div style={{ fontSize: "0.75rem", color: "var(--green)", marginTop: "4px" }}>
          {REP[pClass].find(r => r.title === player.repTitle)?.passive}
        </div>
      )}
    </div>

    {/* Signature Skill */}
    <div>

```

```

<label style={{ fontSize: "0.75rem", color: "var(--dim)" }}>Signature Skill</label>
<select className="scroll-input" value={player.sigSkill || ""} onChange={e => handleChange("sigSkill", e.target.value)}>
  <option value="">-- Select Skill --</option>
  {SIGNATURE_SKILLS[pClass].map(s => <option key={s.name} value={s.name}>{s.name}</option>)}
</select>
{player.sigSkill && (
  <div style={{ fontSize: "0.75rem", color: "var(--green)", marginTop: "4px" }}>
    {SIGNATURE_SKILLS[pClass].find(s => s.name === player.sigSkill)?.desc}
  </div>
)}
</div>

{/* Shadow Trait */}
<div>
  <label style={{ fontSize: "0.75rem", color: "var(--dim)" }}>Shadow Trait</label>
  <select className="scroll-input" value={player.shadowTrait || ""} onChange={e => handleChange("shadowTrait", e.target.value)}>
    <option value="">-- Select Shadow Trait --</option>
    {SHADOW_TRAITS[pClass].map(s => <option key={s.name} value={s.name}>{s.name}</option>)}
  </select>
  {player.shadowTrait && (
    <div style={{ fontSize: "0.75rem", color: "var(--red)", marginTop: "4px" }}>
      {SHADOW_TRAITS[pClass].find(s => s.name === player.shadowTrait)?.desc}
    </div>
  )}
</div>

{/* Destiny Thread */}
<div>
  <label style={{ fontSize: "0.75rem", color: "var(--dim)" }}>Destiny Thread</label>
  <select className="scroll-input" value={player.destiny || ""} onChange={e => handleChange("destiny", e.target.value)}>
    <option value="">-- Select Destiny Thread --</option>
    {DESTINY_THREADS[pClass].map((d, i) => (
      <option key={i} value={d.thread}>{d.thread.substring(0, 40)}...</option>
    ))}
  </select>
  {player.destiny && (
    <div style={{ fontSize: "0.8rem", color: "var(--purple-light)", marginTop: "6px", fontStyle: "italic", borderLeft: "2px solid var(--purple)",
paddingLeft: "8px" }}>
      "{player.destiny}"
    </div>
  )}
</div>

</div>
)}

```

```

{compendiumTab=== "weapons" && (
  <div className="fade-in">
    {Object.keys(ARSENAL_WEAPONS).map(cls => (
      <div key={cls} style={{ marginBottom: "24px" }}>
        <h3 style={{ color: "var(--blue)", borderBottom: "1px solid rgba(79, 195, 247, 0.2)", paddingBottom: "4px", fontFamily: "'Cinzel', serif" }}>{cls} Arsenal</h3>
        <div style={{ display: "grid", gap: "8px" }}>
          {ARSENAL_WEAPONS[cls].map((w, i) => (
            <div key={i} className="dark-card" style={{ padding: "10px", borderLeft: `3px solid ${w.color} || "var(--dim)"` }}>
              <div style={{ display: "flex", justifyContent: "space-between", alignItems: "flex-start" }}>
                <strong style={{ color: "var(--text)", fontSize: "0.9rem", fontFamily: "'Cinzel', serif" }}>{w.name}</strong>
                <span className="rarity-badge" style={{ color: w.color, border: `1px solid ${w.color}` }}>{w.rarity}</span>
              </div>
              <div style={{ fontSize: "0.75rem", margin: "4px 0", fontFamily: "'IM Fell English', serif" }}>
                <span style={{ color: "var(--red)" }}>✂ {w.damage}</span> | <span style={{ color: "var(--green)" }}>🔥 {w.effect}</span>
              </div>
              <div style={{ fontSize: "0.7rem", fontStyle: "italic", color: "var(--dim)", fontFamily: "'IM Fell English', serif" }}>
                "{w.lore}" — Found: {w.location}
              </div>
            </div>
          )))
        </div>
      </div>
    ))
  </div>
)}

```

```

{compendiumTab=== "armor" && (
  <div className="fade-in">
    {Object.keys(ARSENAL_ARMOR).map(cls => (
      <div key={cls} style={{ marginBottom: "24px" }}>
        <h3 style={{ color: "var(--blue)", borderBottom: "1px solid rgba(79, 195, 247, 0.2)", paddingBottom: "4px", fontFamily: "'Cinzel', serif" }}>{cls} Armor</h3>
        <div style={{ display: "grid", gap: "8px" }}>
          {ARSENAL_ARMOR[cls].map((a, i) => (
            <div key={i} className="dark-card" style={{ padding: "10px", borderLeft: `3px solid ${a.color} || "var(--dim)"` }}>
              <div style={{ display: "flex", justifyContent: "space-between", alignItems: "flex-start" }}>
                <strong style={{ color: "var(--text)", fontSize: "0.9rem", fontFamily: "'Cinzel', serif" }}>{a.name}</strong>
                <span className="rarity-badge" style={{ color: a.color, border: `1px solid ${a.color}` }}>{a.rarity}</span>
              </div>
              <div style={{ fontSize: "0.75rem", margin: "4px 0", fontFamily: "'IM Fell English', serif" }}>
                <span style={{ color: "var(--blue)" }}>🛡 {a.defense}</span> | <span style={{ color: "var(--green)" }}>🔥 {a.effect}</span>
              </div>
              <div style={{ fontSize: "0.7rem", fontStyle: "italic", color: "var(--dim)", fontFamily: "'IM Fell English', serif" }}>

```

```

        "{a.lore}" — Found: {a.location}
      </div>
    </div>
  )}
</div>
</div>
)}
</div>
)}

```

```

{compendiumTab=== "enchantments" && (
  <div className="fade-in">
    <div style={{ display: "grid", gap: "8px" }}>
      {ARSENAL_ENCHANTMENTS.map((e, i) => (
        <div key={i} className="dark-card" style={{ padding: "10px", borderLeft: `3px solid ${e.color} || "var(--purple)"` }}>
          <div style={{ display: "flex", justifyContent: "space-between" }}>
            <strong style={{ color: "var(--text)", fontFamily: "'Cinzel', serif" }}>{e.name}</strong>
            <span className="rarity-badge" style={{ color: e.color, border: `1px solid ${e.color}` }}>{e.rarity}</span>
          </div>
          <div style={{ fontSize: "0.75rem", color: "var(--green)", margin: "4px 0", fontFamily: "'IM Fell English', serif" }}>{e.effect}</div>
          <div style={{ fontSize: "0.7rem", fontStyle: "italic", color: "var(--dim)", fontFamily: "'IM Fell English', serif" }}>{e.lore}</div>
        </div>
      )})
    </div>
  )}
</div>
</div>
)}

```

```

{compendiumTab=== "merchants" && (
  <div className="fade-in">
    {/* --- BUYER SELECTION --- */}
    <div className="dark-card" style={{ marginBottom: "16px", borderColor: "var(--gold)" }}>
      <label className="form-label">Select Traveler for Trade</label>
      <select
        className="scroll-select"
        value={merchantBuyer}
        onChange={e => setMerchantBuyer(e.target.value)}
        aria-label="Select buyer"
      >
        <option value="">-- Select a Traveler --</option>
        {players.filter(p => !p.isDead).map(p => (
          <option key={p.id} value={p.id}>{CLASS_ICON_EMO[p.cls]} {p.charname}</option>
        )})
      </select>
      {!merchantBuyer && (
        <div style={{ fontSize: '0.6rem', color: 'var(--parch-shadow)', marginTop: '4px' }}>

```

Select an active traveler to enable direct purchasing from NPCs.

</div>

}}

</div>

{/* --- MERCHANT LISTING --- */}

<div style={{ display: "grid", gap: "12px" }}>

{Object.keys(WORLD_NPCs).map(npckey => {

const npc = WORLD_NPCs[npckey];

return (

<div key={npckey} className="dark-card glow" style={{ padding: "12px", border: `1px solid \${npc.color}40` }}>

<h3 style={{ margin: "0 0 4px 0", color: npc.color, fontFamily: "'Cinzel', serif" }}>

{npc.icon} {npc.name}

</h3>

<div style={{ fontSize: "0.7rem", color: "var(--dim)", textTransform: "uppercase", letterSpacing: "1px", marginBottom: "8px", fontFamily: "'Cinzel', serif" }}>

{npc.title} | {npc.region}

</div>

<strong style={{ fontSize: "0.75rem", color: "var(--gold)", fontFamily: "'Cinzel', serif" }}>Inventory / Services:

<div style={{ marginTop: "8px", display: "flex", flexDirection: "column", gap: "6px" }}>

{npc.inventory.map((inv, idx) => (

<div key={idx} style={{

display: 'flex', alignItems: 'center', justifyContent: 'space-between',

background: 'rgba(0,0,0,0.3)', padding: '6px 8px', borderRadius: '4px',

borderLeft: `2px solid \${npc.color}`

}}>

<div>

<div style={{ fontSize: '0.75rem', color: "var(--parch)", fontFamily: "'IM Fell English', serif" }}>{inv.item}</div>

<div style={{ fontSize: '0.65rem', color: "var(--dim)", fontFamily: "'Cinzel', serif" }}>

Cost: {inv.cost} | Chance: {inv.chance}

</div>

</div>

<button

type="button"

className="rune-btn sm"

disabled={!merchantBuyer}

style={{ padding: '4px 8px', fontSize: '0.55rem', opacity: !merchantBuyer ? 0.4 : 1 }}

onClick={() => {

const p = players.find(x => x.id === merchantBuyer);

if(!p) return;

if(!window.confirm(`Buy \${inv.item.split(" ")[0].trim()} for \${p.charname}? \nCost: \${inv.cost}`)) return;

```

// Auto-parse rarity from the name/cost text
let itemRarity = "Common";

if (inv.item.includes("Mythic") || inv.item.includes("Artifact")) itemRarity = "Mythic";
else if (inv.item.includes("Legendary") || inv.cost.includes("10,000c")) itemRarity = "Legendary";
else if (inv.item.includes("Rare") || inv.cost.includes("4,000c")) itemRarity = "Rare";
else if (inv.item.includes("Uncommon") || inv.cost.includes("1,") || inv.cost.includes("800c")) itemRarity = "Uncommon";

// Auto-parse effect if there are parentheses (e.g., "Pure Void-Essence (Full Heal, applies minor curse)")
const extractedEffect = inv.item.includes("(")
  ? inv.item.match(/\(((^)+)\)/)[1]
  : `Purchased from ${npc.name}`;

const newItem = {
  name: inv.item.split("(")[0].trim(),
  type: inv.item.toLowerCase().includes("weapon") || inv.item.toLowerCase().includes("fang") ? "weapon" : "consumable",
  rarity: itemRarity,
  condition: "Pristine",
  effect: extractedEffect
};

// Push to player's inventory
updatePlayer(p.id, { inventory: [...(p.inventory || []), newItem] });

// Write to the Session Log automatically
addLog({ type: "item", who: p.charname, what: `Bought ${newItem.name} from ${npc.name} for ${inv.cost}` });

// Reset the dropdown so you don't accidentally buy to the wrong person next time
setMerchantBuyer("");
}}
>
Buy
</button>
</div>
)}}
</div>
</div>
);
}}
</div>
</div>
))

{compendiumTab==="loot" && (

```

```

<div className="fade-in">
  {/* --- LOOTER SELECTION --- */}
  <div className="dark-card" style={{ marginBottom: "16px", borderColor: "var(--green)" }}>
    <label className="form-label">Select Traveler for Looting</label>
    <select
      className="scroll-select"
      value={lootFinder}
      onChange={e => setLootFinder(e.target.value)}
      aria-label="Select looter"
    >
      <option value="">--- Select a Traveler --</option>
      {players.filter(p => !p.isDead).map(p => (
        <option key={p.id} value={p.id}>{CLASS_ICON_EMO[p.cls]} {p.charname}</option>
      ))}
    </select>
    {!!lootFinder && (
      <div style={{ fontSize: '0.6rem', color: 'var(--parch-shadow)', marginTop: '4px' }}>
        Select an active traveler to distribute loot or trigger hazards.
      </div>
    )}
  </div>

  <div style={{ display: "grid", gap: "16px" }}>
    {/* --- SPECIAL CHESTS --- */}
    <div>
      <h3 style={{ color: "var(--gold)", marginBottom: "8px", fontFamily: '"Cinzel', serif", borderBottom: "1px solid var(--border-rune)", display: 'flex',
        alignItems: 'center', gap: '6px' }}>
        <span role="img" aria-label="chest"><img alt="chest icon" data-bbox="325 578 342 591"/></span> Special Chests
      </h3>
      {ARSENAL_CHESTS.map((c, i) => (
        <div key={i} className="dark-card glow" style={{ padding: "10px", border: `1px solid ${c.color}50`, marginBottom: "8px" }}>
          <strong style={{ color: c.color, fontFamily: '"Cinzel', serif" }}>{c.name}</strong>
          <span style={{fontSize: "0.6rem", color:"var(--parch-shadow)", marginLeft: '6px'}}>{c.type}</span>

          <div style={{ fontSize: "0.75rem", color: "var(--dim)", margin: "4px 0", fontFamily: '"IM Fell English', serif" }}>
            <strong style={{color:"var(--ember)", fontFamily:"'Cinzel', serif"}}>Lock:</strong> {c.lockDifficulty}
          </div>

          <div style={{ marginTop: "8px", display: "flex", flexDirection: "column", gap: "4px" }}>
            {c.loot.map((l, idx) => {
              const isHazard = l.item.toLowerCase().includes("trap") || l.item.toLowerCase().includes("curse") || l.item.toLowerCase().includes("hazard");
              const isGold = l.item.toLowerCase().includes("coin") || l.item.toLowerCase().includes("gold");

              return (
                <div key={idx} style={{ display: 'flex', alignItems: 'center', justifyContent: 'space-between', background: 'rgba(0,0,0,0.3)', padding: '6px 8px',

```

```

borderRadius: '4px', borderLeft: `2px solid ${isHazard ? 'var(--blood)' : (isGold ? 'var(--gold-bright)' : 'var(--blue)')}` } }>
<div>
  <div style={{ fontSize: '0.75rem', color: isHazard ? '#ef9a9a' : 'var(--parch)', fontFamily: '"IM Fell English', serif" }}>{l.item}</div>
  <div style={{ fontSize: '0.65rem', color: "var(--dim)", fontFamily: '"Cinzel', serif" }}>Chance: {l.chance}</div>
</div>
<button
  type="button"
  className={ `rune-btn sm ${isHazard ? 'danger' : (isGold ? '' : 'success')}` }
  disabled={!lootFinder}
  style={{ padding: '4px 8px', fontSize: '0.55rem', opacity: !lootFinder ? 0.4 : 1, borderColor: isGold ? 'var(--gold)' : '' }}
  onClick={() => {
    const p = players.find(x => x.id === lootFinder);
    if(!p) return;

    if(isHazard) {
      if(!window.confirm(`Trigger hazard: ${l.item} on ${p.charname}?`)) return;
      addLog({ type: "consequence", who: p.charname, what: `Triggered Hazard: ${l.item} from ${c.name}` });
      alert(`Hazard triggered! Don't forget to manually adjust ${p.charname}'s HP or apply the Curse tag.`);
    } else if (isGold) {
      if(!window.confirm(`Loot ${l.item} for ${p.charname}?`)) return;
      addLog({ type: "item", who: p.charname, what: `Looted ${l.item} from ${c.name}` });
      alert(`Coins looted! Remember to add this to their economy balance.`);
    } else {
      if(!window.confirm(`Grant ${l.item} to ${p.charname}?`)) return;

      let itemRarity = "Common";
      if (l.item.includes("Mythic") || l.item.includes("Artifact")) itemRarity = "Mythic";
      else if (l.item.includes("Legendary")) itemRarity = "Legendary";
      else if (l.item.includes("Rare")) itemRarity = "Rare";
      else if (l.item.includes("Uncommon")) itemRarity = "Uncommon";

      const newItem = {
        name: l.item,
        type: l.item.toLowerCase().includes("weapon") || l.item.toLowerCase().includes("blade") ? "weapon" : "consumable",
        rarity: itemRarity,
        condition: "Pristine",
        effect: `Looted from ${c.name}`
      };

      updatePlayer(p.id, { inventory: [...(p.inventory || []), newItem] });
      addLog({ type: "item", who: p.charname, what: `Looted ${newItem.name} from ${c.name}` });
    }
    setLootFinder("");
  }}
>

```

```

        {isHazard ? 'Trigger' : (isGold ? 'Loot Coins' : 'Grant Item')}}
      </button>
    </div>
  );
  }}}
</div>
</div>
  )))
</div>

/* --- ENVIRONMENTAL BINS --- */
<div>
  <h3 style={{ color: "var(--gold)", marginBottom: "8px", fontFamily: "'Cinzel', serif", borderBottom: "1px solid var(--border-rune)", display: 'flex',
alignItems: 'center', gap: '6px' }}>
    <span role="img" aria-label="ruins">  </span> Environmental Loot Bins
  </h3>
  {ARSENAL_LOOT_BINS.map((b, i) => (
    <div key={i} className="dark-card glow" style={{ padding: "10px", borderLeft: `3px solid ${b.color}`, marginBottom: "8px" }}>
      <strong style={{color: "var(--text)", fontFamily: "'Cinzel', serif" }}>{b.name}</strong>
      <span style={{ fontSize: "0.7rem", color: "var(--dim)", marginLeft: '6px' }}>{b.location}</span>

      <div style={{ fontSize: "0.75rem", color: "var(--green)", margin: "4px 0", fontFamily: "'IM Fell English', serif" }}>
        <strong style={{color:"var(--ember)", fontFamily:"'Cinzel', serif"}}>Search:</strong> {b.roll}
      </div>

      <div style={{ marginTop: "8px", display: "flex", flexDirection: "column", gap: "4px" }}>
        {b.loot.map((l, idx) => {
          const isHazard = l.item.toLowerCase().includes("hazard") || l.item.toLowerCase().includes("hostile") || l.item.toLowerCase().includes("curse") ||
l.item.toLowerCase().includes("nothing");
          const isGold = l.item.toLowerCase().includes("coin") || l.item.toLowerCase().includes("gold");

          return (
            <div key={idx} style={{ display: 'flex', alignItems: 'center', justifyContent: 'space-between', background: 'rgba(0,0,0,0.3)', padding: '6px 8px',
borderRadius: '4px', borderLeft: `2px solid ${isHazard ? 'var(--blood)' : (isGold ? 'var(--gold-bright)' : 'var(--blue)')}` }}>
              <div>
                <div style={{ fontSize: '0.75rem', color: isHazard ? '#ef9a9a' : 'var(--parch)', fontFamily: "'IM Fell English', serif" }}>{l.item}</div>
                <div style={{ fontSize: '0.65rem', color: "var(--dim)", fontFamily: "'Cinzel', serif" }}>Chance: {l.chance}</div>
              </div>
              <button
                type="button"
                className={`rune-btn sm ${isHazard ? 'danger' : (isGold ? '' : 'success')}`}
                disabled={!lootFinder}
                style={{ padding: '4px 8px', fontSize: '0.55rem', opacity: !lootFinder ? 0.4 : 1, borderColor: isGold ? 'var(--gold)' : '' }}
                onClick={() => {
                  const p = players.find(x => x.id === lootFinder);
                  if(!p) return;

```

```

    if(isHazard) {
      if(!window.confirm(`Trigger ${l.item} on ${p.charname}?`)) return;
      addLog({ type: "consequence", who: p.charname, what: `Encountered: ${l.item} in ${b.name}` });
      alert(`Event triggered! Update ${p.charname}'s status accordingly.`);
    } else if (isGold) {
      if(!window.confirm(`Loot ${l.item} for ${p.charname}?`)) return;
      addLog({ type: "item", who: p.charname, what: `Looted ${l.item} from ${b.name}` });
      alert(`Coins looted!`);
    } else {
      if(!window.confirm(`Grant ${l.item} to ${p.charname}?`)) return;

      let itemRarity = "Common";
      if (l.item.includes("Mythic") || l.item.includes("Artifact")) itemRarity = "Mythic";
      else if (l.item.includes("Legendary")) itemRarity = "Legendary";
      else if (l.item.includes("Rare")) itemRarity = "Rare";
      else if (l.item.includes("Uncommon")) itemRarity = "Uncommon";

      const newItem = {
        name: l.item,
        type: l.item.toLowerCase().includes("weapon") || l.item.toLowerCase().includes("blade") ? "weapon" : "consumable",
        rarity: itemRarity,
        condition: "Pristine",
        effect: `Scavenged from ${b.name}`
      };

      updatePlayer(p.id, { inventory: [...(p.inventory || []), newItem] });
      addLog({ type: "item", who: p.charname, what: `Scavenged ${newItem.name} from ${b.name}` });
    }
    setLootFinder("");
  }}
>
{isHazard ? 'Trigger Event' : (isGold ? 'Loot Coins' : 'Grant Item')}
</button>
</div>
);
}}
</div>
</div>
)}}
</div>
</div>
))
</div>
</div>
))

```

```

/* ——— PLAYER DETAIL MODAL ——— */

const PlayerModal = () => {
  const p = players.find(x => x.id === playerModalId);
  if (!p || !playerModalId) return null;
  const stats = getPlayerStats(p);
  return (
    <div className="modal-overlay open" onClick={e=>{if (e.target===e.currentTarget)setPlayerModalId(null);}}>
      <div className="modal-box" style={{maxWidth:620}}>
        <div style={{display:"flex",justifyContent:"space-between",alignItems:"center",marginBottom:16}}>
          <div style={{fontFamily:"Cinzel Decorative",serif,fontSize:"0.9rem",color:"var(--ember-glow)}}>{CLASS_ICON_EMO[p.cls]} {p.charname}</div>
          <button className="rune-btn danger sm" onClick={()=>setPlayerModalId(null)}>x</button>
        </div>
        <div style={{display:"grid",gridTemplateColumns:"1fr 1fr",gap:8,marginBottom:14}}>
          {[[["Race",p.race||"—"],["Class",p.cls],["Origin",p.origin||"—"],["Joined",p.joined||"—"],["Trait",p.traitLevel||"—"],["Title",p.title||"—"]].map(([[k,v]]=>{
            <div key={k} style={{background:"rgba(0,0,0,0.3)",borderRadius:3,padding:"6px 10px"}}>
              <div style={{fontFamily:"Cinzel",serif,fontSize:"0.48rem",color:"var(--ember)",letterSpacing:"1px"}}>{k}</div>
              <div style={{fontFamily:"IM Fell English",serif,fontSize:"0.72rem",color:"var(--parch)"}>{v}</div>
            </div>
          ))}
        </div>
        <div className="stat-grid" style={{marginBottom:14}}>
          {Object.entries(stats).map(([s,v])=><div key={s} className="stat-cell"><div className="stat-label" style={{color:STAT_COLORS[s]}}>{s}</div><div className="stat-val" style={{color:STAT_COLORS[s]}}>{v}{s===PRIMARY_STAT[p.cls]?"★":"."}</div></div>)}
        </div>
        {p.signatureSkill && <div className="info-bar gold" style={{marginBottom:8}}><b>Signature Skill:</b> {p.signatureSkill} — {p.signatureSkillDesc}</div>
        {p.shadowTrait && <div className="info-bar danger" style={{marginBottom:8}}><b>Shadow Trait:</b> {p.shadowTrait} — {p.shadowTraitDesc}</div>
        {p.destinyThread && <div className="info-bar" style={{marginBottom:8;borderColor:"rgba(156,39,176,0.4)",background:"rgba(156,39,176,0.06)}}>
          <b>Destiny Thread:</b> {p.destinyThread}</div>
        {p.repPassive && <div className="info-bar" style={{marginBottom:8}}><b>Reputation Passive:</b> {p.repPassive}</div>
        {p.notes && <div style={{fontFamily:"IM Fell English",serif,fontStyle:"italic",fontSize:"0.7rem",color:"var(--parch-shadow)",marginBottom:12}}>{p.notes}
        </div>
        <PlayerLoreTab player={p} setPlayers={setPlayers} />
        <div className="rune-divider"><span>Inventory</span></div>
        {(!p.inventory||p.inventory.length===0) && <div className="empty-state" style={{padding:12}}>No items.</div>
        {(p.inventory||[]).map((item,i)=>{
          <div key={i} style={{display:"flex",alignItems:"center",gap:8,padding:"6px 0",borderBottom:"1px solid rgba(200,129,58,0.06)"}>
            <span style={{fontFamily:"IM Fell English",serif,fontSize:"0.72rem",color:item.equipped?"var(--ember-glow)":"var(--parch)",flex:1}}>{item.name}</span>
            {item.rarity && <span className="rarity-badger" style={{color:RARITY_COLOR[item.rarity]||"#aaa",border:"1px solid ${RARITY_COLOR[item.rarity]||"#aaa"}30`}}>{item.rarity}</span>
            {item.condition && <span style={{fontFamily:"Cinzel",serif,fontSize:"0.48rem",color:"var(--parch-shadow)"}>{item.condition}</span>
            <button className="rune-btn sm" style={{padding:"2px 6px",fontSize:"0.48rem"}} onClick={()=>toggleEquip(p.id,i)}>{item.equipped?"⚡ Eq":"Equip"}
            </button>
            <button className="rune-btn danger sm" style={{padding:"2px 6px"}} onClick={()=>rmlItem(p.id,i)}>x</button>
          </div>
        )}
      )}
    </div>
  )
}

```

```

<div className="rune-divider"><span>Add Item</span></div>

<div style={{display:"grid",gridTemplateColumns:"2fr 1fr 1fr",gap:6,marginBottom:8}}>

<input className="scroll-input" placeholder="Item name" value={addItemForm.name} onChange={e=>setAddItemForm(f=>({...f,name:e.target.value}))}
onKeyDown={e=>{if(e.key==="Enter")addItem(p.id);}} />

<select className="scroll-select" value={addItemForm.rarity} onChange={e=>setAddItemForm(f=>({...f,rarity:e.target.value}))}>

{Object.keys(RARITY_COLOR).map(r=><option key={r} value={r}>{r}</option>)}

</select>

<select className="scroll-select" value={addItemForm.condition} onChange={e=>setAddItemForm(f=>({...f,condition:e.target.value}))}>

{["Pristine","Worn","Damaged","Broken"].map(c=><option key={c} value={c}>{c}</option>)}

</select>

</div>

<button className="rune-btn sm full" onClick={()=>addItem(p.id)}>+ Add Item</button>

<div className="rune-divider"><span>Consequences</span></div>

<div style={{display:"flex",gap:6,flexWrap:"wrap",marginBottom:10}}>

[[["curse","Cursed","tag-curse"],["wanted","Wanted","tag-wanted"],["exile","Exiled","tag-exile"],["servant","Servant","tag-
servant"]].map(([type,label,cls])=>{

<button key={type} className="rune-btn sm" style={{fontSize:"0.5rem"}} onClick={()=>addConsequence(p.id,type,label)}>{label}</button>

)}}

</div>

{(p.consequences||[]).map((c,i)=><div key={i} style={{display:"flex",alignItems:"center",gap:8,marginBottom:4}}><span className={`tag tag-${c.type}`}>
{c.label}</span><button className="rune-btn danger sm" style={{padding:"1px 5px"}} onClick={()=>rmConsequence(p.id,i)}>x</button></div>)}

<div style={{display:"flex",gap:8,marginTop:14,flexWrap:"wrap"}}>

<button className="rune-btn sm" onClick={()=>{updatePlayer(p.id,{giftPowerUsed:!p.giftPowerUsed});}}>{p.giftPowerUsed?"🔒 Gift Used":"🌟 Use
Gift Power"}</button>

<button className="rune-btn sm" onClick={()=>{if(!p.companion){const type=Object.keys(COMPANION_TYPES)
[0];assignCompanion(p.id,type);}else{updatePlayer(p.id,{companion:null});addLog({type:"note",who:p.charname,what:"Companion dismissed."});}}>
{p.companion?"Release Companion":"Assign Companion"}</button>

<button className={`rune-btn sm${p.protectionBalance>0?" success":""}`} onClick={()=>updatePlayer(p.id,{protectionBalance:p.protectionBalance>0?
0:1})}>{p.protectionBalance>0?"🛡️ Protected":"Add Protection"}</button>

</div>

</div>

</div>

);

};

/* — NAV CONFIG — */

const NAV = [

{sec:"Play",items:[

{id:"players",label:"The Roster",icon:"🗡️"},

{id:"backpack",label:"Rucksack",icon:"🎒"},

{id:"dice",label:"Oracle's Dice",icon:"🎲"},

{id:"combat",label:"Box Battle",icon:"⚔️"},

{id:"log",label:"Session Log",icon:"📖"},

]},

{sec:"World",items:[

{id:"map",label:"World Map",icon:"🗺️"},

{id:"villains",label:"Villains",icon:"🔴"},

]}

]

```

```

{id:"hq",label:"Moon Squad HQ",icon:"🏠"},
  ]},
  {sec:"Reference",items:[
    {id:"compendium",label:"Compendium",icon:"📖"},
  ]},
];

const currentPanel = () => {
  switch(tab) {
    case "players": return <PlayersPanel />;
    case "backpack": return <MobileBackpack players={players} updatePlayer={updatePlayer} addLog={addLog} toggleEquip={toggleEquip} rmltem={rmltem} />; //
    case "map": return <MapPanel />;
    case "dice": return <DicePanel />;
    case "log": return <LogPanel />;
    case "combat": return <CombatPanel />;
    case "villains": return <VillainsPanel />;
    case "hq": return <HQPanel players={players} updatePlayer={updatePlayer} addLog={addLog} />;
    case "compendium": return <CompendiumPanel />;
    default: return <PlayersPanel />;
  }
};

const NavItems = () => (
  <>
    {NAV.map(section => (
      <div key={section.sec} className="nav-section">
        <div className="nav-section-label">{section.sec}</div>
        {section.items.map(item => (
          <div key={item.id} className={`nav-item${tab===item.id?" active":""}` } onClick={()=>{setTab(item.id);setDrawerOpen(false);}}>
            <span className="nav-icon">{item.icon}</span>
            {item.label}
          </div>
        ))}
        <div className="nav-divider" />
      </div>
    ))}
    <div style={{padding:"10px 16px",marginTop:"auto"}}>
      <div style={{fontFamily:"'Cinzel',serif",fontSize:"0.42rem",color:"var(--parch-shadow)",letterSpacing:"2px",textAlign:"center",opacity:0.5}}>THE HOLLOW REALM v2.0<br/>SANDY SANDALS</div>
    </div>
  </>
);

return (

```

```

<>

<style>{CSS}</style>

<div id="hollow-app">

  {/* Desktop sidenav */}

  <div className="sidenav">

    <div className="sidenav-logo">

      <h1>● THE HOLLOW REALM</h1>

      <div className="subtitle">GM SUITE — SEASON II</div>

      <div className="session-pip">

        <div className={`session-dot${sessionActive?"":" off"}`}>

          {sessionActive ? `SESSION ${sessionNumber} LIVE` : "OFFLINE"}

        </div>

      </div>

      <div style={{flex:1,overflowY:"auto",overflowX:"hidden"}}>

        <NavItems />

      </div>

    </div>

    ...

    {/* Main content */}

    <div className="main-content">

      {/* Mobile topbar */}

      <div className="topbar">

        <div className="topbar-title">● HOLLOW REALM</div>

        <button className="topbar-menu-btn" onClick={()=>setDrawerOpen(true)}>≡ Menu</button>

      </div>

      {/* Panel area */}

      <div className="panel-area">

        {currentPanel()}

      </div>

      {/* Mobile bottom nav */}

      <div className="bottom-nav">

        <div className="bottom-nav-inner">

          {NAV.flatMap(s=>s.items).map(item=>(

            <div key={item.id} className={`bnav-item${tab===item.id?" active":""}` } onClick={()=>setTab(item.id)}>

              <span className="bnav-icon">{item.icon}</span>

              {item.label}

            </div>

          ))}

        </div>

      </div>

    </div>

  </div>

```

```

    { /* Mobile drawer */ }

    <div className={ ` drawer-overlay${drawerOpen?" open":""}` } onClick={()=>setDrawerOpen(false)} />

    <div className={ ` drawer${drawerOpen?" open":""}` }>

        <div style={{padding:"16px",borderBottom:"1px solid var(--ember)",fontFamily:"'Cinzel Decorative',serif",fontSize:"0.7rem",color:"var(--ember-glow)",letterSpacing:"2px"}}>● THE HOLLOW REALM</div>

        <NavItems />

    </div>

    { /* Modals */ }

    <AddPlayerModal />

    <PlayerModal />

    { /* Add item quick modal */ }

    {addItemTarget && () => {

        const p = players.find(x=>x.id===addItemTarget);

        return p ? (

            <div className="modal-overlay open" onClick={e=>{if(e.target===e.currentTarget)setAddItemTarget(null);}}>

                <div className="modal-box" style={{maxWidth:400}}>

                    <div style={{display:"flex",justifyContent:"space-between",alignItems:"center",marginBottom:16}}>

                        <div style={{fontFamily:"'Cinzel',serif",fontSize:"0.72rem",color:"var(--ember-glow)}}>+ Add Item to {p.charname}</div>

                        <button className="rune-btn danger sm" onClick={()=>setAddItemTarget(null)}>x</button>

                    </div>

                    <label className="form-label">Item Name</label>

                    <input className="scroll-input" value={addItemForm.name} onChange={e=>setAddItemForm(f=>({...f,name:e.target.value}))} placeholder="Item name..." style={{marginBottom:8}} />

                    <div style={{display:"grid",gridTemplateColumns:"1fr 1fr",gap:8,marginBottom:8}}>

                        <div><label className="form-label">Rarity</label><select className="scroll-select" value={addItemForm.rarity} onChange={e=>setAddItemForm(f=>({...f,rarity:e.target.value}))}>{Object.keys(RARITY_COLOR).map(r=><option key={r} value={r}>{r}</option>)}</select></div>

                        <div><label className="form-label">Condition</label><select className="scroll-select" value={addItemForm.condition} onChange={e=>setAddItemForm(f=>({...f,condition:e.target.value}))}>{["Pristine","Worn","Damaged","Broken"].map(c=><option key={c} value={c}>{c}</option>)}</select></div>

                    </div>

                    <button className="rune-btn full" onClick={()=>addItem(addItemTarget)}>+ Add Item</button>

                </div>

            </div>

        ) : null;

    })() }

</div>

</>
...

);

}

```